DTU Electro
Department of Electrical and Photonics Engineering

Design of an End-to-End IoT System for Monitoring Vulnerable Users

Developing a Battery-Efficient IoT Prototype for Fall Detection and Location Tracking

Master's Thesis - Report

Andrei Farcas(242999)
Tomás Venda(s243028)

July, 2026

Design of an End-to-End IoT System for Monitoring Vulnerable Users
Developing a Battery-Efficient IoT Prototype for Fall Detection and Location Tracking

Master's Thesis - Report
February 9 - July 30, 2026

By
Andrei Farcas, Tomás Venda

Copyright: Reproduction of this publication in whole or in part must include the customary bibliographic citation, including author attribution, report title, etc.
Published by: DTU, Department of Electrical and Photonics Engineering, Ørsteds Plads, Building 343, 2800 Kgs. Lyngby Denmark
www.electro.dtu.dk

Abstract

This thesis presents the design, implementation, and evaluation of a battery-efficient IoT wearable bracelet for monitoring vulnerable users. The proposed system combines on-device machine-learning-based fall detection, tiered localization (Wi-Fi, A-GNSS, and cellular fallback), and LTE-M connectivity to provide real-time tracking and emergency alerts through a caregiver mobile application. The prototype integrates a custom six-layer PCB built around the nRF9151 SiP, embedded firmware, a Flutter mobile application, and a cloud backend. Experimental evaluation demonstrates a fall detection accuracy of 98.4%, emergency alert delivery times below 15 seconds, reliable localization in both indoor and outdoor environments, and a battery life exceeding one week under typical operating conditions.

Acknowledgements

We would like to sincerely thank our thesis supervisors for their invaluable guidance, support, and feedback throughout this project.

Martin Nordal Petersen
Associate Professor, DTU Department of Electrical and Photonics Engineering

Anas Mohamad Al Shalyan
Research Assistant, DTU Department of Electrical and Photonics Engineering

List of Acronyms

Acronym	Definition
API	Application Programming Interface
AoA	Angle of Arrival
BLE	Bluetooth Low Energy
CNN	Convolutional Neural Network
CoAP	Constrained Application Protocol
FCM	Firebase Cloud Messaging
FIFO	First-In, First-Out
FSM	Finite State Machine
GNSS	Global Navigation Satellite System
IMU	Inertial Measurement Unit
IoT	Internet of Things
JSON	JavaScript Object Notation
LDO	Low-Dropout Regulator
LPWAN	Low-Power Wide-Area Network
LTE-M	Long Term Evolution for Machines
MEMS	Micro-Electromechanical Systems
MQTT	Message Queuing Telemetry Transport
PCB	Printed Circuit Board
PMIC	Power Management Integrated Circuit
SiP	System-in-Package
SVM	Support Vector Machine
ToF	Time of Flight
UWB	Ultra-Wide Band

List of Symbols

Symbol	Description	Unit
c	Speed of light in a vacuum	m/s
d	Calculated distance between nodes	m
$t_{arrival}$	Timestamp of signal arrival	s
a	Linear acceleration	m/s^2
ω	Angular velocity	$^\circ/s$
x	Displacement of a MEMS proof mass	m
k	Spring constant	N/m
C	Capacitance	F
ϵ	Permittivity of the dielectric medium	F/m
n	Path loss exponent	-
μ	Arithmetic mean of a dataset	-
σ	Standard deviation of a dataset	-
x	Input vector of sensor measurements	-
y	Classification output label	-

Contents

1 Introduction **1**
- 1.1 Background . . . 1
- 1.2 State-of-the-art / Related Work . . . 3
- 1.3 Formulation of the problem . . . 4
- 1.4 System requirements and design constraints . . . 5

2 Theoretical Overview **9**
- 2.1 IoT in Healthcare . . . 9
- 2.2 Localization technologies for wearable safety devices . . . 10
- 2.3 Connectivity Technologies for Wearable Devices . . . 14
- 2.4 Embedded sensing and edge intelligence for fall detection . . . 16
- 2.5 Low-Power Design in GNSS and Cellular Wearables . . . 19
- 2.6 Cloud backends and mobile applications for remote monitoring systems . . . 20

3 System Design **25**
- 3.1 System Overview . . . 25
- 3.2 Hardware Design Choices . . . 26
- 3.3 Software and Communication . . . 33

4 Hardware Implementation **39**
- 4.1 System Architecture Overview . . . 39
- 4.2 Power Management Unit . . . 40
- 4.3 RF and Antenna Design . . . 41
- 4.4 Digital Interfaces and Peripherals . . . 42
- 4.5 PCB Layout, Routing and Manufacturing . . . 43
- 4.6 Hand soldered components and adjustments . . . 45
- 4.7 Case Design . . . 47

5 Software and Algorithm Implementation **49**
- 5.1 Embedded Software Development . . . 49
- 5.2 Cloud Server Backend . . . 54
- 5.3 Caregiver's Mobile Application . . . 59

6 System Evaluation and Testing **67**
- 6.1 Machine Learning and Threshold Model Comparison . . . 67
- 6.2 Power Consumption Profiling . . . 68
- 6.3 Hardware Bring-up Challenges . . . 73

7 Conclusion **75**
- 7.1 Requirements Satisfaction . . . 75
- 7.2 Challenges and Limitations . . . 75
- 7.3 Future Work . . . 76

Bibliography **79**

A Impedance Calculations **83**

B Bill of Materials **84**

List of Figures

1.1 Garmin Forerunner 201: the world's first all-in-one GPS-enabled watch . . 1
1.2 Evolution of mainstream health wearables. 2
1.3 Medical Guardian MGMini Lite 3
1.4 Global 5G SA deployments 4

2.1 Typical IoT architecture involving wearable devices, gateways, and a cloud platform [13]. 9
2.2 LoRa connectivity to the network 11
2.3 Localization using WiFi fingerprinting [27] 14
2.4 Operating principle of a MEMS accelerometer [33]. 16
2.5 Low Power Modes for Cellular IoT: PSM and eDRX 20
2.6 Comparison between possible network topologies. 21
2.7 Data flow example in a possible MQTT implementation 22

3.1 System architecture 25
3.2 State machine governing the bracelet's operating modes. 35

4.1 Hardware Architecture Block Diagram 39
4.2 Standard Li-ion charging profile executed by the nPM1300 PMIC (Adapted from Nordic Semiconductor) 41
4.3 GNSS antenna matching network design. 41
4.4 WiFi antenna matching network design. 42
4.5 Top Side of the PCB 44
4.6 Bottom Side of the PCB 44
4.7 The manual fix required for the Buck Boost Converter 45
4.8 GNSS quarter-wave monopole antenna 46
4.9 Case 3D model and manufactured result 48

5.1 Accelerometer and pressure delta behavior during a fall event 52
5.2 Spectral analysis performance on training data 53
5.3 Overview of the backend components and the communication interfaces. . 56
5.4 The main dashboard interface of the Caregiver Application. 60
5.5 High-level architectural block diagram of the Flutter Application 62
5.6 The Bracelet Details Screen of the Caregiver Application. 63

6.1 ML vs Threshold power consumption 69
6.2 Cellular modem on/off vs PSM 71
6.3 Power consumption during various states 71

A.1 50 Ω coplanar waveguide impedance calculation for Layer 2 83
A.2 50 Ω coplanar waveguide impedance calculation for Layer 4 83

1 Introduction

1.1 Background

The rapid demographic shift toward an aging population has placed unprecedented pressure on global healthcare infrastructures. For elderly individuals living in isolation, the lack of prompt assistance during emergencies is a critical concern. Internet of Things (IoT) and wearable technologies have emerged as a robust solution, providing an autonomous safety net that reduces healthcare expenditures while improving quality of life [1, 2].

The clinical necessity is stark: over one-third of home-dwelling seniors fall annually, and two-thirds of those are prone to recurrent incidents [3]. Beyond physical injury, post-fall syndrome often leads to decreased mobility and a loss of independence. To mitigate this, engineering efforts prioritize discrete design and simplicity to ensure high user compliance and reduce the social stigma associated with medical assistive devices [2, 4].

Evolutionary Milestones

The history of Telecare began in the 1970s with Personal Emergency Response Systems (PERS), which utilized radio-frequency (RF) communication between a pendant and a landline base station [5]. These were entirely reactive and the user had to be conscious to manually trigger an alert. Furthermore, they were functionally limited to the home, leaving seniors unprotected the moment they stepped outside.

The shift toward mobile autonomy arrived with the Garmin Forerunner (2003), but it was the era of passive monitoring between 2006 and 2012 that truly addressed the needs of the elderly. Devices like the Nike+ iPod and Jawbone UP transitioned the industry toward autonomous data collection, replacing stigmatized panic buttons with sleek forms that tracked activity without user intervention. This was a psychological breakthrough: a senior was no longer wearing a disability tracker, but a modern health device. However, this era faced a massive battery barrier. Early GPS-enabled watches were power-hungry, often requiring daily charging, a significant risk for seniors with cognitive decline who might forget to plug in the device, rendering the safety net useless during a wandering event.



Figure 1.1: Garmin Forerunner 201: the world's first all-in-one GPS-enabled watch

By 2015, the Apple Watch and Fitbit's clinical pivot introduced multi-constellation localization, utilizing GLONASS and Galileo alongside GPS. This achieved the sub-meter accuracy required to locate a user in dense urban environments. To solve the dangerously slow

Time to First Fix (TTFF) of earlier devices, Assisted GPS (A-GPS) began using cellular tower data to pre-calculate satellite positions, ensuring emergency teams receive precise locations within seconds. For the indoor blind spot where most falls occur, modern devices now leverage Wi-Fi Fingerprinting and Bluetooth beaconing, creating a continuous safety net that follows the user from the living room to the street.



(a) First Fitbit Tracker (2009)



(b) First Apple Watch (2015)

Figure 1.2: Evolution of mainstream health wearables.

Sensor Development and Fall Detection

Automated fall detection evolved from 1990s mechanical sensors to modern micro electromechanical systems (MEMS) technology. A 1998 prototype by Williams et al. [6] used a piezoelectric shock sensor and a mercury tilt switch to detect if a user was horizontal. However, this binary logic struggled to distinguish between a medical emergency and a user sitting down quickly, leading to alarm fatigue for caregivers.

In 2001, Digital Angel pioneered the integration of GPS with thermistors to monitor body temperature. However, the hardware was too bulky for frail users and the battery life was depleted in hours. The true breaktrhough arrived with the Inertial Measurement Unit (IMU). Moving beyond 3-axis accelerometers, the integration of 3-axis gyroscopes created 6-axis systems capable of measuring both linear impact and angular velocity. By incorporating a magnetometer, modern 9-axis IMUs now track a wearer's orientation in 3D space with great precision.

Following Kang et al. (2006) [7], wearables became comprehensive diagnostic tools. Photoplethysmography (PPG) was introduced to measure heart rate and SpO_2 via LED light, while Barometric Altimeters began detecting minute changes in atmospheric pressure. This allows the device to recognize a sudden loss of altitude, such as a fall down stairs, which a standard accelerometer might miss. By 2026, this sensor fusion allows the wearable to identify the subtle physiological declines that often precede an accident.

Evolution of Communication

The final hurdle in geriatric IoT was achieving wide-area independence without a smartphone. Early wearables relied on the phone as a gateway, but this was a critical failure point for dementia care; if a user left their phone behind, they were effectively invisible to their caregivers. The release of Bluetooth Low Energy (BLE) and Zigbee provided short-range efficiency, but the real breakthrough for wandering patients came with Low Power Wide Area Networks (LPWAN).

Technologies like NB-IoT and LTE-M allowed wearables to connect directly to cellular towers over long distances with minimal energy drain, significantly extending battery life. In

2024, the launch of 5G RedCap (Reduced Capability) offered high speed and low latency but it introduced a significant disadvantage: dependency on 5G Standalone (SA) network infrastructure. For geriatric applications, this lack of backward compatibility with legacy 4G networks limits its deployment in rural or underserved areas where seniors often live. Consequently, protocols like NB-IoT remain more viable for wide-area coverage due to their deeper penetration and established global infrastructure.

1.2 State-of-the-art / Related Work

The current landscape of wearable assistive technology in 2026 is defined by a divergence between high-end multi-functional smartwatches and specialized devices. While mainstream consumer devices like recent Apple Watch and Samsung Galaxy iterations have integrated sophisticated sensor fusion for fall detection, they remain hampered by interface complexity that can alienate elderly users [8, 9]. To address this, current research and industry efforts have focused on three critical pillars: sensing intelligence, power autonomy, and adaptive localization.

Sensing Intelligence and Edge AI

Fall detection has moved toward Edge AI to solve the historic problem of ”alarm fatigue” caused by false positives. Modern prototypes increasingly leverage TinyML to process data directly on low-power microcontrollers, such as the ARM Cortex-M55, which features dedicated micro-Neural Procssing Units. This allows for real-time impact analysis without the latency or privacy concerns of cloud-based processing. This architecture combined with hardware interrupts allow the main processing unit to remain in a deep sleep state, drawing negligible current, until an event is detected, further reducing power consumption.

Specialized medical-grade solutions, such as the Medical Guardian MGMini Lite, have achieved high accuracy by focusing on wrist-worn kinematics, though academic research is currently pushing toward multi-modal sensing—combining Inertial Measurement Unit (IMU) data with barometric pressure to better distinguish between vertical falls and controlled movements like sitting [10]. Although this device possesses a minimal form factor while achieving great reliability in emergency events, its battery life constitutes a serious problem for its target audience.



Key Characteristics:

Dimensions: 38.1 x 35.6 x 12.7 mm
Weight: 19.8g
Battery Life: Up to 36 hours
Cost: 149.95 upfront + 44.95/mo (USD)
Main Features: 4G LTE, 24/7 emergency button, WiFi location tracking, water resistant

Figure 1.3: Medical Guardian MGMini Lite

The Battery-Connectivity Trade-off

A critical vulnerability in 2026 remains the 24-hour battery cycle common in high-performance devices, which poses a significant risk for users with cognitive decline who may neglect

daily charging [10]. Continuous monitoring of both location and sensor activity introduces a great challenge in wearable devices which relies on precision trade-offs in order to be solved. The industry has responded with the finalization of 5G RedCap (Reduced Capability). This standard acts as a middle ground, providing 5G-level reliability with the power profile of older LTE-M modules [11].

However, a notable disadvantage of 5G RedCap is its strict dependency on 5G Standalone (SA) network infrastructure. Unlike LTE-M or NB-IoT, which benefit from nearly universal global coverage, RedCap's lack of backward compatibility with legacy 4G networks limits its effectiveness in rural or underserved regions (Figure 1.4).

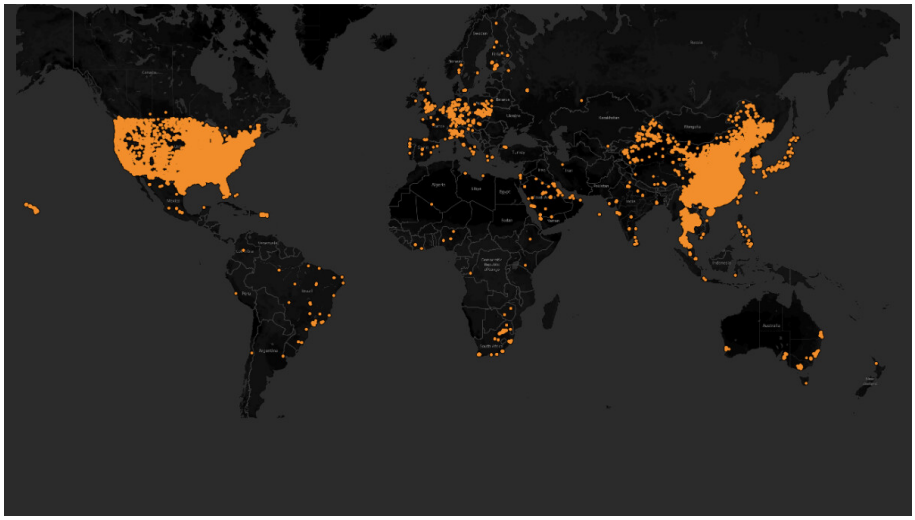


Figure 1.4: Global 5G SA deployments

Adaptive Geofencing and Behavioral Analysis

Geofencing becomes particularly relevant for elderly wearable devices as it allows a caregiver to set a virtual physical boundary for the user's location and activate automated actions such as push notifications. Thus, localization research has shifted from standard circular geofencing towards adaptive, context-aware boundaries. Rather than triggering an alert based on a simple distance threshold, modern frameworks utilize historical user behavior and environmental data to identify anomalies. By learning a user's specific "Life Space," these systems can distinguish between a routine walk and a confused wandering event. This direction, seen in recent high-end assistive projects, aims to transform the wearable from a reactive alarm into a proactive guardian that can predict potential incidents before they occur [10, 12].

1.3 Formulation of the problem

Despite the rapid growth of consumer wearables, a significant gap remains between high-end multi-purpose devices and the specialized needs of the geriatric population. Current market solutions often fail due to a several barriers: prohibitive costs, excessive UI complexity, and the social stigma associated with bulky medical trackers. Furthermore, the high power consumption of always-on, high-resolution displays, active tracking and frequent data polling results in poor battery longevity, which undermines the reliability required for continuous emergency monitoring. This research addresses the critical need for a low-cost, specialized IoT architecture that prioritizes passive monitoring, specifically ad-

dressing the risks of wandering and accidental falls, while maintaining a non-stigmatizing form factor with a simplified, low-power display for time and essential feedback only.

The primary objective of this thesis is to develop and evaluate a standalone wearable prototype optimized for the frail and cognitively impaired. The technical focus lies in balancing power efficiency with localization accuracy by exploring alternative methodologies for primary geofencing, such as low-power Wi-Fi scanning and cellular tower data. Power-intensive GNSS tracking will be implemented as a fallback mechanism, reserved for when the wearer is outdoors in more remote areas or during active emergency. Moreover, a seamless screen will keep the design discrete by showing local time. This screen will also meet this population segment's requirements since in most cases it is an indispensable feature..

To ensure a robust safety net, the device integrates an Inertial Measurement Unit (IMU) and a barometer for automated fall detection, supplemented by a manual SOS trigger for conscious emergencies. This ecosystem is completed by a server-mediated communication architecture, enabling real-time data transmission to a dedicated caretaker application. By streamlining the hardware to essential life-safety functions, this project aims to provide a durable, accessible, and dignified solution for long-term elderly care.

1.4 System requirements and design constraints

The proposed bracelet targets vulnerable users such as older adults or patients at elevated risks of falls and disorientation. As already established in the introduction, such populations require continuous awareness of the localization, reliable fall detection and timely alerts to the caretaker, all without dependency on a nearby smartphone or special infrastructure in the environment. In addition, the caregivers need a reliable and user friendly software interface, in the form of a mobile application, to receive alerts, view available information about the wearer status and location and manage device settings.

At a high architectural level the bracelet can be decomposed into four tightly coupled subsystems. The following subsections describe the role and main functionality of each subsystem, and formulate key requirements that guide the upcoming design process.

1.4.1 Sensing and decision subsystem (on-device)

The sensing and decision subsystem acquires raw physiological and kinematic data and converts it into events such as *fall detected*, *emergency detected* or *normal activity*.

Formally, this subsystem must satisfy the requirements in Table 1.1.

1.4.2 Localization subsystem (on-device)

The localization subsystem estimates the geographic position of the wearer so that the caregiver can respond appropriately to potential alarms and emergencies. The bracelet must operate across both urban and rural environments and without reliance to the proximity of a smartphone or an existing network of pre-installed beacons.

The key requirements of this subsystem can be seen in Table 1.2.

1.4.3 Connectivity subsystem (on-device)

The connectivity subsystem is responsible for delivering alarms and periodic status reports to from the bracelet to the backend. As the wearer may sometimes move over large

distances, the subsystem must support wide-area mobility, timely delivery of emergency alerts, and energy-efficient communication compatible with multi-day battery life.

The formal requirements of the connectivity subsystem are summarized in Table 1.3.

Table 1.1: Sensing and decision subsystem requirements

ID	Requirement description
R1	The system shall detect at least 99% of fall events on representative test data (**false negative rate** $\leq$ **1%**).
R2	The system shall limit the false alarm rate to an average of maximum 1 false alert per 24 hours during normal daily activities. Upon triggering a false positive, the system must provide the wearer with a clear mechanism to cancel the emergency alert before it is transmitted to the caregiver.
R3	The fall detection algorithm shall execute entirely on the embedded microcontroller without continuous transmission of raw IMU data to external systems.

Table 1.2: Localization subsystem requirements

ID	Requirement description
R4	Under normal outdoor conditions, the system shall achieve a median horizontal localization error below **10 m**, and should aim for errors in the range of **3–5 m** when GNSS reception is favorable.
R5	In indoor or GNSS-denied conditions, the system shall provide a location estimate with an error not exceeding **500 m** in at least **90%** of cases, and shall indicate reduced accuracy to the caregiver.
R6	Localization shall be computed autonomously on the bracelet, without requiring a tethered smartphone or pre-installed local infrastructure, to ensure functionality in arbitrary environments.

1.4.4 Cloud and caregiver oriented subsystem (off-device)

The cloud and caregiver-oriented subsystem connects the wearable device to the end-user through a combination of backend services and client applications. The backend manages the secure ingestion of device messages, persistent data storage, essential business logic like geofence evaluation, and the routing of emergency notifications. This infrastructure seamlessly supports the caregiver-facing application, which translates the processed data into a clear, actionable interface, allowing non-technical users to monitor the wearer's real-time location, receive immediate alerts, and review historical events.

The formal requirements of this subsystem are summarized in Table 1.4.

1.4.5 Physical design and form factor constraints

In addition to functional performance, the physical design of the bracelet must ensure that it can be worn continuously in everyday life without causing discomfort or stigmatization. Since the intended users include older adults and vulnerable individuals, the device must be non-obtrusive, comfortable, and socially acceptable in appearance.

The physical design requirements are summarized in Table 1.5.

Table 1.3: Connectivity subsystem requirements

ID	Requirement description
R7	The system shall maintain wide-area network connectivity while the wearer moves across cellular coverage areas, including seamless handover between cells in LTE-M networks, without requiring user intervention.
R8	For critical events such as confirmed falls or SOS button activations, the system shall achieve an end-to-end latency of less than **30 s**, measured from event detection on the device to successful reception by the backend server under normal network conditions.
R9	Under typical usage conditions, including periodic status reporting, periodic localization updates, and event-driven alert transmission, the system shall achieve a minimum operational battery life of **7 days** on a single battery charge.

Table 1.4: Cloud and caregiver-oriented subsystem requirements

ID	Requirement description
R10	The system shall provide caregivers with access to the current status and most recent known location of the wearer, as well as a history of past events, through a user interface suitable for non-technical users.
R11	The system shall allow caregivers to configure essential system parameters, like emergency contact information, Geo-fencing restrictions and notification preferences, through the caregiver interface.
R12	The system shall ensure that access to device data and system functionality is restricted to authorized caregivers who have been explicitly paired with the device.

Table 1.5: Physical design and form factor requirements

ID	Requirement description
R13	The bracelet shall be designed for continuous wear and shall be comfortable to wear on the wrist throughout normal daily activities, including walking, sitting, and sleeping.
R14	The device shall have an unobtrusive and socially acceptable appearance, such that it does not stigmatize the wearer or draw unnecessary attention.
R15	The bracelet should provide one button with two functionalities. It allows the user to trigger an emergency notification and also provides the option of canceling a false positive.

Collectively, requirements R1–R15 define the primary functional and non-functional constraints that guide the design and implementation of the wearable device, communication infrastructure, and supporting software systems.

2 Theoretical Overview

This chapter aims to provide a holistic view of the most important techniques, standards, protocols and strategies involved in the development of a wrist worn bracelet with tracking and fall detection mechanisms. It covers current localization technologies both outdoors and indoors, short and long range. Furthermore, it explores communication technologies, embedded sensing, low-power design strategies and cloud backends.

2.1 IoT in Healthcare

IoT represents a technological evolution that connects physical objects, equipment, and sensors through the internet to enable continuous data sharing. While this interconnection transforms multiple industrial sectors, the healthcare industry benefits significantly from its adoption [13]. In this domain, connected technologies facilitate real time data collection and remote monitoring. These capabilities are essential for developing personalized treatment strategies and improving overall patient outcomes.

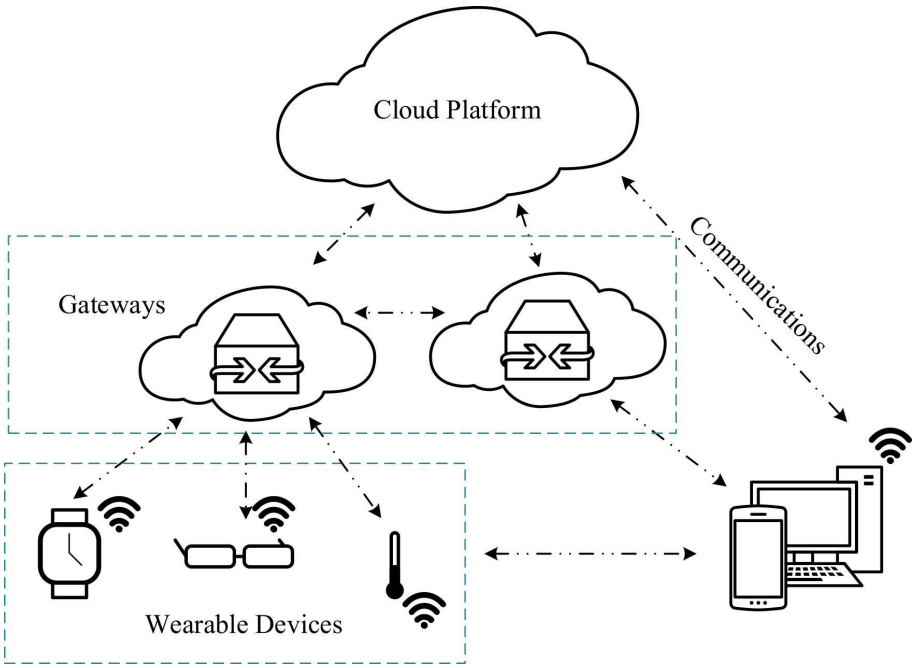


Figure 2.1: Typical IoT architecture involving wearable devices, gateways, and a cloud platform [13].

Integrating these connected devices in healthcare facilitates administrative operations and automates routine clinical tasks. Continuous monitoring is highly valuable for early anomaly detection, which can prevent severe medical complications and reduce the need for expensive hospital interventions [13]. However, strict data privacy and security requirements are crucial to protect sensitive patient information.

2.2 Localization technologies for wearable safety devices

2.2.1 Satellite-Based Navigation Systems (GNSS)

Satellite-based navigation systems are the most established technologies for tracking wearable safety devices in outdoor environments. While often referred to by the specific US system name, GPS, the broader field is defined as Global Navigation Satellite Systems (GNSS).

The Global Positioning System (GPS) was developed by the United States in 1978 for military use and made available to the public in 1994, using an array of 31 satellites [14]. These satellites are positioned 22,000 kilometers above the Earth in geosynchronous orbits, ensuring they appear at the same sky position after one sidereal day.

GNSS is a generic term for satellite navigation providing global geo-spatial positioning. It includes the US GPS (31 satellites), Russia's GLONASS (24), Europe's Galileo (30), China's Beidou (35), and Japan's QZSS (7) [15, 16]. This multi-constellation approach benefits from redundancy, which improves positioning accuracy and allows receivers to filter out weaker signals [15].

Each satellite constantly broadcasts its location and time with accuracy down to the picosecond. An IoT wearable captures signals from three or more satellites simultaneously and uses the differences to multilaterate its position, providing longitude, latitude, and altitude [14]. While GNSS is free to use and accurate to a few feet, it requires a clear view of the sky and broadcasts at 1.2 or 1.5 GHz, frequencies that struggle to penetrate roofs or walls. Most importantly for wearable design, continuous GNSS localization drains the battery quickly which is a major disadvantage for this type of localization technique [17, 15].

GNSS receivers calculate the distance (d) by determining how long a signal took to travel from the satellite to the antenna at the speed of light (c):

$$d = c \times (t_{arrival} - t_{departure}) \tag{2.1}$$

A minimum of four satellites is required for redundancy to solve for the clock alignment between the receiver and the satellites [14]. However, despite its precision, several error sources exist:

- **Receiver Error:** Electromagnetic interference from components within the wearable can affect signal quality.
- **Satellite Error:** Even precise atomic clocks have slight inaccuracies, and orbital positions can drift.
- **Atmospheric Error:** Signals are delayed as they pass through the atmosphere, where the speed of light is no longer a constant as it would be in a vacuum [14].
- **Environment Error:** Signal jamming or obstructions in urban areas with tall buildings can render the system useless [18].

To mitigate these, control stations provide real-time corrections via radio, satellite, or internet connections to account for these localized errors [16].

2.2.2 Long-Range and Wide-Area Networks (LPWAN)

Terrestrial long-range networks offer an alternative to satellite systems, particularly when indoor penetration or lower power consumption is required.

Cellular Network Positioning

Cellular localization uses the radio signal strength between the wearable and nearby cell towers to calculate a relative position. Each tower provides information on its own fixed location. While cellular systems work reasonably well indoors, they are generally poor at detecting elevation and, like GNSS, are very power hungry [17].

LoRaWAN and Chirp Spread Spectrum (CSS)

Long Range Wide Area Network (LoRaWAN) is an open MAC protocol built on the LoRa physical layer [19]. LoRa operates at low frequencies (e.g., 915 MHz in the US or 868 MHz in Europe) and provides long-range (up to 15 km), low bit-rate, and low-power communication [20].

Compared to GNSS, LoRa has lower power consumption, robust indoor penetration, and a larger coverage area. However, it requires a separate infrastructure; messages are received by a LoRa gateway and forwarded to the cloud rather than being sent directly to the internet [19, 20].

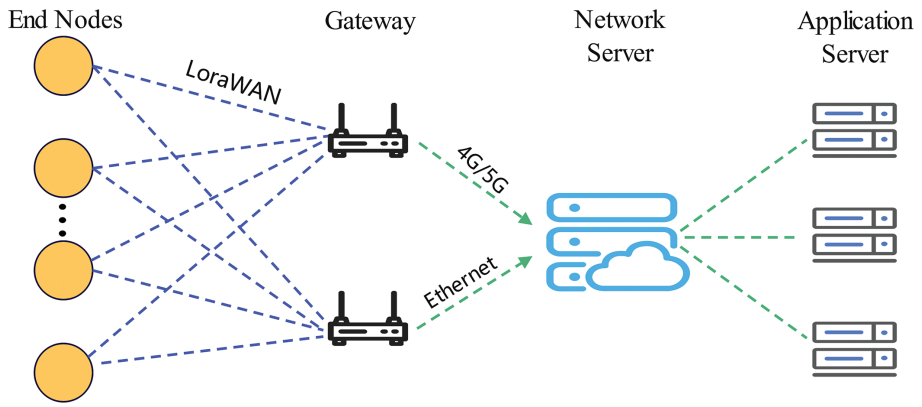


Figure 2.2: LoRa connectivity to the network

2.2.3 Short-Range Wireless Technologies

Short-range wireless technologies are the primary choice for indoor localization in wearable safety devices, as they leverage existing infrastructure and offer a balance between power efficiency and positioning accuracy.

Wi-Fi Positioning Systems (WPS)

Wi-Fi localization serves as a critical backup when GNSS signals are unavailable, particularly in dense urban or indoor environments. It functions as a geo-location system that scans nearby Wi-Fi Access Points (APs) without requiring a formal connection. This technique utilizes the widespread availability of APs, identifying them via their unique SSID and MAC address [14].

The primary driver of this method is the Received Signal Strength Indicator (RSSI). While traditional Wi-Fi had a range of approximately 100 meters, modern standards like IEEE 802.11ah, which operate at much lower frequencies (around 900MHz), have extended this to nearly 1 kilometer [14, 21]. High-accuracy positioning is achieved by looking up

AP identifiers in a database, which is often populated by correlating mobile device location data (from GPS or Galileo) with Wi-Fi MAC addresses [22].

The downside of the Wi-Fi positioning system is that it is limited to wireless infrastructure which is usually only available in urban areas. Also it depends on an external provider such as Apple Location Services, Skyhook Wireless, Qualcomm Terrestrial Positioning Service, or Google Geolocation API, the most extensive global database, which is crowd-sourced from Android devices and Google Street View scanning vehicles.

Bluetooth Low Energy (BLE) and Beacons

Introduced in 2010 by the Bluetooth Special Interest Group, BLE was designed for applications requiring low power consumption and cost rather than high data transfer rates. BLE devices communicate directly using low-power radio waves in the 2.4 GHz band (2400 MHz to 2483.5 MHz). It employs adaptive frequency hopping across 79 channels to maintain connectivity in interference-heavy environments [14].

BLE uses specialized devices known as beacons. These systems can utilize various methodologies, including RSSI, Angle of Arrival (AoA), and Time of Flight (ToF), to pinpoint a device. The communication is secured through authenticated access codes and frequency-hopping sequences known only to the communicating devices [14].

2.2.4 High-Precision and Proximity Technologies

For applications requiring either extreme hardware simplicity or centimeter-level accuracy, specialized radio frequency technologies such as RFID or Ultra-Wide Band (UWB) are employed.

Radio Frequency Identification (RFID)

An RFID system consists of a reader and one or more tags. The reader emits electromagnetic waves that activate the tag (for passive RFID) or communicate with an already powered tag over hundreds of meters (for active RFID). The tag responds by transmitting its stored identification information back to the reader, allowing the reader to identify the object or person carrying the tag[14].

In its simplest form, localization is based on reader proximity: the detected position is assumed to be the known location of the reader. When several readers cover overlapping areas, RSSI or phase measurements can improve the position estimate. Its accuracy is lower than technologies specifically designed for positioning, such as UWB, which reinforces its non-optimal usage for wearable tracking.

Ultra-Wide Band (UWB)

Ultra-Wide Band (UWB) is a high-precision positioning technology that transmits billions of ultra-short pulses over a very large bandwidth ($>$500 MHz) within the 3.1 to 10.6 GHz range [14]. Its pulse-based nature makes it highly resistant to interference and multipath effects (signals bouncing off surfaces)

UWB functions by calculating the Round Trip Time of Flight (ToF) of these pulses, enabling centimeter-level accuracy for digital keys and precise indoor navigation.. Furthermore, the low frequency of UWB pulses allows the signal to effectively penetrate obstacles like walls and furniture. While historically limited by slow standard development, UWB is increasingly becoming a standard feature in high-end portable user devices [14, 23].

Consequently, UWB is particularly suitable for applications in controlled indoor environments where the necessary anchor infrastructure can be deployed. In such settings, its

high positioning accuracy makes it an attractive solution for personnel tracking, asset management, and safety systems. However, the dependence on fixed infrastructure limits its applicability for wearable devices that are expected to provide localization across diverse and previously unknown environments.

2.2.5 Kinematic Estimation: Dead Reckoning

Dead reckoning is a localization strategy that estimates a user's current position by projecting their past known location based on measured speed, elapsed time, and direction. This method represents a solution for wearables in environments where external signals, such as GNSS or Wi-Fi, are unavailable.

Pedestrian Dead Reckoning (PDR)

For human-centric IoT, Pedestrian Dead Reckoning (PDR) is considered the standard approach. PDR focuses on human activity characteristics by detecting individual steps, estimating stride length, and determining heading [24].

However, the fundamental limitation of dead reckoning is the accumulation of systematic and stochastic errors, often described as a "random walk." Because each new position is calculated from the previous one, even minor sensor offsets propagate over time. For instance, a tiny acceleration error of just 0.01 m/s^2 can result in a position error of 9 meters after only 30 seconds [25]. Velocity errors grow proportionally with time, while position errors grow proportionally to the square of the elapsed time.

Nevertheless, the combination of this strategy with absolute localization, such as GNSS or WiFi, or the usage of deep learning models mitigate the error propagation. In fact, experimental settings testing pure PDR showed a Root Mean Squared Error (RMSE) of over 78 meters in a covered distance of 1,260 meters. Conversely, when fusing it with GNSS, the error dropped to 3-4 meters [26, 24].

2.2.6 Fundamental Localization Methodologies

The translation of physical signals into geographic coordinates relies on a set of core geometric and statistical methodologies. These techniques are broadly categorized into proximity, angulation, and lateration.

Proximity and Fingerprinting

The simplest form of localization is proximity, where a device is assumed to be located within the coverage area of a known reference point such as an access point, beacon, or RFID reader. Fingerprinting (or scene analysis) is a more advanced derivative of this, consisting of two phases:

- **Offline Phase:** An environmental survey is conducted to map "fingerprints" of RSSI or Channel State Information (CSI) at specific coordinates, stored in a database.
- **Online Phase:** The wearable compares its real-time signal readings against the database to find the closest match.

Matching algorithms typically include probabilistic methods (Bayes' theorem), Artificial Neural Networks (ANN), or k-Nearest Neighbor (kNN)[14].

Angulation: Angle of Arrival (AoA)

Angulation determines position by measuring the angle at which a signal strikes a receiver's antenna array. While highly precise at short ranges, AoA accuracy decreases

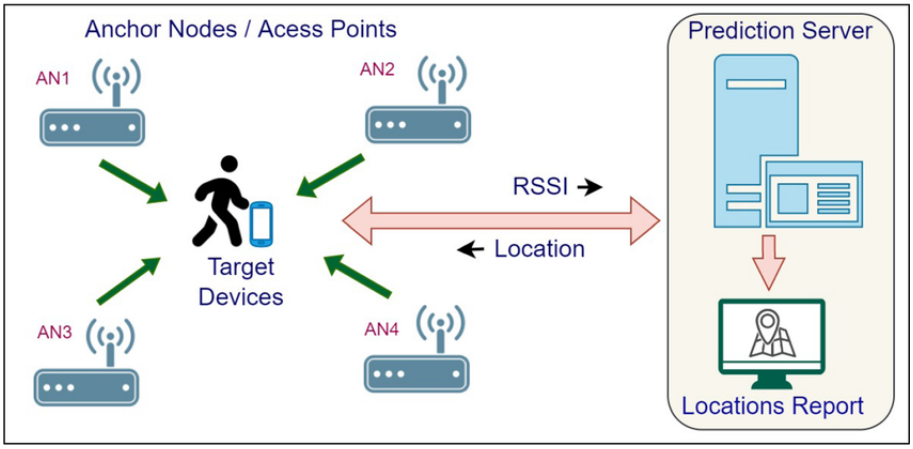


Figure 2.3: Localization using WiFi fingerprinting [27]

rapidly with distance, as minor angular errors result in significant coordinate offsets [14].

Lateration: RSSI and ToF

Lateration uses distance measurements from multiple reference points to calculate a position. The method for determining distance (d) varies by technology:

1. Received Signal Strength Indicator (RSSI):

RSSI measures the power of the incoming signal, typically in dBm. Using a propagation model, distance is derived from the signal strength by:

$$RSSI = -10n \log_{10}(d) + A \tag{2.2}$$

where n is the path loss exponent (ranging from 2 in free space to 4 indoors) and A is the RSSI at a reference distance. While cost-effective, RSSI is highly susceptible to multipath fading [21].

2. Time of Flight (ToF) and Return Time of Flight (RToF):

ToF measures the absolute travel time of a signal. Multiplying this time by the speed of light (c) provides the physical distance. RToF (or Round Trip Time) avoids the need for strict clock synchronization between the transmitter and receiver by measuring the total time for a signal to be sent and acknowledged, though it is limited by processing delays in the receiver's electronics [14].

2.3 Connectivity Technologies for Wearable Devices

The purpose of the connectivity subsystem is to provide a reliable and efficient method of transmitting alerts and telemetry from the wearable to the cloud backend. Considering the previously defined formal requirements, the technologies explored must balance wide-area mobility (Requirement R7), low latency for emergency transmission (Requirement R8), and strict energy efficiency to maintain a multi-day battery life (Requirement R9). The options can be broadly categorized into tethered short-range connections, unlicensed low-power wide-area networks (LPWAN), and licensed cellular IoT technologies.

Tethered Short-Range Networks (BLE and Wi-Fi)

This approach must be considered due to widespread adoption in similar commercial applications. Tethered architectures are designed to rely on an intermediate gateway (typically the users smartphone). For this purpose, using short-range protocols like Bluetooth Low Energy (BLE) or Wi-Fi significantly reduces the wearable's power consumption by offloading long-range communication to a gateway device.

However, a significant limitation of these technologies is that the wearer must remain in continuous close proximity to the gateway. If the user forgets their smartphone or its battery depletes, the wearable loses all means of transmitting critical data to the backend. While alternative tethered approaches utilize stationary wireless beacons, this requires modifying the physical environment. Such a setup is realistically only viable for confined settings like nursing homes, and it provides no coverage outdoors.

Unlicensed LPWAN (LoRaWAN)

As noted in Section 2.2.2, unlicensed LPWAN technologies can transmit data up to 15 kilometers in rural areas. However, inconsistent public network coverage across Europe presents a major drawback [28]. To guarantee connectivity, users would have to buy and set up their own private gateways, which defeats the purpose of providing a simple, out-of-the-box safety device.

Additionally, due to extremely low data rates (typically under 5 kbps) and small payload limits, the wearable cannot run standard TCP/IP protocols like MQTT directly. Instead, it must transmit highly compressed binary payloads that the backend server translates into standard messages [29]. Finally, European ETSI regulations enforce a strict 1% duty cycle in the 868 MHz band, limiting devices to just 36 seconds of airtime per hour [30]. Although this is enough to transmit the bracelet's short location updates and rare emergency alerts, it remains a hard regulatory ceiling.

Licensed Cellular IoT (LTE-M and NB-IoT)

Cellular IoT leverages existing commercial telecommunications infrastructure which is widely available and completely eliminates the need for user-installed gateways. If a cellular connection is chosen, the device communicates directly with nearby cell towers. Because the network operator centrally schedules when each device is allowed to transmit, packet collisions are avoided. As long as the cellular reception is favorable, a highly reliable connection is almost guaranteed.

5G RedCap: Standard 5G networks are engineered for massive broadband speeds, which consumes too much power for a battery-operated wearable. To address this, the telecommunications industry introduced 5G RedCap (Reduced Capability). RedCap strips away unnecessary broadband features to create a ”mid-tier” 5G connection designed specifically for IoT devices like smartwatches, industrial sensors, and video surveillance [31]. While 5G RedCap successfully lowers power consumption compared to standard 5G, it is still intended for devices that require moderate-to-high data transmission capabilities [31]. Therefore, the legacy 4G-based IoT technologies (NB-IoT and LTE-M) remain the most appropriate choices for a bracelet tracker that only transmits location data and short alerts.

NB-IoT and LTE-M: While both NB-IoT and LTE-M are low-power cellular technologies, a critical operational distinction lies in their handling of device mobility. NB-IoT was engineered primarily for stationary sensors, such as smart utility meters [32]. Consequently,

when a moving user walks out of one cell tower's range, an NB-IoT wearable must drop its connection and execute a complete, energy-intensive network renegotiation with the next tower [32].

In contrast, LTE-M natively supports seamless cell handover, maintaining an uninterrupted data link as the user travels. This seamless mobility is strictly necessary to satisfy the autonomous, wide-area connectivity mandated by Requirement R7. Furthermore, both LTE-M and NB-IoT utilize advanced cellular sleep mechanisms, namely Power Saving Mode (PSM) and extended Discontinuous Reception (eDRX) [32]. These features allow the radio module to sleep for extended periods while the network still considers the device "connected," effectively maximizing battery life without compromising alert readiness.

2.4 Embedded sensing and edge intelligence for fall detection

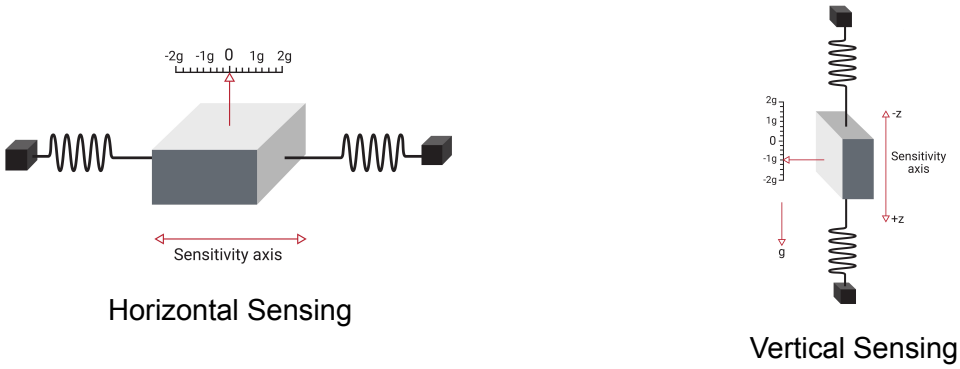
Embedded sensing refers to the integration of sensors and the processing unit inside a device. Edge intelligence is the application of AI/Machine Learning models directly at the "edge" of the network, which is the physical location where the data is produced. In this case, it means all the data should be collected and processed on the wearable device.

2.4.1 Primary sensing components

Linear acceleration and orientation measurements

The fundamental sensing unit for fall detection systems is usually the IMU (Inertial Measurement Unit), which integrates sensors that are capable of capturing acceleration and orientation of the wearable during movements. Both of these two measurements are important to distinguish between normal activities like walking or sitting down and falling accidents.

The first important component of an IMU is the accelerometer, which measures linear acceleration. In wearable devices, this is typically implemented using a micro-electromechanical systems (MEMS) accelerometer. The operating principle is based on Newton's second law of motion. The sensor contains a small proof mass suspended by microscopic springs. When acceleration is applied, the mass is displaced relative to its equilibrium position. A visual representation of such a sensing component can be observed in Figure 2.4.



Horizontal Sensing

Vertical Sensing

Figure 2.4: Operating principle of a MEMS accelerometer [33].

This displacement is detected using capacitive sensing. As modeled in Equation 2.3, the displacement x of a proof mass m with spring constant k subject to an external acceleration a is:

$$x = \frac{m}{k}a \tag{2.3}$$

The resulting change in capacitance C is proportional to this displacement, as defined in Equation 2.4, where ϵ is the permittivity of the dielectric medium, A is the plate area, and d is the baseline distance between the electrodes:

$$C = \frac{\epsilon A}{d \pm x} \tag{2.4}$$

This change in capacitance is proportional to the applied acceleration and is converted into an electrical signal. MEMS accelerometers usually measure acceleration along three orthogonal axes (x, y, and z), expressed in units of gravitational acceleration (g) or meters per second squared (m/s^2) [33]. These measurements are important for a fall detection system as falls normally produce a spike in acceleration on impact, which is more significant than those found in usual daily activities.

The second component of an IMU is the gyroscope, which measures angular velocity. In modern wearable systems, this is typically implemented using a micro-electromechanical systems (MEMS) gyroscope. The operating principle is based on the Coriolis effect. The sensor contains a vibrating mass that oscillates at a constant velocity. When the device rotates, the Coriolis force causes a secondary vibration perpendicular to the original motion. This displacement is proportional to the angular velocity and is detected using capacitive sensing, which converts it into an electrical signal. [33].

Gyroscopes measure angular velocity along three orthogonal axes, typically expressed in degrees per second (°/s) or radians per second (rad/s). These measurements enable the estimation of orientation and detection of rapid rotational movements, which are important to identify falls and distinguish them from normal daily activities.

Height displacement measurements

Another important sensor that helps fall detection is the barometer. The operating principle of MEMS barometers is based on the deformation of a thin diaphragm caused by changes in air pressure. This deformation is measured using piezo-resistive or capacitive sensing elements which convert the mechanical strain into an electric signal proportional to the pressure applied [34]. Since atmospheric pressure changes with altitude, barometers can be used to detect relative height changes, which in the context of fall detection is used to confirm the vertical displacement caused by falling.

2.4.2 On-Device Machine Learning for Fall Classification

The fall detection task can be formally defined as a binary classification problem: As input it takes a window of sensor measurements $\mathbf{x} \in R^{N \times C}$, where N is the number of time steps and C is the number of sensor channels, and the model must output a label $y \in \{0, 1\}$ which represents the absence or presence of a fall detection. Because requirement R3 states that inference runs entirely on the embedded microcontroller without continuous raw data transmission, the chosen algorithm must operate within the tight memory, latency, and power constraints of the bracelet.

Signal pre-processing

Before any classification happens, the raw sensor data must be pre-processed to reduce noise, extract meaningful characteristics, and produce consistent predefined dimensions

for the input representation. Therefore, the standard processing pipeline typically consists of the following steps:

1. **Filtering:** A low-pass digital filter is applied to the continuous sensor streams to suppress high-frequency mechanical noise and electrical artifacts, isolating the true kinematic and environmental changes.
2. **Windowing:** Because a fall is a continuous dynamic event, the inputs must represent a duration of time rather than a single instantaneous measurement. The continuous sensor stream is segmented into fixed-length temporal windows (typically ranging from 2 to 5 seconds) to capture the full pre-fall, impact, and post-fall phases. The stride of these windows defines how the data is sequentially fed into the classifier.
3. **Feature Extraction:** To resolve the orientation dependency of the accelerometer, the Signal Vector Magnitude (SVM) is commonly computed to combine the three spatial axes into a rotation-invariant scalar:

   $$\text{SVM}_{acc} = \sqrt{a_x^2 + a_y^2 + a_z^2}$$

   Beyond basic time-domain metrics, modern standard processing relies on spectral analysis to extract frequency-domain features from the accelerometer window, identifying the frequency bands and signal power associated with human falls. Furthermore, when multimodal sensing is used, barometric pressure data is processed by extracting pressure differentials to mathematically correlate kinetic impacts with vertical displacement.
4. **Normalization:** To ensure the neural network or classifier receives inputs on a consistent scale, the extracted features are standardized using Z-score normalization. Each feature x is brought to zero mean and unit variance using the mean (μ) and standard deviation (σ) computed from the overall dataset:

   $$x' = \frac{x - \mu}{\sigma}$$

Algorithm types

Threshold based methods: The simplest approach is fixed decision rules. The widely accepted models define three phases: free fall, impact and post fall. Threshold methods are extremely lightweight and deterministic, but are sensitive to parameter tuning and produce elevated false positive rates during vigorous activities such as jumping or dropping objects.

Deep learning: The most recent approach is to feed raw or lightly pre-processed data to a neural network. This allows the model to learn the discriminating features automatically without human supervision. Many architectural approaches can be explored for this task, including standard Neural Networks (NN) or one-dimensional Convolutional Neural Networks (1D-CNNs). Long Short-Term Memory (LSTM) networks and Temporal Convolutional Networks (TCNs) can additionally model long-range temporal dependencies. Although deep learning approaches can achieve higher sensitivity and specificity, their larger parameter counts require explicit compression before embedded deployment. [35]

Model optimization for embedded deployment

Deploying a trained neural network on a resource-constrained microcontroller requires applying compression techniques that reduce model size and inference latency without significantly degrading classification performance [36]. The principal techniques are:

- **Post-training quantization**: Converting weights and activations from 32-bit floating-point to 8-bit integers (INT8). This reduces memory footprint by approximately a factor of four and accelerates inference on processors without a hardware floating-point unit.
- **Structured pruning**: Removing entire filters or neurons that contribute minimally to the output, producing a smaller dense model that requires no special sparse-matrix runtime support.
- **Knowledge distillation**: Training a compact *student* model to reproduce the soft output distribution of a larger, more accurate *teacher* model, transferring generalization capability into a smaller architecture.

Hardware-integrated vs. Custom MCU-based inference

When implementing a fall detection system, choosing between the Inertial Measurement Unit's (IMU) built-in hardware algorithms and custom machine learning on the microcontroller (MCU) involves a strict trade-off between power consumption and accuracy. Relying exclusively on the IMU's basic logic allows the MCU to remain in deep sleep. However, these built-in algorithms rely on simple thresholds originally designed to protect consumer electronics from free-falls. Because elderly falls rarely exhibit a pure 0g phase, relying solely on this logic causes missed falls and high false-positive rates. Conversely, continuously streaming raw sensor data to the MCU for more accurate ML classification drains the battery rapidly.

The solution that bridges this gap is utilizing an IMU's hardware First-In-First-Out (FIFO) buffer alongside hardware interrupts. The IMUs continuously logs raw data into the buffer, overwriting the oldest readings to maintain a rolling window of recent movement without waking the MCU. The IMU can also leverage its basic kinetic threshold (e.g., an acceleration spike > 1.5 g). Only when this specific threshold is breached does the IMU send an interrupt to wake the MCU. Upon waking, the MCU downloads the pre-impact kinematic data from the buffer and processes it through the fall detection algorithm to accurately verify if a genuine fall occurred.

2.5 Low-Power Design in GNSS and Cellular Wearables

Battery life and power management are the definitive constraints in the design of Internet of Things (IoT) devices, particularly for GNSS and cellular wearables that function as mobile gadgets in often remote locations [37]. The engineering challenge lies in balancing high-performance with extended operational longevity, as the right power management strategy often distinguishes a functional product from an optimal one.

Software and Communication Strategies

Because wireless communication, namely cellular handshakes and GNSS satellite acquisition, is the most energy-intensive operation, software must be designed to consider this factor. Beyond edge computing, where TinyML analyzes data locally to reduce transmission frequency [38], the integration of specialized Narrowband IoT (NB-IoT) and LTE-M power-saving protocols is essential for geriatric autonomy.

The most significant advancements in this area are Power Saving Mode (PSM) and extended Discontinuous Reception (eDRX). PSM allows the wearable to enter a deep-sleep state similar to being powered off, while remaining registered with the network. This eliminates the massive energy cost of re-attaching to a cell tower every time a fall is detected.

Complementing this, eDRX extends the intervals at which the device ”pings” the tower to check for down-link pages, such as a remote location request from a caregiver.

From an execution standpoint, code must be optimized in order to group transmission data together so the radio only wakes up once to send a burst of information. By synthesizing these energy-aware programming frameworks with PSM and eDRX, modern wearables can maintain a persistent safety net for several days, directly addressing the risk of users with cognitive decline forgetting their daily charging routines.

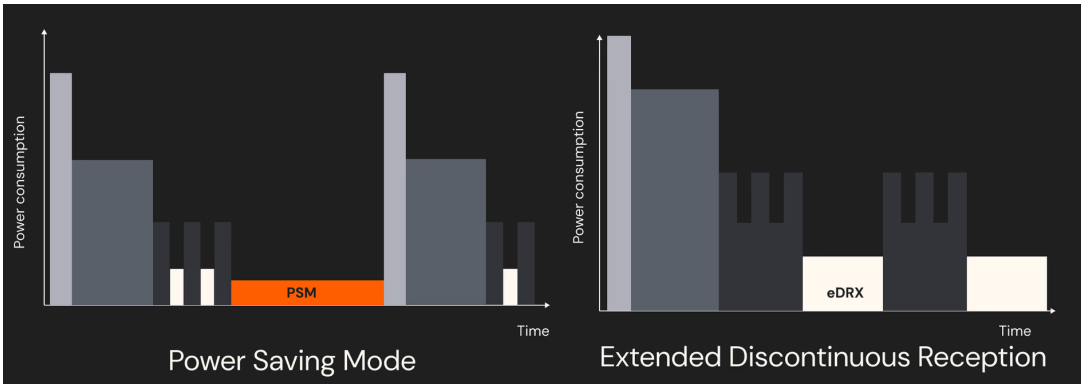


Figure 2.5: Low Power Modes for Cellular IoT: PSM and eDRX

2.6 Cloud backends and mobile applications for remote monitoring systems

2.6.1 Network Topologies

The architectural topology of a remote monitoring system defines how data flows from the sensing node (the bracelet) to the end-user (the caregiver). In the context of IoT health monitoring, three dominant architectural paradigms exist: Gateway-based, Peer-to-Peer (P2P), and Centralized Client-Server.

Gateway-Based Architecture: the wearable does not communicate directly through internet. It uses short range and low powered communication, such as Bluetooth Low Energy, to transmit data to an intermediate bridge device (often a smartphone) [39]. Then, the gateway device is responsible for sending the data to the backend via Wi-Fi or Cellular networks. Fitness trackers tend to use this approach since power consumption is reduced by offloading wide band communication to the gateway. However, this architecture introduces a significant physical dependency: the wearer needs to stay within range of the gateway (often 10-30 meters).

Peer-to-Peer Architecture: eliminates the central server by establishing a direct communication link between the wearable device and the caregiver’s application. The primary advantage of P2P is data privacy and reduced cost to setup infrastructure, as no data is stored on a third party server [40]. However, the wearable and the application need to be online at the same time in order for an alert to be delivered. This is problematic because if the caregiver’s smartphone restricts data usage for the application, the alert cannot be delivered and it also cannot be stored in an external database, resulting in loss of critical information.

Client-Server architecture: fundamental for these types of applications. In this architecture, the wearable device is an independent node in the network [41] and it has its own

cellular modem (LTE-M). It communicates exclusively to a central cloud backend, which processes, stores and directs the information to the right caretaker. The caretaker's mobile application also connects to the same server to retrieve status messages or receive push notifications. A visual comparison of these topological differences is illustrated in Figure 2.6.

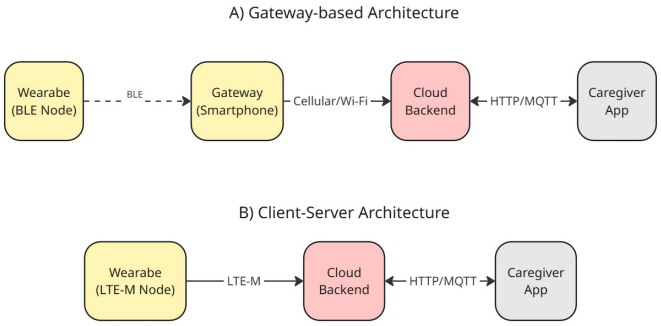


Figure 2.6: Comparison between possible network topologies.

2.6.2 Communication Protocols

Due to the different specifications, the communication requirements for the wearable and the caregiver's application differ. In the context of wearables where battery life is a critical element, standard web protocols are often too heavy. Consequently, specific IoT protocols have been developed to handle unstable connections and limited power.

MQTT (Message Queuing Telemetry Transport)

MQTT is currently the dominant protocol in the IoT landscape. It runs over the TCP/IP (Transmission Control Protocol / Internet Protocol) stack and is designed specifically as a lightweight, binary protocol. A key advantage of MQTT is its minimal protocol overhead. Overhead refers to the metadata headers required to wrap the actual sensor payload so it can be routed correctly. In text-based protocols like HTTP, this metadata is verbose. However, MQTT's fixed header size can be as small as 2 bytes. This efficiency allows the wearable device to minimize the time the radio modem is active, thereby saving resources when transmitting data through energy-intensive LTE-M networks.

The reliability of MQTT is derived from the underlying TCP transport layer which is a connection-oriented protocol that guarantees data delivery [42]. Before data transmission begins, a Three-Way Handshake (SYN, SYN-ACK, ACK) is performed to establish a reliable session. Similarly, a Four-Way Handshake is used to terminate the connection safely. During transmission, TCP sends an acknowledgment (ACK) for every segment received, automatically detecting errors and retransmitting lost packets. However, this robustness comes at a cost: to maintain a persistent connection, devices must send periodic "heartbeat" packets (Keep-Alive) to keep the channel open. In scenarios with poor cellular signal or frequent deep-sleep cycles, the energy required to constantly re-establish the Three-Way Handshake can negate the benefits of the lightweight headers.

Architecturally, MQTT differs from traditional REST APIs by using a Publish-Subscribe model rather than Request-Response. The system requires a central component called a Broker. Devices do not communicate directly with each other, as they Publish messages to a specific Topic (e.g., `bracelet/123/alerts`). As illustrated in Figure 2.7, the broker itself does not require custom application logic to route this data. Instead, a dedicated

data processing service within the cloud backend acts as an MQTT subscriber. Whenever the wearable publishes an alert, the broker automatically pushes this event to the subscribed processing service, which then interprets and stores the data so the caregiver's mobile application can retrieve it via standard HTTP REST APIs or WebSockets [43]. This decoupling allows the wearable to push data without knowing if the server is currently processing it, and allows the server to push configuration updates to the device whenever it comes online.

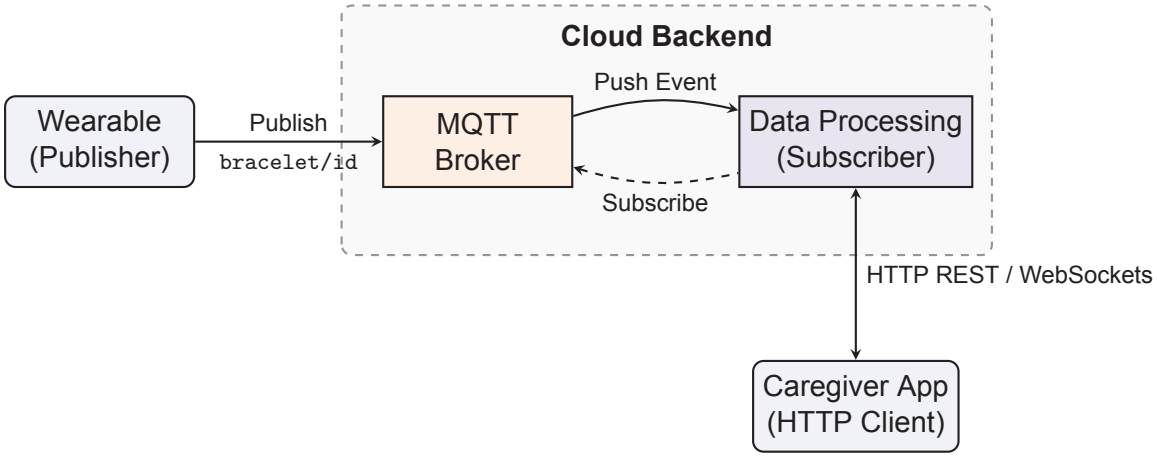


Figure 2.7: Data flow example in a possible MQTT implementation

CoAP (Constrained Application Protocol)

CoAP is an alternative protocol designed specifically for constrained nodes that cannot maintain the persistent connections required by MQTT. Unlike MQTT, CoAP runs over UDP (User Datagram Protocol). UDP is a ”connectionless” protocol. It does not require a handshake to establish a link before sending data, as the device essentially sends the data packet towards the server and does not ensure successful transmission. This model closely mimics the RESTful architecture of the web (GET, POST, PUT, DELETE) but is optimized to use very compact binary headers [44].

The use of UDP offers distinct advantages for devices with weak connectivity. Because there is no connection to maintain or rebuild, a device can wake up, send a single packet, and immediately go back to sleep, resulting in very fast transmission times and lower power consumption. It is particularly robust in poor signal areas where a TCP connection might time out during the handshake phase [44].

The trade-off for this efficiency is reliability. Since UDP does not naively verify that the receiving end got the message, packets can be lost in transit without the sender knowing. While CoAP implements a mechanism to mark messages as ”confirmable” (requiring an acknowledgement from the application layer), it lacks the robust, built-in flow control of TCP. Additionally, many cellular firewalls are configured to drop UDP traffic or close UDP ports very quickly.

HTTP/HTTPS (REST)

HTTP (Hypertext Transfer Protocol) is the universal standard for web communication. It typically follows a request-response model over TCP, where the client opens a connection, requests a resource (or posts data), receives a response, and closes the connection. It is text-based and can be easily read and understood by developers.

The main strength of HTTP is its ease of development and debugging. It requires no

special middle-ware or brokers and passes through almost all network firewalls without configuration. For the mobile application side of the project, HTTP is the logical choice due to the rich ecosystem of RESTful APIs and the high bandwidth available to smartphones. However, for the wearable device, HTTP is generally considered inefficient. Because it is text-based, the headers are simply too large. A simple request to send 1 byte of data might require hundreds of bytes of headers, and also modern security standards require a HTTPS connection which is done over TLS, that requires a very complex handshake that also consumes a lot of resources.

2.6.3 Backend Architecture

Once the data is successfully transmitted from the wearable to the cloud via the chosen protocol, it must be processed, stored and made available to authorized caretakers. This backend forms the core of the monitoring system and must be reliable, scalable and must guarantee secure access to sensitive data such as precise localization of the users.

Modern IoT backend systems are typically implemented using cloud computing infrastructure. Cloud computing provides on-demand access to computing resources such as servers, databases, and networking, allowing systems to scale dynamically based on workload without requiring dedicated physical infrastructure [45]. This means that instead of managing the hardware, the infrastructure provided by a third party company is used. Within this context, there are two major architectural choices:

Monolithic architectures

A monolithic backend is a single, unified application that contains all core functions of the system, including device communication, business logic, data storage, and interfaces for clients. All parts of the software are tightly connected and deployed together as one service [46].

This approach is straightforward to develop and deploy because everything is located in one place, which also makes debugging easier. However, it becomes harder to scale as the system grows, since the entire application must be expanded even if only one part needs more resources. Failures in one component can also affect the whole system, and updating individual parts independently is difficult. Despite these limitations, monolithic architectures remain practical for smaller systems and early prototypes.

Microservices architectures

In a microservices architecture, the backend is divided into many small and independent services, each responsible for a specific function of the system [47]. Instead of one large application, the system is composed of separate components such as data processing, device communication, authentication, or notifications, all running as individual services.

These services communicate with each other through well-defined interfaces, usually over the network. This separation allows each service to scale independently, improves fault isolation, and makes it easier to update or replace parts of the system without affecting the rest. The trade-off is that the overall system becomes more complex to manage, since coordinating multiple services and handling network communication requires additional infrastructure. Despite this added complexity, microservices are widely used in cloud-based IoT platforms because they support high scalability and modular system design [48].

2.6.4 Backend interface for client applications

To make device data accessible to caregivers, the backend must expose interfaces through which mobile applications can retrieve information and receive alerts. These interfaces are typically implemented using web technologies that support both periodic data access and real-time event delivery.

REST APIs

REST is a widely adopted architectural style in which clients interact with server resources using HTTP requests [49]. REST APIs are simple to implement, broadly supported, and well suited for mobile applications that need to fetch device status, retrieve historical information, or update configuration parameters.

Real-time communication mechanisms

Some monitoring scenarios require immediate delivery of critical information. Technologies such as WebSockets enable persistent connections between clients and servers, allowing the backend to push updates as soon as events occur [50]. This capability can be essential for safety alerts where caregivers have to be notified without delay.

3 System Design

This chapter describes the system architecture and engineering decisions behind the wearable prototype. The first section explores the hardware domain, justifying the selection of the microcontroller, sensors, radio frequency components, and power unit to meet strict size, power, and connectivity constraints. The second section examines the software infrastructure, outlining the embedded firmware's event-driven state machine, the cascading localization pipeline, and the secure, cloud-centric backend that ensures reliable emergency alerting.

3.1 System Overview

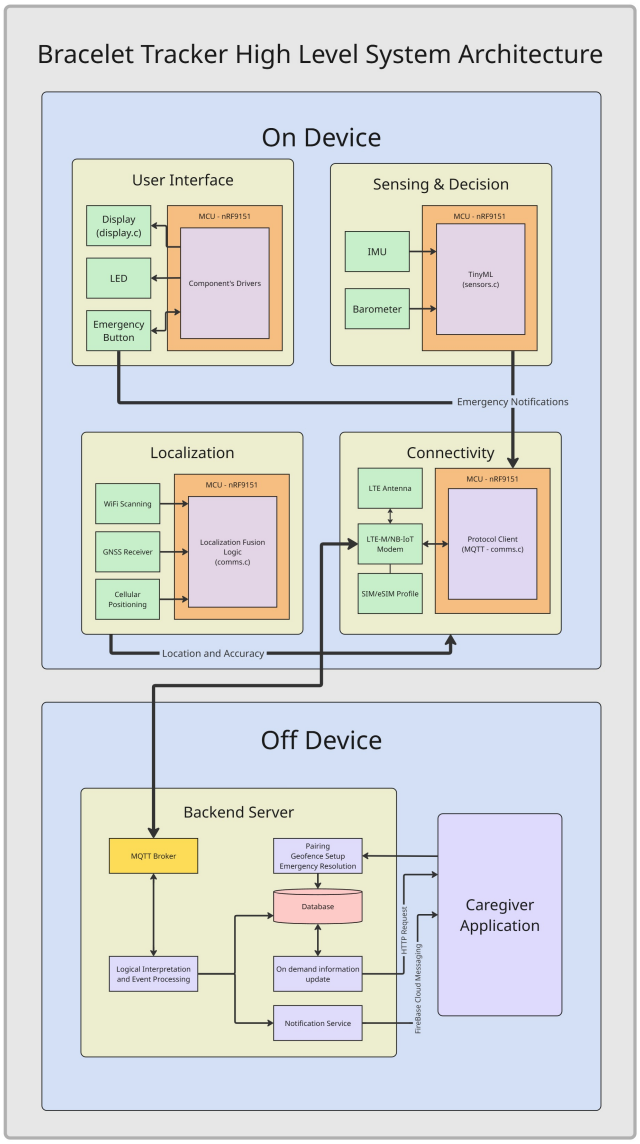


Figure 3.1: System architecture

The bracelet prototype consists of a set of components whose combined purpose is to provide the localization, fall detection and connectivity capabilities as a standalone wearable device.

Its main objective is to remain reliable both indoors and outdoors, delivering accurate fall detection and localization data to a caregiver application, so that the user's activity can be tracked at all times and an alert issued whenever a fall or a panic event occur. This objective naturally comes with a significant challenge: power consumption, which may be driven by various factors such as energy-intensive components or inefficient software management.

3.2 Hardware Design Choices

The hardware architecture for this project builds upon the foundational schematics and PCB design developed in a previous DTU Master's thesis by Marc Lorenz [51]. Because the original design successfully integrated the essential components for active tracking and fall detection, its core architecture was retained. However, to address documented limitations from the previous iteration, the board was upgraded to correct existing errors and expand its functionality. Specifically, to meet stricter size constraints, optimize power consumption, and improve fall detection accuracy, several components were modified and the PCB layout was redesigned.

3.2.1 Connectivity

Since the bracelet is intended to operate while the wearer is moving, LTE-M was selected as the primary cellular technology. Unlike NB-IoT, LTE-M supports seamless handover between cellular base stations, allowing the device to maintain connectivity while traveling. This is essential for a wearable tracker intended to function continuously outdoors.

LTE-M also provides better support for the localization pipeline adopted in this project. The bracelet uses Assisted GNSS (A-GNSS), which downloads satellite assistance data over the cellular network before acquiring a GNSS fix. The higher data rate and lower latency of LTE-M reduce the assistance download time, contributing to the shorter localization times and lower energy consumption.

3.2.2 Localization

Selecting the appropriate localization technology requires evaluating the industry standard options for Internet of Things devices based on precision, power consumption, and range. In the scope of this application, Bluetooth Low Energy, RFID, Ultra Wideband, LoRaWAN, Sigfox, Wi-Fi, GNSS, cellular positioning, and inertial navigation were considered.

Table 3.1: Comparison of Evaluated Localization Technologies [14]

Technology	Precision	Peak Active Power	Typical Range	Infrastructure Need
BLE & RFID	< 5 m	< 10 mA	< 50 m	Beacons / Scanners
Ultra Wideband	< 10 cm	< 50 mA	< 50 m	Anchor Nodes
LoRaWAN & Sigfox	100 - 1000 m	< 50 mA	> 10 km	Network Gateways
Inertial Navigation	Drifts over time	< 10 mA	N/A	None
Wi-Fi (Passive)	10 - 50 m	10 - 50 mA	< 150 m	Access Points
GNSS (Assisted)	1 - 10 m	> 30 mA	Global	Satellites / Clear Sky
LTE-M (Cellular)	500 - 5000 m	100 - 200 mA	> 10 km	Cellular Towers

As detailed in Section 2.2, several localization methods were discarded based on the constraints outlined in Table 3.1. Short-range protocols like BLE, RFID, and UWB offer excellent precision and low power consumption (< 50 mA), but their restricted range (< 50 m) and strict reliance on localized infrastructure (beacons or anchor nodes) make them unsuitable for an autonomous, wide-area tracker. Unlicensed LPWANs (LoRaWAN and Sigfox) offer sufficient range (> 10 km) without high power penalties, but their native localization precision is unacceptably low (100–1000 m) and depends heavily on external gateway density. Finally, inertial navigation was rejected because its precision inherently drifts over time, failing to provide the absolute global coordinates required for emergency response.

Localization Pipeline

To overcome these individual limitations, a localization waterfall comprising Wi-Fi, GNSS, and LTE-M is deployed. This waterfall methodology evaluates technologies sequentially to balance power and precision. The sequence starts with passive Wi-Fi scanning. If this fails, the system activates GNSS, and if satellite data is unavailable, it falls back to LTE-M cellular positioning.

Wi-Fi scanning serves as the primary localization method. In urban areas, a single passive scan can detect multiple router access points. Cross referencing these access points with a spatial database constitutes a state of the art localization technique offering high accuracy and reliability. Because passive Wi-Fi scanning only listens for signals without establishing a network connection, it remains a highly power efficient methodology. Furthermore, vulnerable populations tend to spend a significant portion of their time indoors, which justifies prioritizing an indoor optimized technique.

When a user moves outdoors or out of urban range, Wi-Fi scanning may fail. In this scenario, the system escalates to GNSS. GNSS relies on satellite signals that are incredibly weak and easily blocked by roofs, making it useless indoors. A standard GNSS fix consumes a significant amount of power. However, this can be reduced by utilizing Assisted GNSS. This technology leverages the cellular connection to download satellite ephemeris data, which can reduce the time required to lock onto a location by a factor of ten, drastically decreasing overall power consumption. Combining this highly precise outdoor technique with the indoor reliability of Wi-Fi equips the device to handle almost all everyday scenarios.

Finally, cellular positioning via LTE-M serves as the ultimate fallback mechanism. Because the LTE-M modem is already architecturally required to transmit emergency alerts to the caregiver application, utilizing it for location data adds no extra hardware constraints. Cellular positioning has low precision, often measuring in hundreds of meters, but it possesses incredible penetration power. This makes it a vital final safety net for extreme situations, such as when a user is located deep inside a concrete building lacking both Wi-Fi routers and satellite visibility.

3.2.3 Micro Controller Unit

The Nordic Semiconductor nRF9151 is maintained as the primary microcontroller unit (MCU) and cellular modem for this design due to its highly integrated System-in-Package (SiP) architecture. It combines a power-efficient Arm Cortex-M33 application processor, a multimode LTE-M/NB-IoT baseband, and a complete RF Front-End (RFFE) into a single compact package with 134.3 mm^2. A deciding factor for this project is its native optimization for cellular IoT since it tightly couples the processor's sleep states with the cellular

network's Power Saving Mode (PSM) and extended Discontinuous Reception (eDRX) intervals.

Furthermore, the SiP also integrates a Global Navigation Satellite System (GNSS) receiver, eliminating the need for an external positioning module. From a hardware compliance perspective, the module is globally pre-certified for major telecommunication standards which substantially reduces development time. On the software side, the nRF Connect SDK, built on the Zephyr RTOS, provides a scalable development environment which allow to integrate low power sleep states easily.

Utilizing a standard MCU, such as an STM32 or ESP32, paired with an external cellular modem, such as a Quectel or u-blox module, is less efficient for this type of application. A discrete two-chip approach needs high-speed UART buses and sensitive RF lines routed across the PCB, which increases the physical footprint, the cost of production and the susceptibility to electromagnetic interference. Opting for this alternative would translate into an added cost of 6 to 20 USD, depending on the supplier, which combined with higher power consumption makes the nrf9151 SiP a better choice for this project.

3.2.4 WiFi Chip

To establish Wi-Fi as the primary indoor localization technique, the Nordic Semiconductor nRF7002 Wi-Fi 6 companion IC was selected to actively scan nearby access points, perfectly complementing the nRF9151's GNSS and LTE-M capabilities. The nRF7002 is a low-power, dual-band (2.4 GHz and 5 GHz) Wi-Fi 6 (802.11ax) solution operating with a 20 MHz channel bandwidth. Crucially, it supports Target Wake Time (TWT), which drastically reduces idle listening current compared to legacy Wi-Fi 4 alternatives like the ESP32 series.

Operating strictly as a companion IC via a high-speed SPI bus, the nRF7002 lacks an independent application processor because its drivers are natively integrated into the nRF9151's Zephyr RTOS environment via the nRF Connect SDK. This unified architecture allows the nRF9151 to manage both cellular and Wi-Fi operations seamlessly orchestrating hardware coexistence.

On the other hand, pairing the nRF9151 with discrete external modules, such as the Espressif ESP32-C3 or Microchip ATWINC1500, which are widely used in many IoT applications requiring WIFI features, introduces significant architectural overhead. These setups rely on asynchronous AT commands or proprietary SPI protocols managed across separate toolchains, reducing system efficiency and complicating development.

Beyond architectural simplicity, the nRF7002 offers superior power efficiency for scanning operations compared to traditional standalone modules. As detailed in Table 3.2, the nRF7002 demonstrates lower peak active currents during both transmission and reception. Because passive indoor localization heavily relies on brief reception windows (RX mode) to sniff access point beacons, the reduced 60 mA and 56 mA RX current draws of the nRF7002 directly contribute to extending the device's operational battery life.

3.2.5 Power Management Unit

Powering a multi-radio system with high peak transmission currents, such as LTE-M, requires a Power Management IC (PMIC) that balances dynamic current delivery with low quiescent current while in a sleep state. The architecture for this project requires a small PCB footprint, an integrated battery charger with precise fuel gauging, and I2C compatibility. Furthermore, software integration with the primary application processor is critical

Table 3.2: Power consumption comparison of evaluated Wi-Fi solutions

Component	Operating Mode / Standard	Peak TX Current (mA)	RX Current (mA)
Nordic nRF7002	2.4 GHz (MCS7, Max Power)	191	60
	5 GHz (MCS7, Max Power)	260	56
Espressif ESP32	2.4 GHz (802.11b, @ 21 dBm)	335	84
Microchip ATWINC1500	2.4 GHz (802.11b, @ 17.5 dBm)	268	61

to dynamically manage voltage rails and sleep states without introducing code complexity.

The Nordic nPM1300 was selected as the primary PMIC due to its architectural alignment with the nRF9151 processor. This component provides two independent high-efficiency (up to 93%) buck regulators and two low-dropout (LDO) load switches. This topology enables strict power domain isolation, allowing the nRF9151 to completely shut off power to the nRF7002 Wi-Fi companion chip during sleep periods.

Additionally, the nPM1300 integrates an accurate algorithmic fuel gauge, a major advantage comparing to its market competitors. Unlike the Maxim MAX77654, which requires a dedicated external IC for state-of-charge tracking, or the TI BQ25120A, which relies on basic voltage monitoring, the nPM1300 handles these tasks internally. Finally, its native integration with the Zephyr-based nRF Connect SDK allows the nRF9151 to manage battery charging profiles and power rails directly via standard system APIs.

Buck Boost Converter

Regarding the power rails, the previously implemented LDO was replaced with a buck-boost converter. This component is critical to the design, as it provides the required voltage and current to the Wi-Fi companion chip. If supplied by the PMIC's internal LDOs, the Wi-Fi chip would fail due to insufficient current capacity. Specifically, the nRF7002 has peak current consumption of up to 260 mA, whereas the PMIC's LDOs max out at 100 mA, despite being able to supply the necessary 3.3V.

The previous design utilized a MIC5219-3.3YM5 low dropout (LDO) regulator, which assumed a fully charged battery voltage of 4.2V. However, for this specific LDO, the maximum dropout voltage (V_{DO}) at full load (500 mA) is 500 mV. Given that the required output is 3.3V the minimum input voltage has to be 3.8V (3.3V + 0.5V). Because a standard Li-ion battery's voltage naturally drops from 4.2V down to 3.0V as it discharges, this configuration would cause system malfunction.

To resolve this, the current project implements a buck-boost converter. Unlike an LDO, which can only step voltage down, a buck-boost converter steps the voltage down when the battery is full (4.2V) and steps it up when the battery is nearly empty (3.0V), maintaining a stable 3.3V output throughout the entire battery life. The Texas Instruments TPS63031 single-inductor buck-boost converter was selected because its specifications align perfectly with the design requirements. It supports an adequate input range (1.8V to 5.5V) and outputs a fixed 3.3V. It easily handles the system's power demands, delivering 800 mA at 3.3V in step-down mode (V_{in} from 3.6V to 5.5V) and up to 500 mA in boost mode ($V_{in} > 2.4V$). With up to 96% efficiency, a quiescent current under 50 μA, a small 6.25 mm^2 footprint, and full compatibility with Li-ion batteries, it is an ideal fit for the design.

3.2.6 Accelerometer and Barometer

To accurately capture the three distinct phases of fall detection, the system requires both an accelerometer and a barometer capable of operating with minimal current consumption while maintaining high precision. The predecessor of this device relied solely on an accelerometer with satisfactory results but it was prone to false positives. To mitigate these false alarms, and the avoidable power consumption resulting from falsely triggered localization events, a barometer was integrated into the new design.

Inertial Measurement Unit (IMU)

The Bosch BMI270 IMU was retained from the previous iteration due to its precision and low-power architecture. This sensor features built-in automated free-fall detection, which forms the foundation of an efficient fall detection pipeline. Crucially, the BMI270 provides hardware interrupt capabilities, allowing it to send a signal that wakes the microcontroller unit (MCU) from deep sleep only when a specific motion event is detected. Combining these 3-axis acceleration triggers with pressure data enables a highly robust fall detection model. Hardware specifications such as its 2 KB FIFO buffer and I2C interface further justify its choice. Power-wise, it requires a minimal 685 µA in active operation and supports a wide supply range.

Barometer

The addition of a barometer is critical for filtering out false positives caused by sudden movements, such as jumping or work involving intense hand movements. Unlike a true fall, these events lack a sustained drop in altitude. By integrating a sensor capable of detecting pressure changes with centimeter-level altitude precision, the confidence level of the fall detection model is significantly increased.

Initially, the STMicro LPS22DF was chosen but due to hardware shortage, the TDK InvenSense ICP-20100 was selected to fulfill this role. This barometer operates within a 30 to 110 kPa pressure range and utilizes digital filtering to reduce signal noise and glitches producing low noise. It is highly optimized for battery operation, featuring a typical active current consumption of 211 µA and dedicated low-power modes when idle. Although the STMicro barometer fits better into this project due to its larger FIFO buffer and lower consumption, the chosen alternative fulfills the project requirements. To validate this component selection, similar low-power barometers are compared in table 3.3.

Table 3.3: Comparison of low-power barometric pressure sensors.

Parameter	TDK ICP-20100	Bosch BMP390	STMicro LPS22DF
Active Current [μA]	211	340	70
Relative Accuracy [cm]	$\pm$ 8.5	$\pm$ 25	$\pm$ 2.8
Supply Voltage Range [V]	1.8 – 3.3	1.65 – 3.6	1.7 – 3.6
FIFO Buffer Size [samples]	16	73	128
Footprint [mm^2]	4	4	4

3.2.7 User Interface & Flash Memory

In terms of user interface, the main concern is to accommodate the product for elderly people's needs so we kept the emergency button, the buzzer, an RBG LED and we added a display.

Emergency Button

The emergency button acts as a manual trigger for emergencies. In this case, whether the alert state is manually activated or autonomously, the user can also use this button to cancel it.

Buzzer

A standard piezoelectric buzzer is used to notify the user of an emergency state through an audio signal which can alert nearby spectators.

RGB LED

An RGB LED is implemented in the board to provide visual cues and for debugging. This LED is crucial for people with hearing impairments which is common among vulnerable users.

Display

To facilitate the integration of the device into daily life, a display was added into the design. This component provides the date, time, battery status, and critical emergency states.

A Wisevision OLED module was selected over alternative technologies like E-ink and Memory in Pixel displays. These alternatives offer excellent baseline power metrics but their inherently reflective nature requires a dedicated front light for readability in dark environments. This addition introduces a 10 to 20 milliamp current draw and increases hardware complexity, effectively negating their primary advantage. Beyond power considerations, the OLED better fulfills the goal of seamless daily integration. Its higher refresh rate and superior contrast ensure easier visualization and reduce optical strain for elderly users.

This display has an operating current of 34 milliamps in a scenario where all pixels are fully illuminated. Because OLED pixels are self illuminating and draw zero current when black, power scales directly with the interface design. The user interface utilizes minimalist text to represent time resulting in only 15% of pixel usage. Moreover, software controlled dimming limits the brightness to 50%. This scaling results in an effective active current consumption of just 2.55 milliamps.

To further minimize power consumption, the screen can be triggered upon detecting a wrist tilt gesture with the accelerometer, waking only when the user actively checks the device. Otherwise, the display draws negligible current, 10 microamps, in deep sleep mode. Finally, its standard communication interface and compatible voltage range align perfectly with the system architecture.

3.2.8 Connectors

Several connectors from the original design were retained to support debugging and core functionalities:

- A UART connector to allow for serial terminal connections.
- An SWD connector to provide an interface for flashing new firmware.
- A USB Type C connector to supply power when the battery is absent.
- A standard SIM card connector to enable cellular connectivity.
- A standard 30 pin ribbon cable connector to integrate the display module.

3.2.9 Radio Frequency

A standard radio frequency architecture consists of a modem, a radio frequency front end (RFFE), a matching network, and an antenna. The modem processes the digital signals, while the front end manages the analog transmission and reception. The matching network ensures maximum power transfer by matching the impedance between the front end and the antenna, which is responsible for the emission and reception of electromagnetic waves.

Wi-Fi and GNSS

For Wi-Fi and GNSS communication, an Ignion NANO mXTEND NN02-101 antenna booster was selected. This component uses the printed circuit board ground plane as the actual radiating element. Because its operating frequency depends entirely on the matching network, this exact same component type can be used for both Wi-Fi and GNSS hardware paths. Furthermore, this specific architecture was proven highly effective in the previous hardware iteration, achieving an average efficiency of 55% with peak gain of 2.4 dBi and costing less than 1 USD.

Regarding Wi-Fi, it operates across two distinct bands at 2.4 GHz and 5 GHz. Thus, a diplexer is required to route the low and high frequency signals from the Wi-Fi chip onto a single shared antenna path. This ensures both bands can be transmitted and received efficiently through the same matching network. For this purpose, a Murata RF Diplexer supporting the 2.4 to 2.5 GHz and 4.9 to 5.95 GHz bands was integrated into the design.

The nRF9151 SiP includes a GNSS compatible processor which eliminates the need for an external GNSS module. Thus, an Ignion ground plane antenna is chosen to operate between 1559 and 1610 MHz. Unlike Wi-Fi signals which originate locally, GNSS signals travel from satellites in orbit and are incredibly weak upon reaching the surface. To amplify these faint signals, a Skyworks SKY65943 Front End Module was selected. This specific module integrates a low noise amplifier with pre and post filters. It provides a 16 dB signal gain while consuming only 2.9 milliamps of current, making it highly suitable for a low power space constrained wearable device.

LTE-M

The nRF9151 integrates the LTE-M modem and its front end directly. This front end includes a power amplifier that boosts the outgoing signal up to 20 dBm and a low noise amplifier for incoming signals. The physical connection to the external antenna is routed through a standard 50 ohm trace directly to a U.FL connector. For LTE-M, instead of relying on a ground plane antenna which requires a large copper clearance area to radiate properly, an external flexible antenna connected via a U.FL connector was chosen. This design choice not only optimizes space but also allows for simultaneous LTE-M and WiFi operation.

For this purpose, a Molex 212570 flexible cellular antenna attaches to the U.FL connector. This solution does not require any matching network, further reducing the required printed circuit board area. The selection of this specific antenna was based on its compact size (4 cm), physical flexibility, and low cost (2 USD).

3.2.10 Inital Development

Before the custom hardware implementation, initial software development and debugging were conducted on the Thingy:91 X. This development kit by Nordic Semiconductor serves as a prototyping device built around the nRF9151 System in Package. It features

Table 3.4: Summary of RF Front End and Antenna Specifications

Protocol	Antenna Type	Frequency Band	Peak Gain	Efficiency	Required Front End Hardware
LTE-M	Molex Flexible	824 to 2170 MHz	1.76 dBi	$>$ 50%	Integrated PA and LNA
Wi-Fi	Ignion NANO Booster	2.4 GHz and 5 GHz	2.4 dBi	$>$ 55%	Diplexer
GNSS	Ignion NANO Booster	1559 to 1610 MHz	2.4 dBi	$>$ 55%	External LNA

GNSS and LTE capabilities alongside the nRF7002 Wi-Fi companion IC, matching the designed localization pipeline. Furthermore, the nPM1300 Power Management IC provided the charging features required for a battery operated IoT device. Using this platform allowed the project's core components to be thoroughly tested and the firmware to be developed before the custom printed circuit board was manufactured.

3.3 Software and Communication

3.3.1 Backend Architectural Selection

Based on the system requirements outlined in Section 2.1, the Centralized Client-Server architecture is the only viable option for this thesis. The Gateway-based model is incompatible with Requirement R7 (wide-area mobility) and R6 (no smartphone dependency). The P2P model fails to meet the reliability implied by Requirement R8, as a fall detection system cannot rely on the caregiver's device being reachable at the exact moment an accident occurs. Therefore, a cloud-centric approach is selected to ensure that emergency events are reliably captured and delivered asynchronously to the caregiver.

3.3.2 Communication Protocol Selection

Given the specific constraints of the project, specifically the requirement for reliable emergency alerting (R8) and the wide-area mobility provided by LTE-M (R7), the choice of communication protocol requires a trade-off between energy efficiency and data reliability.

CoAP was considered as a potential choice for this application. It offers the lowest theoretical power consumption due to its connectionless nature, but it introduces significant risks regarding reliability. UDP provides no retransmission of its own, and CoAP's optional "confirmable" message mode only adds a basic, limited-attempt retry on top of it. In a safety-critical application the device must be certain that a fall alert has actually reached the backend, even across a temporary loss of cellular coverage. Implementing a more robust reliability layer manually over UDP would require additional application-layer logic, increasing code complexity and adding potential points of failure [52].

Therefore, MQTT over TCP/TLS is selected as the optimal compromise for this system.

- **Reliability:** Two independent layers of reliability work together, each solving a different potential failure point. First, TCP guarantees that within a single, live connection, every packet is delivered in order, automatically retransmitting anything lost or corrupted in transit. Second, MQTT's QoS 1 delivery mode guarantees that a published message survives even a full connection drop. If the bracelet loses LTE coverage mid-transmission, MQTT keeps the message queued and automatically republishes it once the connection is restored, until the broker confirms receipt. TCP alone cannot do this, since a dropped connection loses whatever was in flight.

  It is important to note that this MQTT-level guarantee only confirms that the message reached the broker, not that the backend has finished processing it. For this reason,

the bracelet firmware additionally implements an *application-level* acknowledgment: the alert payload is republished every 60 seconds until the backend explicitly confirms that it has processed the event, which guarantees all alerts are processed and forwarded to the caretaker.

- **Security:** TLS provides robust encryption and authentication (R12), preventing unauthorized access to location data.
- **Duty Cycle Management:** To satisfy the 7-day battery requirement (R9), the device will implement a connect, publish and sleep cycle that minimizes power consumption and maximizes localization performance. When the bracelet doesn't need to report, the MQTT session is torn down and the modem is put to sleep or power saving mode so that the protocol's higher power consumption is only used in the exact moments when data actually needs to be sent.

Communication between the backend and the caregiver application is implemented using a RESTful API. Unlike the bracelet, which maintains an MQTT connection to periodically transmit telemetry, the mobile application performs user-driven operations such as pairing a bracelet, retrieving historical events, configuring geofences, and viewing device status. These operations are naturally request-response interactions, making REST a more appropriate communication model. Furthermore, REST is the industry standard for mobile application development and it integrates directly with the standard HTTP libraries available the mobile development frameworks like Flutter.

3.3.3 Embedded Software Design

The firmware running on the bracelet is built on the Zephyr RTOS, using Nordic's nRF Connect SDK (version 3.3.0). This choice follows directly from the MCU selection in Section 3.2.3 (nRF9151), since Zephyr is the naively supported operating system for this chip family.

Event-Driven Firmware Architecture

A single sequential loop was deemed insufficient for the complex functionality of the bracelet. Therefore, an event-driven architecture was implemented. Three logical components are running concurrently: the sensing domain (consisting of IMU/Barometer sampling and interrupt handling), the communication domain (LTE-M, MQTT, Wi-Fi, GNSS), and a central decision domain that orchestrates the other two.

This decoupling is a fundamental design choice since multiple events can happen at the same time (eg. a harsh impact while performing localization) and they need to be detected and processed based on priority. This is essential to satisfy the responsiveness required by requirement R8 (sub-30 s alert latency): the panic button must be able to interrupt an in-progress fall evaluation without being queued behind it.

The communication between independent software components happens through Zephyr's ZBUS message bus, where producers publish typed `bracelet_event` structures to a shared channel, and the decision logic subscribes to that channel and reacts to whatever arrives. Time-based behavior (periodic location updates, cool-down periods, per-state timeouts) is implemented using dedicated Zephyr kernel timers.

The Bracelet State Machine

The decision domain is implemented as a finite state machine (FSM) using Zephyr's State Machine Framework (SMF). A state machine is a suitable architectural choice for this

problem because the bracelet's fundamental behavior is about switching between different functional modes. At any time, the device is either conserving power, verifying its location, actively tracking, evaluating a possible fall or broadcasting an emergency. Each of these operational modes enables different hardware. Encoding this as a state machine, rather than a set of independent flags, guarantees that the system is in exactly one well defined mode at all times, preventing contradictory behavior.

Figure 3.2 shows the five states of the bracelet FSM and the events that drive the transitions between them.

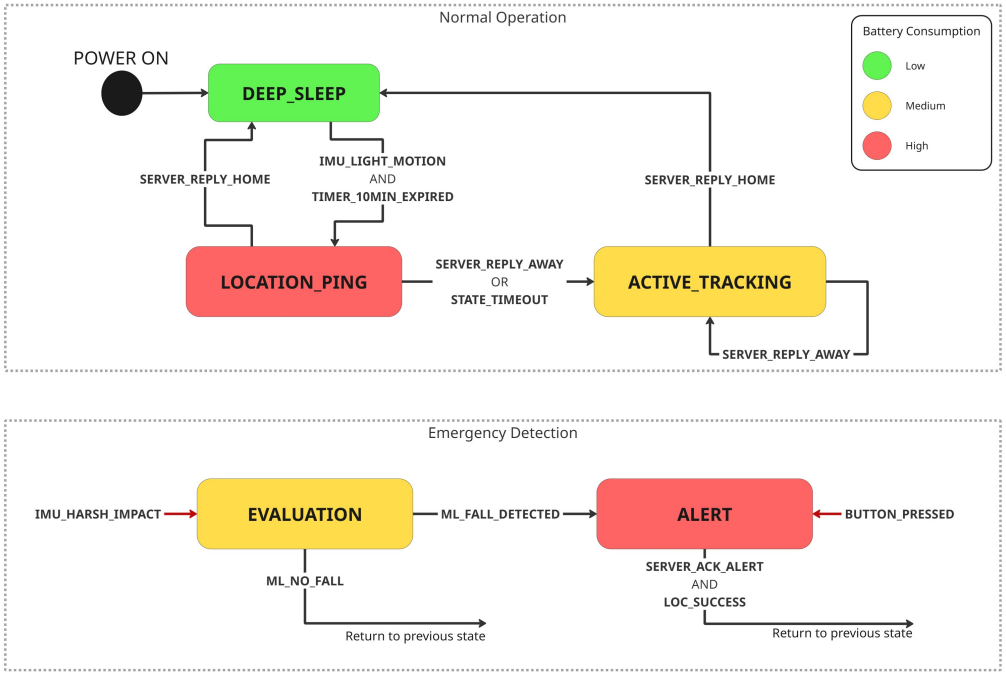


Figure 3.2: State machine governing the bracelet's operating modes.

Deep Sleep is the default, power-optimised state. While the device is located inside the Geofence, this state triggers a localization update every 10 minutes, only as long as the user is moving. The wi-fi chip, cellular and GNSS modules are powered off, and the IMU's motion-detection interrupt is armed so that the MCU itself can remain asleep. On the very first boot, the motion trigger is enabled immediately, while on subsequent entries, the state starts a 10-minute cooldown timer before re-arming the trigger.

A light motion event transitions the FSM to Location Ping, and a harsh impact event bypasses this state entirely and transitions directly to Evaluation. If the device is entering Deep Sleep after a fall evaluation, the remaining cooldown time is restored rather than restarted, evaluations do not reset the 10 minutes timer.

Location Ping briefly wakes the network stack and the localization modules (Wi-Fi and GNSS), provides data to the server and awaits a response about the wearer's position (in or out of the geofence). A 270-second timeout guards this state: if no verdict is received in time, the FSM fails safely by transitioning to Active Tracking rather than remaining stuck. A "home" verdict returns the device to Deep Sleep, while an "away" verdict promotes it to Active Tracking.

Active Tracking issues a location update once every three minutes for as long as the

server continues to report the wearer as away. As soon as the server reports the wearer as home, the FSM returns to Deep Sleep.

Evaluation is entered when the IMU reports a harsh impact. Capturing the surrounding 4.5-second motion window and running the on-device TinyML classifier takes roughly 5 seconds of blocking work, so this is offloaded to a background work item rather than run on the FSM thread itself. This choice guarantees that the panic button can still override the FSM (transitioning directly to Alert) while a fall is still being evaluated. A negative classification returns the FSM to whichever state it was in before the impact, while a positive classification transitions to Alert.

Alert is the emergency state, entered either by a confirmed fall or by a panic button press (repeated button presses while already in Alert are ignored to avoid restarting an in-progress emergency sequence). On entry, the buzzer and red LED are activated and an alert message is sent to the backend immediately, then re-sent every 60 seconds until acknowledged. Leaving this state requires *both* a server acknowledgement of the alert *and* a successful localization fix, since a caregiver receiving an alert without an updated location is of limited use.

Two design properties are present across the whole state machine. First, every state that depends on an external reply (Location Ping, Alert) has an explicit timeout that resolves to a safe default, so the bracelet can never become permanently stuck waiting on the network.

Second, state transitions are triggered exclusively by events delivered over the ZBUS channel described above, meaning the FSM has no direct knowledge of sensor drivers or radio protocols, and only reacts to the event flags. This separation is what allows the hardware and communication modules discussed in Section 3.2 to be modified or replaced without requiring changes to the decision logic itself.

3.3.4 Localization Methods

Wi-Fi positioning is used as the primary localization method because it offers a favorable balance between positioning accuracy, energy consumption, and availability in urban environments. When sufficient Wi-Fi access points are detected, the bracelet transmits the collected MAC addresses and RSSI measurements to the backend. The backend subsequently queries the Google Geolocation API and returns the estimated geographic coordinates.

Delegating Wi-Fi localization to the backend rather than performing it directly on the bracelet significantly simplifies the embedded firmware by avoiding the implementation of HTTPS communication, API request handling, and JSON response parsing on the resource-constrained device. Furthermore, the Google Geolocation API credentials remain securely stored on the backend instead of being embedded in the firmware, preventing unauthorized use should the firmware be extracted from the device.

If Wi-Fi positioning is unavailable or fails to yield a valid estimate, the system falls back to Assisted GNSS (A-GNSS). Instead of keeping the receiver active for several minutes to slowly download orbital data directly from satellites, the bracelet sends its connected cellular tower ID to the backend. The backend, which securely manages the nRF Cloud API credentials in the same manner as the Google API keys, queries the nRF Cloud and returns A-GNSS assistance data. This compact payload contains precise satellite trajectories (ephemerides) and clock corrections, allowing the modem to rapidly lock onto overhead satellites and compute a precise geographical fix in seconds.

However, if the backend fails to deliver this assistance data, the device seamlessly defaults to standard autonomous GNSS, extending its search timeout to accommodate the slower satellite download. Finally, if both Wi-Fi and GNSS fail entirely, basic cellular network information is used by the backend to generate a location estimate. This cascading architecture maximizes localization reliability while strictly minimizing energy consumption.

3.3.5 Geofence Setup and Evaluation

The system implements a single static geofence representing the user's home location. The caregiver defines this geofence through the mobile application by selecting its center coordinates and radius. The bracelet adapts its localization update frequency according to whether it is inside or outside this area in order to reduce energy consumption.

Geofence evaluation is performed entirely by the backend. Since the caregiver configures and modifies the geofence through the mobile application, the backend always maintains the latest geofence parameters. After each localization update, the backend determines whether the bracelet is inside or outside the configured area and returns the corresponding state to the device. The bracelet then adjusts its operating state accordingly.

Performing geofence evaluation on the backend eliminates the need to synchronize geofence parameters with the bracelet whenever the caregiver modifies the configuration. It also keeps the embedded firmware independent of future changes to the geofencing algorithm while enabling additional features, such as multiple geofences or more advanced region definitions, to be introduced without requiring firmware updates.

4 Hardware Implementation

This chapter outlines the practical implementation of the wearable device's hardware. This section details the integration of the nRF9151 SiP with the selected peripheral components, including the PMIC, environmental sensors, and external antennas. It covers the specific schematic designs, power routing, and matching networks required for multi-radio coexistence. Furthermore, it addresses the physical implementation, detailing the PCB stackup, component placement, and the manual hardware corrections necessary to resolve manufacturing errors and optimize the final prototype enclosure.

4.1 System Architecture Overview

This section provides a holistic overview of the hardware architecture. Figure 4.1 illustrates the core components of the system and their respective interconnections.

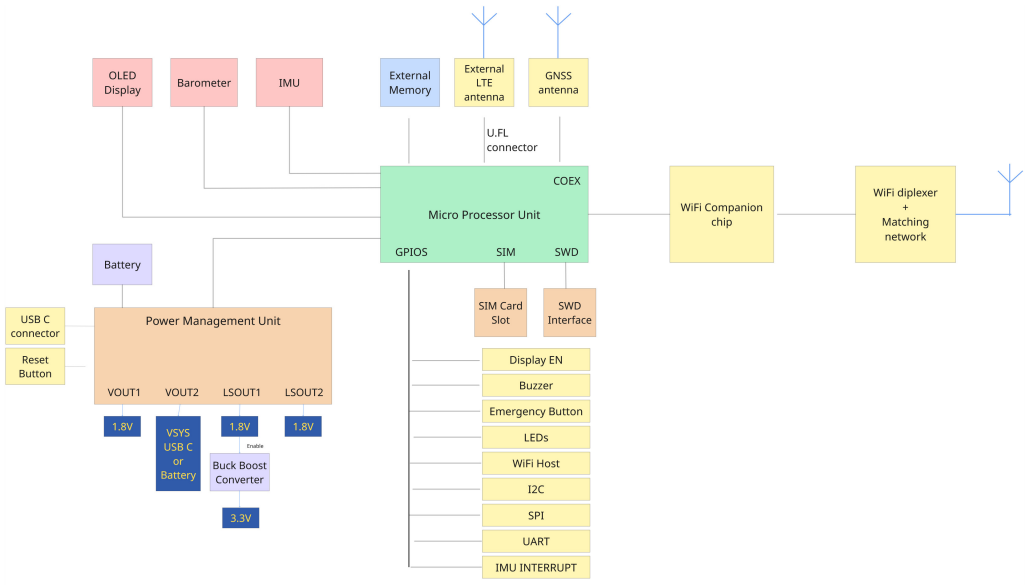


Figure 4.1: Hardware Architecture Block Diagram

The nRF9151 SiP serves as the central processing unit and coordinates all system operations. It manages the localization pipeline through its internal cellular modem and leverages its processors to enable seamless transitions between GNSS and WiFi scanning. Furthermore, it interfaces with all peripheral sensors via SPI and I2C buses while managing the UART and SWD debugging lines. The system is entirely powered by the Power Management Integrated Circuit (PMIC) which regulates all required voltage rails.

As stated previously, even though this project builds on a previous iteration, several significant modifications were implemented to support the new system requirements. A barometer, an OLED display, a buck boost converter, and an external LTE antenna were introduced into the architecture. Additionally, the buzzer and WiFi companion chip circuits, the GNSS low noise amplifier, and the overall layout of the printed circuit board layout were modified in order to improve the previous design.

4.2 Power Management Unit

The power unit is supplied by either a USB Type C connection providing 5 V or the internal battery providing a variable voltage. The system prioritizes the USB connection, seamlessly switching the system load to the USB power path while charging the battery. The PMIC generates four primary outputs utilizing two buck converters and two low drop out regulators. The first buck converter outputs a stable 1.8 V rail. The second buck converter generates the main system voltage.

An internal power switch routes these two buck outputs to generate the switched 1.8 V and switched system voltage rails. These switched lines subsequently serve as the input sources for the two low drop out regulators. The output of the first regulator is connected directly to the enable pin of the external buck boost converter, acting as a hardware logic signal to activate or deactivate the component. The second regulator provides a dedicated supply voltage for the inertial measurement unit.

To manage battery safety, the PMIC is designed to read a negative temperature coefficient thermistor (NTC) attached to the battery cell which is useful to preserve battery health and monitor its temperature. In this design, although this component is not implemented, two solder bridges act as placeholders for future iterations. However, if the NTC is left disconnected and shorted to ground, the PMIC reads infinite resistance, assumes a critical thermal fault, and completely disables battery charging. To prevent this, a standard $10k\Omega$ resistor is attached to the sensor line. This hardware modification forces the PMIC to register a permanent and safe 25 degrees Celsius, allowing the charging sequence to operate normally.

Table 4.1 summarizes the distribution of these generated power rails across the system components.

Table 4.1: System Power Rails and Component Distribution

Power Rail	**Source**	**Powered Components**
VIN_WIFI	Buck Boost Converter	WiFi IC, Buzzer, RGB LED
VSYS_SW	PMIC Buck 2	LDO 1, nRF9151 Core, Display Power, Buck Boost Input
1V8_SW	PMIC Buck 1	SWD, Display Logic, WiFi Logic, nRF9151 GPIOs, Barometer, GNSS LNA, Flash Memory, RF Control Logic
VIN_BMI270	PMIC LDO 2	Inertial Measurement Unit

Finally, the specific battery charging parameters are defined in the DeviceTree software configuration. To prevent system brownouts during boot sequences, which happened in the previous design, the total USB input current limit is set to 1000 mA. However, to safely charge the small wearable battery without causing thermal degradation, the actual charging current is strictly restricted to 150 mA (I_{CHG}), and the charging cycle terminates when the cell reaches 4.2 V (V_{TERM}). Figure 4.2 illustrates this standard charging profile executed by the PMIC.

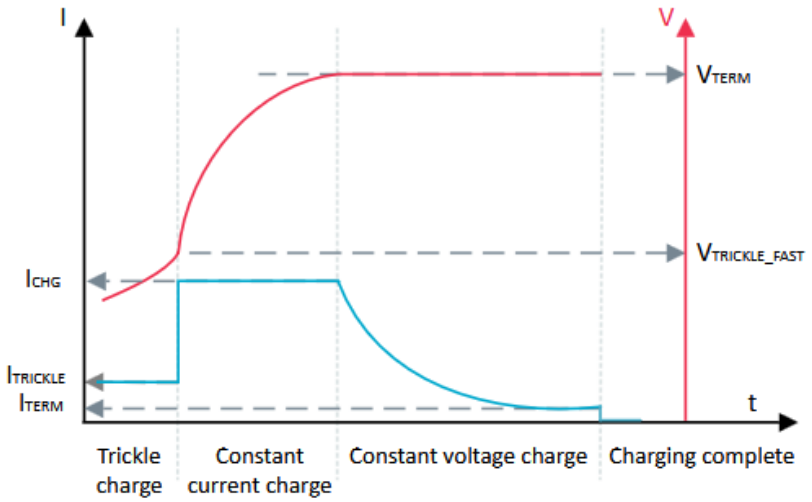


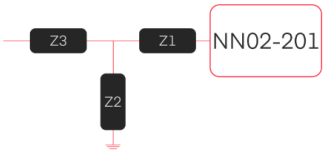
Figure 4.2: Standard Li-ion charging profile executed by the nPM1300 PMIC (Adapted from Nordic Semiconductor)

4.3 RF and Antenna Design

The transition to an external flexible antenna for the LTE-M module allowed for the complete removal of the extensive keep-out zone previously required by the onboard cellular antenna. Consequently, the overall board dimensions were successfully reduced by 9% from 46x39 mm to a more compact 42x39 mm footprint, while creating space for the additional components of this iteration.

This size reduction optimized the wearable's integration and visual aspect but it affected the radio frequency characteristics of the remaining surface-mount antennas. Changing the physical dimensions of the board shifts the resonant frequencies of the WiFi and GNSS antennas. During the initial prototyping phase, the matching networks from the larger board were maintained. The impacts of this configuration are evaluated in Section 4.6.2.

The corrected matching network topologies necessary to tune these antennas to the updated board dimensions were simulated using Ignion's AI powered tool, Oxion. For the GNSS module, Figure 4.3a outlines the required matching circuit topology, while Figure 4.3b specifies the exact Murata inductor values needed to tune the antenna. Similarly, the WiFi matching circuit topology is illustrated in Figure 4.4a, using a specific combination of capacitors and inductors detailed in Figure 4.4b to cover both the 2.4 GHz and 5 GHz bands.

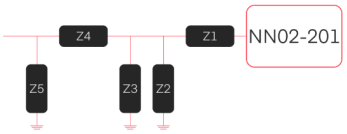


(a) Proposed topology (v2.0)

Comm. Standard	Component	Value	Part Number	Manufacturer
GNSS	Z1	10.0nH	LQW18AN10NG80	Murata
	Z2	13.0nH	LQW18AN13NG80	Murata
	Z3	11.0nH	LQW18AN11NG80	Murata

(b) Component values

Figure 4.3: GNSS antenna matching network design.



(a) Proposed topology (v2.0)

Comm. Standard	Component	Value	Part Number	Manufacturer
Wi-Fi	Z1	1.8pF	GJM1555C1H1R8WB01	Murata
	Z2	12.0nH	LQW18AN12NG80	Murata
	Z3	5.6nH	LQW18AN5N6G80	Murata
	Z4	0.2pF	GJM1555C1HR20WB01	Murata
	Z5	0.1pF	GJM1555C1HR10WB01	Murata

(b) Component values

Figure 4.4: WiFi antenna matching network design.

4.3.1 Impedance Control and Routing

To guarantee maximum power transfer and minimal signal reflection, all RF transmission lines connecting the modems, matching networks, and antennas must maintain a standardized 50 Ω impedance. Because the layer stackup from the previous proven design was retained, the trace widths and coplanar waveguide gaps remain unchanged (see Figures A.1 and A.2 in Appendix A).

4.4 Digital Interfaces and Peripherals

To interface the nRF9151 main processor with the external sensors, memory, and user interface components, shared communication buses and dedicated general purpose input output (GPIO) pins were implemented.

4.4.1 Communication Buses and Memory

Data exchange across the board relies primarily on I2C and SPI protocols. The I2C bus is shared among the barometer, the inertial measurement unit (IMU), the PMIC, and the OLED display. To ensure stable signal integrity across these devices, standard 10 $k\Omega$ pull up resistors were implemented on the serial data and clock lines.

The SPI bus is utilized for high speed communication with the Wi-Fi companion chip and the newly added 8 megabyte NOR flash memory. The latter was integrated using a standard 3-wire SPI configuration (MISO, MOSI, and SCK). Because the memory IC also supports Quad-SPI (QSPI), the unused auxiliary data lines, along with the Chip Select and Reset pins, were securely tied to the 1.8 V rail using 10 $k\Omega$ pull up resistors to prevent floating inputs and ensure stable operation.

4.4.2 WiFi Companion Chip

The nRF7002 physical implementation, including its dedicated 40 MHz crystal oscillator, was carried over from the previous design with a small change reported to cause system brownouts. In the previous design, Pin 32 PWRBUCKVDD, the voltage provider of internal power supply, was wrongfully connected to ground through a 0.47 µF capacitor instead of being connected to VDD_BUCK. Inspection of the correspondent reference design evidenced this mistake which was corrected.

4.4.3 Sensors and Display Integration

The physical implementation of the BMI270 IMU was carried over directly from the reference design. However, the device's sensing capabilities were expanded by introducing the ICP20-100 barometer. This sensor sits on the I2C bus at address 0x63. It is powered

via the 1.8 V rail, with 100 nF decoupling capacitors placed near its supply pins to filter noise. A hardware interrupt line was also routed to the MCU to allow for future firmware optimizations, although it remains unused in the current application state.

The OLED display integration required specific power switching logic. Because the display panel draws current from the higher voltage VSYS_SW rail while the MCU control logic operates at 1.8 V, a discrete load switch was implemented using an N-Channel and a P-Channel MOSFET pair. The MCU toggles the N-Channel gate, which subsequently drives the P-Channel to safely connect or disconnect the main display power. Additional GPIOs were routed for dedicated hardware reset and enable signals, alongside standard decoupling passive components on the 1V8_SW logic supply.

4.4.4 User Interface and Connectivity

The physical user interface consists of an RGB LED, a push button, and a piezoelectric buzzer, all driven by standard MCU GPIOs. The application circuitry for these components remains identical to the baseline, with one minor manufacturing correction: the buzzer's driving transistor was updated to the originally intended NPN SOT-323 package, which was previously unavailable due to supply chain constraints.

Finally, the externally facing connectors, including the SIM card holder, the USB Type C port, and the hardware debugging interfaces (UART and SWD), were implemented using the validated reference schematics.

4.5 PCB Layout, Routing and Manufacturing

Translating the system schematics into a physical prototype requires attention to mechanical constraints, thermal management and radio frequency guidelines. Figures 4.5 and 4.6 illustrate the final component placement and routing on the top and bottom layers of the manufactured board, with corresponding numbered legends for the main hardware blocks. The complete Bill of Materials, detailing all passive and active components utilized in this prototype, is provided in Appendix B.

4.5.1 Stackup and Component Placement

To support the high routing density required by the application and maintain RF impedance control, a 6-layer printed circuit board stackup was retained from the baseline design. This configuration provides dedicated internal ground planes, which are essential for shielding sensitive digital buses from high-frequency cellular interference and providing a solid reference for the antennas. The exact layer stackup and dielectric properties utilized for manufacturing are detailed in Table 4.2.

Both top and bottom sides were populated with components to optimize space. Regarding passive component packaging, a mixing of 0402 footprint sizes and 0201 were utilized throughout the design. Even though one of the objectives of this project was to reduce the wearable's footprint, for a first prototype where debugging and hardware fixes are guaranteed, maintaining a good balance in passive components' size is crucial.

4.5.2 Thermal Management and EMI Mitigation

Thermal vias were placed directly beneath high-power components (modems, PMIC) to dissipate heat evenly across the internal ground planes and ensure safe usage. Additionally, grounded via shielding along the RF transmission lines creates strict signal return

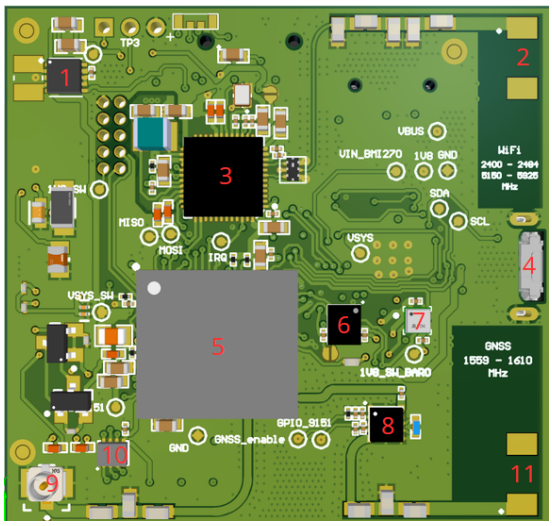


Figure 4.5: Top Side of the PCB

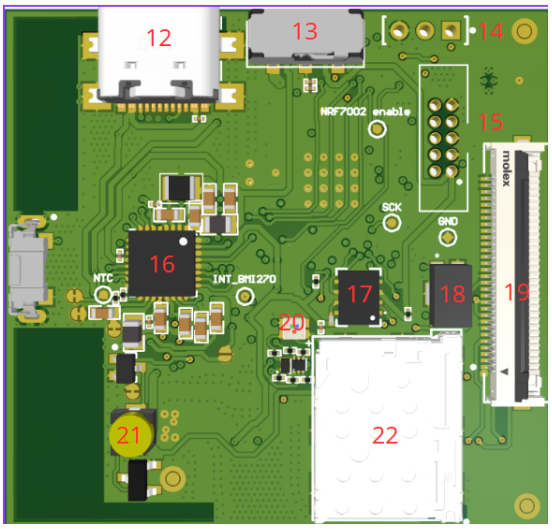


Figure 4.6: Bottom Side of the PCB

Top Side Legend:
1. Buck Boost Converter
2. Wi-Fi Antenna
3. Wi-Fi Chip
4. Emergency Button
5. MCU
6. Accelerometer
7. Barometer
8. Low Noise Amplifier
9. LTE Antenna Connector
10. SIM Card Interface
11. GNSS Antenna

Bottom Side Legend:
12. USB-C Connector
13. ON/OFF Switch
14. UART Interface
15. SWD Interface
16. Power Management Unit
17. Flash Memory
18. Reset Button
19. Display
20. RGB LED
21. Buzzer
22. SIM Card Connector

paths, significantly reducing electromagnetic interference (EMI) between cellular emissions and sensitive onboard sensors.

4.5.3 Manufacturing and Testability

JLCPCB was selected for manufacturing, and the board was designed to meet their automated assembly constraints. To facilitate printed circuit board assembly (PCBA), global fiducial markers (unmasked copper dots) were placed in the board's corners, allowing optical Pick-and-Place machinery to precisely align the miniature surface-mount components.

Moreover, 18 dedicated exposed copper test points were integrated into the layout. These allow for safe multimeter and oscilloscope probing and are distributed as follows:

- **Power Rails:** VBUS, VSYS, VSYS_SW, 1V8, 1V8_SW, VIN_BMI270, VIN_WIFI
- **Digital Buses:** SDA, SCL, SCK, MISO, MOSI
- **Control Logic:** INT_BMI270, GNSS_EN, VDD_9151, WIFI_IRQ
- **Ground:** Two dedicated GND pads (one on each side of the board)

Table 4.2: 6-Layer PCB Stackup and Material Properties

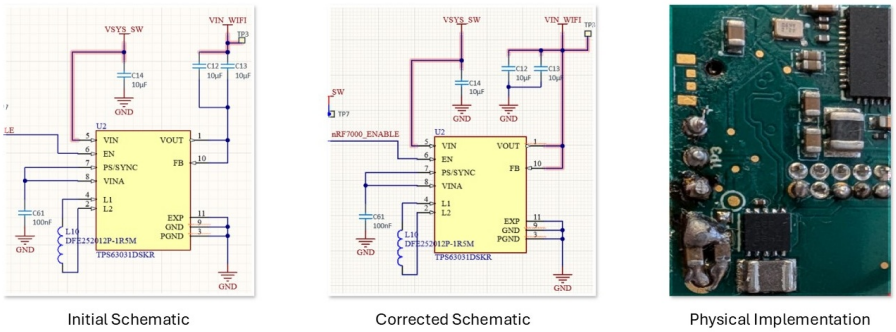
Layer	Type	Copper Weight	Thickness (mm)	Dielectric Constant (Dk)
Top Layer	Signal	1 oz	0.035	-
Prepreg 1	Dielectric	-	0.0994	4.10
Internal 1	Ground	0.5 oz	0.0152	-
Core 1	Dielectric	-	0.5500	4.60
Internal 2	Signal	0.5 oz	0.0152	-
Prepreg 2	Dielectric	-	0.1088	4.16
Internal 3	Ground	0.5 oz	0.0152	-
Core 2	Dielectric	-	0.5500	4.60
Internal 4	Signal	0.5 oz	0.0152	-
Prepreg 3	Dielectric	-	0.0994	4.10
Bottom Layer	Signal	1 oz	0.035	-

4.6 Hand soldered components and adjustments

4.6.1 Buck Boost Converter PCB Schematic Issues

During initial testing of the PCB, issues with the WiFi chip booting and initialization were identified. In some cases, simply powering on the WiFi chip caused the board to brown out. After debugging, an issue in the buck-boost converter schematic was found: the output decoupling capacitors C12 and C13, placed on VIN_WIFI (the regulated 3.3 V output supplying the WiFi chip), were missing their connection to ground. The output capacitors are an essential part of the converter's output filter, providing local energy storage and maintaining a stable output voltage by supplying current during the converter's switching intervals and during rapid load changes. Without a return path to ground, C12 and C13 were unable to store or deliver charge. As a result, the converter was unable to maintain a stable 3.3 V output during the WiFi chip's power-up inrush current, leading to significant output voltage droop.

This issue can be seen in Figure 4.7, which shows the initial schematic, the correction and the manual hardware fix. The physical fix involved rotating the capacitors, exposing the ground plane by scratching the surface of the PCB, and soldering the original terminals directly to this exposed ground copper.



Initial Schematic | Corrected Schematic | Physical Implementation

Figure 4.7: The manual fix required for the Buck Boost Converter

4.6.2 GNSS Antenna

As previously explained in Section 3.2.9, the PCB uses Ignion mXTEND antenna boosters for the WiFi chip and for the GNSS receiver. Because the PCB dimensions were reduced without re-tuning the matching network, the electrical characteristics of the radiating structure changed. Consequently, the original matching network no longer provided an appropriate impedance transformation, resulting in poor impedance matching between the GNSS receiver and the antenna, significantly reducing the received signal strength.

In order to solve this issue, the matching network components could have been re-tuned to values determined empirically with a Vector Network Analyzer, correcting the impedance mismatch. Alternatively, the antenna booster could be replaced entirely with an external antenna, bypassing the ground-plane-dependent design altogether.

The second option was chosen, as shown in Figure 4.8, where a wire of approximately 4.7cm was soldered to the terminal of capacitor L5, effectively bypassing the original antenna booster. A wire antenna behaves like an efficient receiver only if its physical length matches a fraction of the wavelength of the signal. For the GNSS C/A frequency of 1575.42 MHz, the required wire length was determined using the quarter-wave monopole formula [53], where the full wavelength is

$$\lambda = \frac{c}{f} = \frac{300{,}000{,}000}{1{,}575{,}420{,}000} \approx 0.1904\,\text{m} = 19.04\,\text{cm},$$

and the quarter-wave monopole length is

$$L_{\text{quarter}} = \frac{\lambda}{4} \approx 4.76\,\text{cm}.$$

A quarter-wave monopole was selected because it is the shortest practical wire antenna that is naturally resonant at the GNSS operating frequency, providing efficient reception with only a simple conductor.

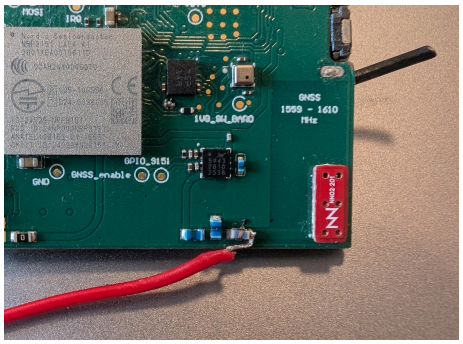


Figure 4.8: GNSS quarter-wave monopole antenna

4.6.3 Hand Soldered Components

Several components were installed manually after PCB assembly for different reasons. The Ignion NN02-201 antenna boosters for the GNSS and WiFi front ends were not available in the LCSC component library during manufacturing and therefore had to be sourced separately and hand soldered onto the PCB.

Similarly, the Molex LTE antenna and the OLED display were also acquired independently. However, both components connect through dedicated connectors already present on the PCB, allowing them to be installed without additional soldering.

In addition, JLCPCB reported that the emergency button could not be assembled because a small protrusion in the PCB outline prevented the component from fitting correctly. The excess PCB material was therefore removed manually, after which the button was soldered onto the board. Finally, the SEGGER 8-pin Cortex Debug Connector was intentionally excluded from the PCB assembly process. Since this connector is inexpensive and readily available, it was hand soldered to reduce the overall manufacturing cost.

4.6.4 OLED Display Connector Pin Mapping Issue

During hardware validation, the OLED display was found to be non-functional despite correct software configuration and power delivery. Further investigation revealed that the issue originated from an incorrect pin mapping during PCB design.

The selected FFC/FPC connector footprint contained a reversed pin numbering orientation compared to the physical display connector. When the schematic symbol was connected in Altium, the pin order was therefore mirrored, resulting in all display signals being routed to the opposite pins on the PCB. Since the connector footprint appeared correct during the PCB layout stage, the error was not detected before manufacturing.

As a consequence, signals such as power, ground, I2C communication and control lines were connected to incorrect pins of the OLED module. Correcting this issue would require ordering a new PCB or extensive hardware modification, making it impractical for the current prototype.

To maximize the functionality of the prototype despite this limitation, an alternative feedback mechanism was implemented using the onboard RGB LED. The LED was integrated into the firmware state machine and provides visual indications for initialization, operating states, localization procedures, and fall detection outcomes.

The implemented LED feedback scheme is summarized in Table 4.3.

Table 4.3: LED feedback implemented as a replacement for OLED indications.

LED Pattern	Meaning
Red and blue alternating	Firmware initialization
Two green flashes	Entry into Deep Sleep state
Two cyan flashes	Entry into Active Tracking state
Yellow blinking	Localization in progress
Solid red	Alert state active
Blue	Fall detection machine learning evaluation started
Green for 2 seconds	Fall classification completed: no fall detected

4.7 Case Design

The enclosure was designed using Shapr3D CAD software and manufactured through 3D printing. The primary design objective was to satisfy the size and comfort requirements defined by Requirements R13 and R14, resulting in a compact form factor suitable for continuous daily wear.

The internal layout was designed to securely accommodate the PCB, battery, and LTE antenna while preventing component movement during normal use. Openings were incorporated to provide access to the emergency button, power switch, charging connector, and debug interface. An extension to the emergency button was also integrated to improve access.

Because the OLED display could not be used due to the connector issue described previously, the enclosure was modified to expose the onboard RGB LED. A small translucent plastic lens was added above the LED to improve the visibility of the visual feedback while maintaining the overall appearance of the device.

The final CAD model and the manufactured enclosure are shown in Figure 4.9.

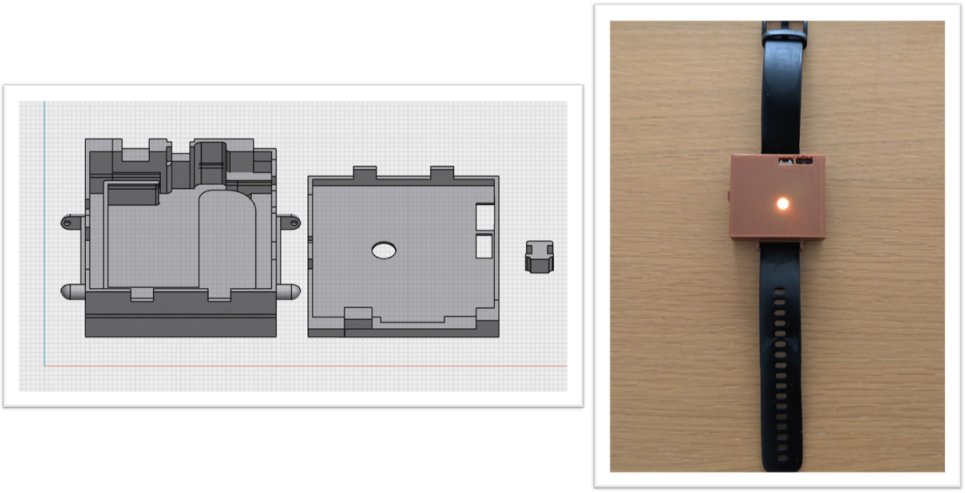


Figure 4.9: Case 3D model and manufactured result

5 Software and Algorithm Implementation

This chapter describes the software implementation required to transform the hardware prototype into a fully functional fall detection and tracking ecosystem. The development is structured across three core sections: the edge, the cloud, and the client. First, the embedded software section explains the Zephyr RTOS firmware, sensor integration, and on-device machine learning execution. Second, the cloud backend section describes the Python-based server that manages data routing between the devices using MQTT and REST APIs. Finally, the mobile application section outlines the Flutter client used by caregivers for real-time tracking, geofencing, and receiving emergency push notifications.

Source Code Availability

The complete source code developed during this project is publicly available on GitHub. The firmware running on the wearable device is available at:

- Bracelet Firmware: https://github.com/tomasvenda/bracelet
- Caregiver Mobile Application: https://github.com/andreiFarcas/caregiver_app

The repositories contain the firmware, mobile application, hardware configuration files, and supporting source code required to reproduce the implementation presented in this thesis.

5.1 Embedded Software Development

While section 3.3.3 justified why the firmware is organised as an event-driven state machine, this section describes how that architecture is implemented on the nRF9151 chip, covering the drivers, threads and timing behind each of the five states shown in Figure 3.2.

5.1.1 Threading Model and Event Bus

The firmware is split across three Zephyr threads plus the system workqueue, each mapped to one functional role: a sensor thread that turns IMU interrupts into events, an FSM thread that owns the state machine and contains no blocking I/O of its own, a localization thread that performs the Wi-Fi/GNSS/LTE fallback sequence, and the system workqueue, onto which the TinyML evaluation and the buzzer's beep pattern are offloaded so that neither blocks the FSM thread. All four communicate exclusively through the `fsm_events_chan` ZBUS channel. No thread calls into another thread directly (except for the FSM).

5.1.2 Localization Execution

The tiered localization strategy described in Section 3.3.3 is implemented as a single blocking sequence, `perform_localization_work()`, which runs on a dedicated localization thread so that it never stalls the FSM. Due to our external cellular antenna, the Wi-Fi scan can happen while LTE is on. The sequence first powers the Wi-Fi chip by turning on the LDO1 which enables the buck converter that supplies power to the Wi-Fi chip, performs a scan, and immediately powers it back down before ensuring the LTE and MQTT

connections are still up. The resulting access-point list is published to the backend, and the thread blocks on a Zephyr kernel event object waiting for either a resolution success/-failure flag or a final home/away verdict, each bounded by a 10-second timeout.

If no access points are found, or the server fails to resolve them, the sequence falls through to A-GNSS: the modem is switched to GNSS operating mode, A-GNSS information is retrieved from the server, a fix is attempted, and the outcome (a coordinate pair, or a further fallback to raw LTE cell-tower identifiers if the fix also fails) is published to the backend the same way. Whichever branch succeeds, the thread reports the final outcome back to the FSM as `EVENT_LOC_SUCCESS` or `EVENT_LOC_FAILURE`, which is what the Alert state waits on before it is allowed to exit (Section 3.3.2).

5.1.3 Sensing and Fall-Window Capture

The data collection pipeline relies on an accelerometer and a barometer, both configured to balance high resolution with low power constraints. The accelerometer operates at a 50 Hz sampling rate with a 2g range. By running in performance mode using only the three accelerometer axes, it consumes just 210 microamps. The gyroscope data was deliberately excluded since the machine learning model was trained on a comprehensive dataset where accelerometer features perfectly described any kind of fall event. Activating both the accelerometer and gyroscope would increase current consumption to 970 microamps, which conflicts with the battery life requirements of the application and adds unnecessary complexity to the model.

In order to track elevation changes during a fall, the pipeline also incorporates an ICP-20100 barometer. This sensor supports five operating modes with various precision and power trade offs. Thus, the high precision and low power requirements led to the selection of MODE0. This configuration yields a 25 Hz sampling rate at 211 microamps.

Rather than continuously evaluating this sensor data, described as polling, and draining the battery, the system relies on a hardware interrupt tuned to detect the free-fall phase that typically precedes a harsh impact. This ”low-g interrupt” is configured by defining a specific low-acceleration threshold and a minimum time duration the device must remain below that threshold to filter out brief movements. These values where fine tuned in the testing phase so that the device was neither too sensitive nor too unresponsive. By triggering this interrupt, the IMU sends an event to the state machine, instantly transitioning the system from a low power mode into the active evaluation phase.

Once in the evaluation phase, the algorithm captures a 4.5 second window that combines historical pre impact data with real time post impact data to accurately classify the event. For the pre impact accelerometer readings, the system retrieves 1.5 seconds of historical data directly from the hardware buffer of the BMI270, which automatically discards its oldest data without freezing. However, the internal buffer of the barometer is limited to 16 samples, meaning it fills in approximately 600 milliseconds at 25 Hz and then freezes. To limit the implications of this hardware constraint, a software ring buffer was designed to hold 2.0 seconds of historical pressure data. Had this ring buffer not be implemented, Table 5.1 compares the hardware constraints for both sensors. For the BMI270, the FIFO buffer can hold a maximum of 437.2 seconds of data while the ICP-20100 can only store 2 seconds.

Maintaining this software buffer requires the microcontroller to wake every 400 milliseconds to fetch data and replenish the memory. Although waking the microcontroller introduces a power penalty, it is a necessary trade off to guarantee the accuracy of the fall

Table 5.1: Sensor FIFO constraints based on ODR

Sensor (FIFO Size)	ODR (Output Data Rate)	Time in the Past
ICP-20100 (16 samples)	2 Hz (Mode 3)	8 s
	25 Hz (Mode 0)	640 ms
	40 Hz (Mode 2)	400 ms
	120 Hz (Mode 1)	133.3 ms
BMI270 (341 samples)	0.78 Hz (Low Power Min)	437.2 s
	12.5 Hz (Normal Min)	27.3 s
	400 Hz (Low Power Max)	852.5 ms
	1600 Hz (Normal Max)	213.1 ms

detection algorithm. When an impact occurs, the software extracts this stored history and matches the 1.5 seconds of accelerometer history. After synchronizing this past data, the system captures an additional 3.0 seconds of post impact data in real time, completing the full evaluation window required by the model.

5.1.4 Data collection and model training

To perform data collection, a test program with the same hardware interrupt implemented in the final project was developed and deployed directly on the device. The RGB LED provided visual feedback during testing and the physical button was used to pre label the events. Pressing the button marked the event as a fall, while not pressing it marked it as a normal event. This setup made data acquisition highly efficient by allowing multiple scenarios to be tested and labeled in a single continuous run. The flash memory allowed for the storage of up to 60 events per collection session. Once the memory was full, the data was dumped over a serial connection and processed with a Python script. This script organized the raw data into correctly structured and timestamped files required for training the machine learning model.

Because the barometer runs at 25 Hz and the accelerometer runs at 50 Hz, linear interpolation was applied to align the pressure readings with the 20 millisecond intervals of the accelerometer. The interpolation calculates the pressure at any intermediate time by taking the previous known pressure and adding the proportional change towards the next known pressure. This relationship is defined mathematically as:

$$p_i = p_j + f \times (p_{j+1} - p_j) \tag{5.1}$$

where p_i is the interpolated pressure, p_j is the previous recorded pressure, and f is the fractional time distance between the two recorded samples.

After aligning the timestamps, a new feature, called pressure delta, was calculated. This metric measures the true difference in pressure between the current interpolated sample and the very first sample of the capture window. The second feature is the pressure step, which represents the overall elevation change of the entire event. This step is calculated by taking the median pressure from the end of the window and subtracting the median pressure from the beginning of the window.

By including these calculated metrics, the model places more emphasis on total elevation changes, which testing proved to significantly increase accuracy. Ultimately, this data is

formatted into a single buffer containing five columns across 225 frames, resulting in a continuous array of 1125 floating point numbers.

$$\text{Buffer} = [a_{x0}, a_{y0}, a_{z0}, dp_0, step, \quad a_{x1}, a_{y1}, a_{z1}, dp_1, step, \dots]$$

5.1.5 Edge Impulse

Edge Impulse is a widely used machine learning tool for IoT applications. It allows for easy integration with devices, live testing, and data acquisition. The data stored in the flash memory of the custom board during data collection was uploaded to this platform. In total, a dataset of approximately 20 minutes of recorded events was collected, consisting of 95 falls and 166 normal events. The fall events were performed on grass by simulating several falling motions on both wrists in order to collect a wide range of data. The normal events included every day activities like sitting, lying or walking which could possibly trigger the low-g interrupt.

The platform automatically split this data, allocating 75 percent of the falls and normal events to the training set and the remaining 25 percent to the testing set. This stratified split ensured both sets maintained the same proportional balance of event types.

Because this is time series data with timestamps, the platform adapts its model automatically to match the data. A 4500 millisecond window with a 4500 millisecond stride was chosen. This configuration ensures every event is treated as a single complete block instead of being subdivided. This approach is highly relevant because it captures all three phases of a fall. Subdividing the data would cause the model to learn incomplete parts of a fall event rather than the entire process. The complete process is what separates a true fall from a normal event. A typical fall behavior illustrating these three phases is shown in Figure 5.1.

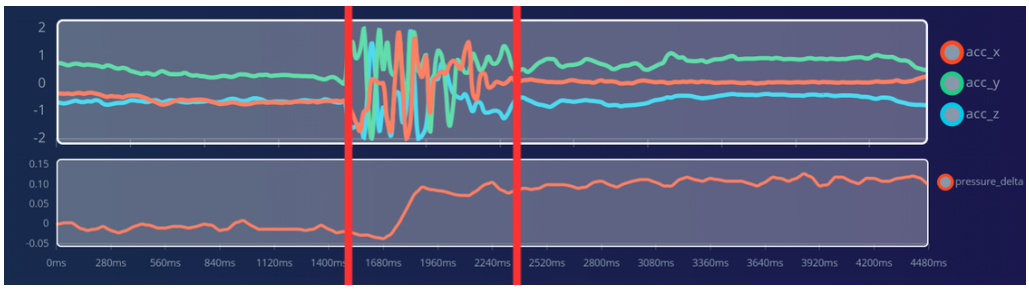


Figure 5.1: Accelerometer and pressure delta behavior during a fall event

Further data processing was performed inside Edge Impulse using three analysis blocks. These included a spectral analysis block for the accelerometer data and a flatten block for each of the two pressure features. Spectral analysis extracts frequency and power characteristics from a signal, helping to identify the intensity and rhythm of movements. The autotune parameters tool was used to analyze the training data and select the best configuration. As expected, the tool chose a wavelet analysis over a standard fast Fourier transform. Wavelets are better suited for this application because accelerometer data from a fall is composed of sudden non periodic impacts, whereas a fast Fourier transform assumes a continuous periodic signal. A high pass filter was also applied to reject very slow variation data, which is not relevant to the harsh impact of a fall. Normalizing this data resulted in clear cloud separation among the different classes, as seen in Figure 5.2.

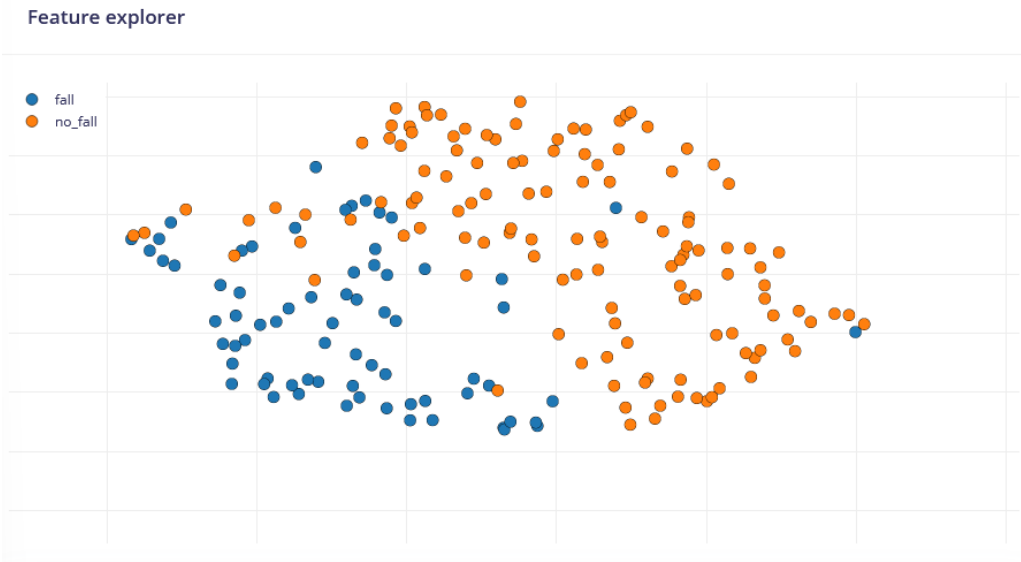


Figure 5.2: Spectral analysis performance on training data

The flatten blocks generate new features from the raw data, such as minimum, maximum, average, and standard deviation. All of these statistical features were generated for the pressure delta. For the pressure step, only the minimum value was used. This is because the pressure step is a constant scalar value repeated across all timestamps for a single event, but the pipeline requires at least one extracted feature to feed the model.

With all processing blocks performing well and successfully differentiating events, the classifier was ready for training. Because the pre processing was highly effective, the classifier did not need to be highly complex. A simple neural network was sufficient to achieve good accuracy. This network consists of an initial input layer of 89 features, two hidden layers with 20 and 10 neurons respectively using rectified linear unit activation functions, and an output classification layer using a softmax activation function. Training this classifier over 70 epochs with a learning rate of 0.0005 achieved a 97.4 percent accuracy on the training set with a loss of 0.07. On the testing set, the model performed exceptionally well. The resulting confusion matrix is presented in Table 5.2.

Table 5.2: Confusion Matrix for the Fall Detection Model on the Testing Set

Actual / Predicted	Predicted: Fall	Predicted: No Fall
Actual: Fall	22	1
Actual: No Fall	0	41

This performance translates to a 98.4% accuracy on the test data. A deeper analysis of the confusion matrix reveals a 100 percent precision rate, meaning there was a complete absence of false positives. This metric is critical for a fall detection system, as false positives trigger wrongful emergency states that drain battery life and inadvertently notify caregivers. The single misclassified event in the testing set was a false negative, where an actual fall was predicted as a normal event. This specific error highlights a known limitation in fall detection research, where simulated falls can mislead the model because anticipating an impact subtly alters natural body mechanics. To mitigate this anticipation effect during data collection, subjects performed realistic falls across various directions

while wearing the device on different wrists. Furthermore, the normal events dataset included everyday motions specifically chosen because they possessed the potential to trigger the hardware interrupt. The resulting model is highly suited for this application by successfully prioritizing absolute precision over perfect recall.

Finally, this optimized model is deployed to the bracelet. The device captures data upon a harsh impact trigger exactly as it did during the data collection phase. Because the Edge Impulse model evaluates the data in just 2 milliseconds, the 4.5 second data collection window remains the longest part of the evaluation process. After inference, the software decides based on a 75% confidence threshold if it triggers a fall event.

5.1.6 Alert Signalling and Network Delivery

On entry to the Alert state, `comms_send_alert_status()` wakes the LTE modem, ensures the MQTT client is connected, and publishes a JSON status payload tagged `fall` or `panic` depending on the trigger source. This publish is repeated automatically every 60 seconds by the FSM's state-timeout timer until an acknowledgment event arrives on the responses topic, guaranteeing delivery of the emergency notification.

The buzzer and RGB LED are driven from the PWM peripheral rather than plain GPIO, which allows for the same driver calls to produce the sound pattern and the LED notifications. The audible 200 ms-on/1800 ms-off buzzer pattern is generated by a kernel timer that alternates the buzzer state and is deliberately handled through a work item rather than directly in the ISR context, since driving the PWM peripheral is not safe for IRQ (Interrupt Request) as Zephyr forbids blocking calls inside the ISR (Interrupt Service Routine) call.

5.2 Cloud Server Backend

The backend server acts as the central coordination point of the system. The following sections describe how its functions are implemented and how the different communication interfaces interact within the backend.

5.2.1 Server Infrastructure and Deployment

The backend is hosted on Microsoft Azure Cloud Services, utilizing a lightweight Linux virtual machine (Ubuntu 22.04) deployed in the Norway East region. In the attempt to maintain reduced costs, the server uses the Azure for Students free subscription, which provides a virtual machine "Standard B2ats v2" that gives access to 2vCPUs and 1GiB of physical memory.

Early during development frequent crashes were noticed due to the limited physical RAM provided by the free-tier instance. When a Linux system exhausts its RAM, it can completely freeze and drop the SSH connections. Therefore, an additional 2 GB of swap memory was configured to ensure system stability. Swap memory is a designated portion of the regular storage, that the system uses as a temporary virtual RAM, when the physical one becomes insufficient. By moving some of the inactive memory to the disk, the system frees up the physical RAM for active processes and prevents crashes.

The swap file was manually provisioned and activated using the following sequential commands:

1. **Allocate the space**

```
sudo dd if=/dev/zero of=/swapfile bs=1M count=2048
```

This creates a contiguous 2 GB file filled with zeros. A block size (`bs`) of 1 MB and a count of 2048 are used to allocate exactly 2 GB.

2. **Secure the file**

```
sudo chmod 600 /swapfile
```

This ensures that only the root user has read and write permissions on the swap file. Restricting access is important because swap space may temporarily contain sensitive application data or credentials that have been paged out of physical memory.

3. **Format the file for swap**

```
sudo mkswap /swapfile
```

This command initializes the file as Linux swap space by writing the required swap metadata and signature. The operating system uses this information to recognize the file as valid swap space when it is activated

4. **Activate the swap**

```
sudo swapon /swapfile
```

This command immediately enables the operating system to use the file as virtual memory.

5. **Verify the configuration**

```
free -m

bracelet_tracker_admin@BraceletServer:~$ free -m
               total        used        free      shared  buff/cache
Mem:             894         186         118           3         589
Swap:           2047          88        1959
```

This command displays the current memory usage in megabytes, allowing verification that the 2048 MB swap space has been successfully activated and is available for use.

5.2.2 System Architecture

The backend is implemented as a monolithic Python application running on the Azure virtual machine. Although a microservice architecture would provide improved scalability, it would also introduce unnecessary complexity for a research prototype. The monolithic approach therefore simplifies development, debugging, and deployment while satisfying the functional requirements of the system.

The backend simultaneously supports two different types of clients and therefore two communication methods. REST API for the applications and MQTT for the bracelets. Both communication interfaces operate concurrently within the same application and share access to the same data. For example, while the mobile application may request the latest bracelet status through the REST API, the bracelet may simultaneously publish a new MQTT message updating its location or reporting an emergency. Without proper synchronization, these concurrent operations could attempt to modify the database at the same time, potentially resulting in corrupted or inconsistent data.

To prevent this, all database read and write operations are protected using a global command `threading.Lock()`. Before accessing the JSON database, each thread must acquire the lock, ensuring that only one thread can modify the file at any given time while the others wait. Although this introduces a small performance bottleneck, it guarantees data consistency and is sufficient for the small number of devices targeted by this prototype. In a production deployment, the JSON file would be replaced by a dedicated database management system capable of safely handling concurrent access while providing significantly better scalability and reliability.

In addition to serving the mobile application and wearable devices, the backend communicates with external cloud services including the Google Geolocation API for Wi-Fi positioning, the nRF Cloud API for A-GNSS assistance data, and Firebase Cloud Messaging for emergency notifications.

5.2.3 Communication Interfaces

As it has already established, two independent interfaces are implemented within the same backend application. MQTT is used for communication with the wearable devices, while a RESTful API provides communication with the mobile application. The interaction between the backend and the other components is illustrated in Figure 5.3.

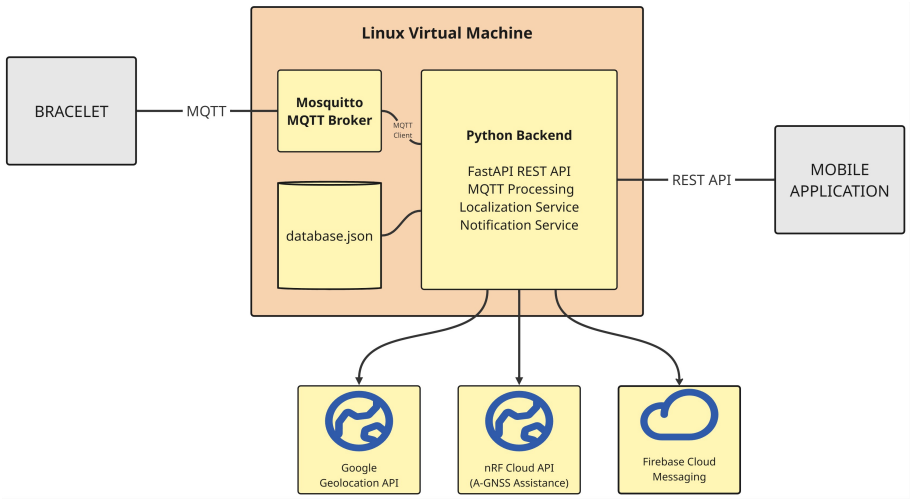


Figure 5.3: Overview of the backend components and the communication interfaces.

MQTT Communication

Communication with the wearable bracelets is implemented through a locally hosted broker (Mosquitto) running on the Azure virtual machine. Mosquitto is an open-source MQTT message broker responsible for receiving messages from connected clients and forwarding them to all subscribers of the corresponding topics. The Python backend connects to the broker as an MQTT client using the Eclipse Paho MQTT library. Once connected, it subscribes to the required topics and processes incoming messages through callback functions.

After a bracelet establishes its LTE-M connection, it publishes telemetry to the broker. The broker immediately forwards these messages to the backend, where they are processed in dedicated worker threads to avoid blocking subsequent MQTT traffic. Depending on the received payload, the backend updates the bracelet state, resolves the device location, evaluates the configured geofence, records historical events, and publishes response

messages back to the bracelet. Wi-Fi scan results are resolved through the Google Geolocation API, while A-GNSS assistance data is retrieved from the nRF Cloud API when requested by the bracelet. Emergency events trigger Firebase Cloud Messaging notifications to the paired caregiver application.

The MQTT implementation uses two topic categories:

Table 5.3: MQTT topics implemented by the backend server.

Topic	Direction	Description
`bracelet/{id}/data`	Subscribe	Receives periodic bracelet telemetry, including battery level, emergency status, and localization data.
`bracelet/{id}/response`	Publish	Sends acknowledgements, Wi-Fi localization results, and geofence status (home/away) back to the bracelet.

REST API

Communication with the caregiver application is implemented through a RESTful API developed using FastAPI and served by the Uvicorn ASGI server. Unlike the persistent MQTT connection used by the bracelets, the mobile application communicates through standard HTTP requests whenever information is required or an action must be performed.

The API provides functionality for bracelet pairing, monitoring device status, retrieving historical events, configuring geofences, registering Firebase notification tokens, and resolving emergency events. All endpoints return JSON responses, allowing direct integration with the Flutter application.

Access to the API is protected using token-based authentication. Standard application requests must provide an `X-App-Token` HTTP header, while administrative endpoints would require an additional `X-Admin-Token`. The authentication tokens are stored separately in the server's environment configuration and are therefore not embedded within the source code for security considerations.

Table 5.4 lists the implemented REST API endpoints.

Table 5.4: REST API endpoints implemented by the backend server.

Method	Endpoint	Purpose
GET	`/`	Returns the server status.
GET	`/api/bracelets`	Returns all bracelets paired with the requesting application.
GET	`/api/bracelets/{id}`	Returns detailed information for a specific bracelet.
GET	`/api/bracelets/{id}/events`	Returns the bracelet's recorded event history.
POST	`/api/bracelets`	Pairs a bracelet with the caregiver application using the device security code.
DELETE	`/api/bracelets/{id}`	Removes the pairing between a bracelet and the caregiver application.
POST	`/api/bracelets/{id}/geofence`	Creates or updates the bracelet's geofence configuration.
POST	`/api/bracelets/{id}/resolve`	Resolves an active emergency and records the acknowledgement.
POST	`/api/notifications/register`	Registers the caregiver application's Firebase Cloud Messaging token (explained below).
GET	`/api/admin/bracelets`	Returns information for all registered bracelets (administrator only).

5.2.4 Push Notification Architecture

Although the REST API provides the communication channel required for normal caregiver interactions, it is not suitable for emergency notifications. REST follows a client-driven request-response model, meaning that the mobile application must actively contact the backend to retrieve information.

To provide immediate notification delivery, regardless of the client's application state, the system integrates Firebase Cloud Messaging (FCM), a cloud-based push notification service provided by Google. FCM is used exclusively as a notification transport layer between the backend server and the caregiver application.

When a fall or panic event occurs, the bracelet publishes an emergency message through MQTT. The backend processes this event, determines the caregiver application associated with the device, and sends a notification request to Firebase containing the emergency information. Firebase then delivers the message to the registered mobile device through the native notification infrastructure provided by the operating system.

The integration requires the mobile application to register with Firebase during startup. Firebase generates a unique registration token identifying the application instance. This token is transmitted to the backend through the REST API and stored together with the corresponding application identifier. Whenever an emergency occurs, the backend uses this token to address the notification to the correct caregiver device.

The fundamental gain from using this approach is that emergency messages can be delivered even when the application is not actively running. Firebase therefore complements the existing communication architecture by separating responsibilities:

- MQTT provides reliable bidirectional communication between the wearable device and backend.
- REST provides request-response communication for caregiver application operations.
- Firebase Cloud Messaging provides emergency notification delivery.

5.2.5 Security Considerations

Several security mechanisms have been incorporated into the backend to protect communication between the wearable devices, the caregiver application, and the server.

Access to the REST API is controlled through token-based authentication. All requests originating from the caregiver application must include an `X-App-Token` HTTP header containing the application authentication token. Administrative operations, such as retrieving information about all registered bracelets, additionally require an `X-Admin-Token`. Both tokens are verified before the requested operation is executed, preventing unauthorized clients from accessing protected API endpoints.

Sensitive credentials are isolated from the application source code by storing them in environment variables loaded from a `.env` configuration file during server startup. These credentials include the application authentication token (`APP_TOKEN`), the administrator authentication token (`ADMIN_TOKEN`), and external service API keys. Separating configuration from the source code prevents credentials from being exposed through the code repository and allows them to be changed without modifying the application itself.

Each wearable bracelet also has a unique security code that is communicated to the backend during its initial connection. This code is stored together with the bracelet's registration information and must be provided by the caregiver application when pairing the device. As a result, knowledge of a bracelet's identifier alone is insufficient to establish ownership, preventing application users that do not have the code from pairing with arbitrary devices.

The implemented security mechanisms provide authentication of trusted applications, controlled access to administrative functionality, secure separation of credentials from the application source code, protection against unauthorized bracelet pairing, and preservation of data integrity during concurrent database access.

The implemented security mechanisms satisfy the functional requirements of the system while maintaining a lightweight deployment. For larger-scale deployments, additional measures such as HTTPS and a dedicated database management system would further improve communication security and system scalability.

5.3 Caregiver's Mobile Application

The mobile application acts as the primary interface between the caretaker and the tracking device. The primary objectives of the application are to provide real-time geolocation tracking, battery life monitoring, and notifications to facilitate timely information about emergency events such as physical falls or triggered panics.

Flutter is an open-source, cross-platform mobile application development framework created by Google, which enables developers to build projects that can be compiled directly into native machine code for iOS, Android, web, and desktop from a single codebase using

Dart as the programming language. This significant advantage, combined with features like hot reload (that allows us to see code changes instantly without reinstalling the application on the testing device) made Flutter an ideal choice for this project. Additionally, Flutter's reactive programming model enables the UI to automatically rebuild and update whenever the application's state changes, meaning that when new location data or panic alerts arrive from the server, the interface responds immediately without requiring manual refresh logic. The native state management `StatefulWidget` ensures that the lifecycle of the data (such as active polling loops and memory allocation) is tightly bound only to the specific screen currently being viewed by the user, immediately disposing of resources when the screen is closed.

Due to hardware limitations, particularly the fact that iOS development requires a macOS device to compile applications, all testing and development was conducted on an Android device (Pixel 8) with the latest version of Android. However, the cross-platform nature of Flutter means that future iOS deployment would require minimal code modifications beyond platform-specific configurations and testing.

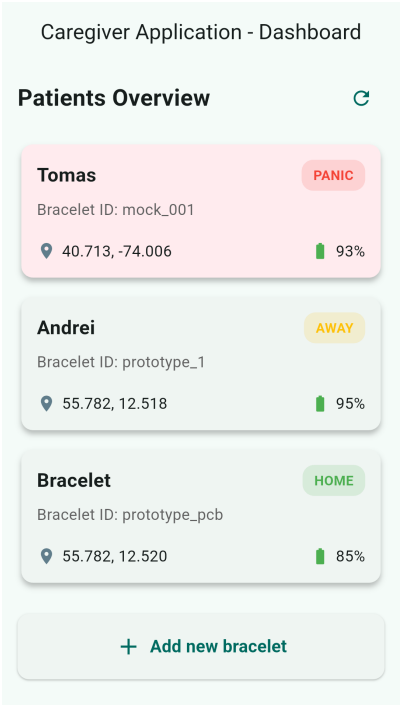


Figure 5.4: The main dashboard interface of the Caregiver Application.

5.3.1 Architectural Design

It is often observed in advanced projects that as mobile applications scale in complexity, traditional "Layer-First" architectures (where all user interfaces are grouped in one directory, and all network calls in another) often lead to tight coupling and reduced maintainability.

To ensure separation of concerns, the application employs a Feature-First architectural pattern. The codebase is separated by domains (e.g., Dashboard, Bracelet Details) rather than by technical layers. Each feature encapsulates its own presentation UI, repository, and state logic, while globally shared resources such as API routing and push notification configurations are part of a centralized core services layer. This ensures that if modification are required for a specific feature, changes are to be made only to that isolated

module without having to transverse the whole project.

Following the presented principle, the application's source code is structured into three primary folders:

- `lib/features/`: The core of the application, containing the actual screens and user interfaces, such as `dashboard`, `bracelet_details` and `pairing`. Each feature encapsulates its own presentation layer (screens and widgets) and its own data management (repositories). For example, the map rendering logic and event history timelines are fully isolated as "widgets" within the `bracelet_details` feature. In the same feature, we also have a BraceletDetailsRepository that takes the complex JSON and converts it into an object (BraceletModel) directly used by the UI in the main BraceletDetailsScreen.
- `lib/services/`: This layer handles all long-running external communications and I/O operations that span across multiple features.
  - `api_service.dart`: This module encapsulates all HTTP web transactions (`GET`, `POST`, `DELETE`) and serves as the centralized gateway to the FastAPI backend. This architectural choice promotes high maintainability, as in case network configurations change, modifications are isolated to this single file.
  - `push_notifications_service.dart`: This service handles all Firebase Cloud Messaging (FCM) functionality. It manages Firebase initialization, requests notification permissions from the device's operating system, and maintains a continuous background listener that receives emergency alerts from the server even when the app is closed.
- `lib/core/`: This directory stores generic, foundational code that doesn't belong to any specific screen or feature. A prime example is `app_config_service.dart`, which utilizes the `shared_preferences` plugin to read and write the device's persistent cache. It generates and stores the unique `app_id` that is sent in every API request, acting as a global utility rather than a feature-specific script.

A diagram that presents the architecture of the application can be observed in figure 5.5.

5.3.2 Security and Device Provisioning

Traditional user authentication systems, such as email and password logins, often introduce unnecessary complexity for the end user, especially in a system such as this one where the caretaker should only set up the application once and not have to deal with logging in and remembering a password. To prioritize usability without compromising system integrity, the application utilizes a token-based provisioning model.

- **Device Identification (UUID):** Instead of manual account registration, the application transparently associates the caregiver's device with the backend using a cryptographically secure Universally Unique Identifier (UUID version 4). This UUID is generated automatically upon the initial application launch when the push notification service boots up to register its Firebase token. Cached locally via *SharedPreferences*, this acts as a permanent `app_id`, allowing the backend to seamlessly map specific bracelets to the correct caregiver's smartphone.
- **API Security & Request Authorization:** While the UUID helps route the information from the paired devices to the matching application, the system must also reject unauthorized external web traffic. To achieve this, the application injects an

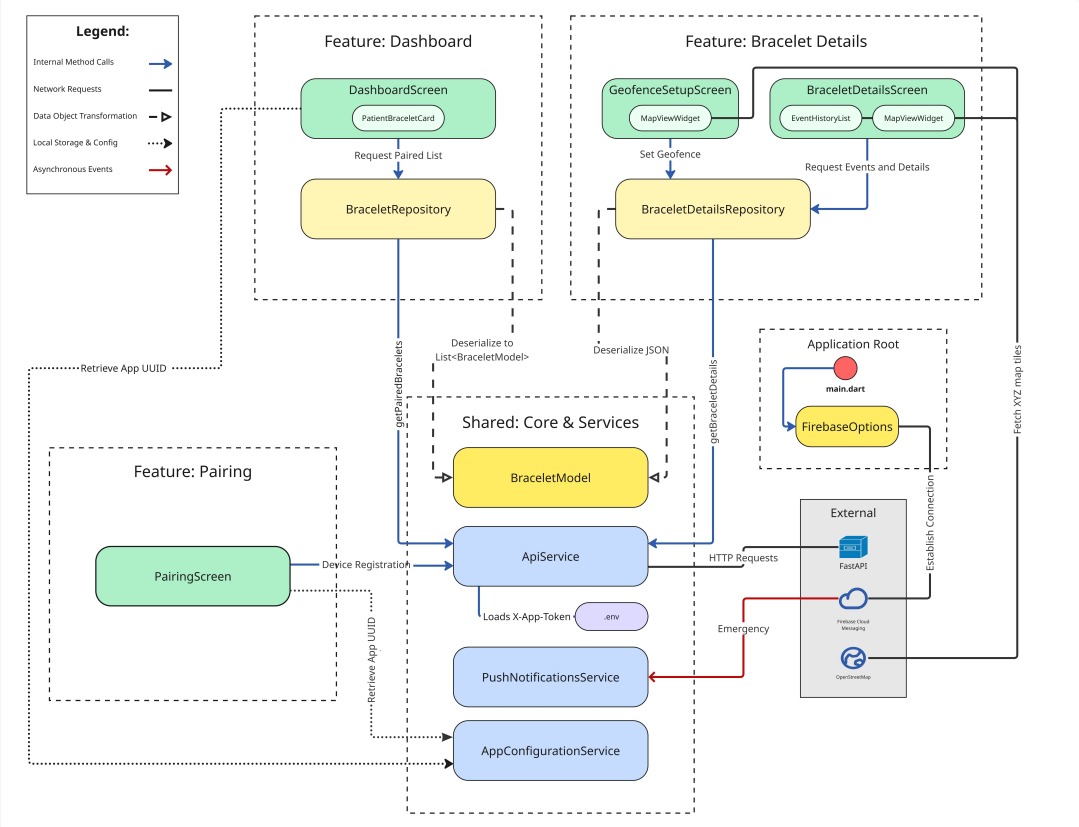


Figure 5.5: High-level architectural block diagram of the Flutter Application

`X-App-Token` into the headers of every HTTP request. This token acts as a static application key, securely loaded from environment variables (`.env`) at runtime rather than being hardcoded. This architecture prevents credential leakage in code repositories while ensuring the FastAPI server can confidently authenticate legitimate client requests.

5.3.3 Core Functionalities

Stateless Synchronization and Mobile Resilience

To maintain updated information when the user has the application open, the dashboard continuously polls the centralized backend on a 10-second interval. While persistent TCP connections (such as WebSockets) offer lower latency, they introduce significant fragility in mobile environments. Smartphones frequently experience ”IP hopping” (seamlessly switching between Wi-Fi and 5G cellular networks, which changes their IP) and aggressive operating system-level protocol termination to preserve battery life. Both of these events instantly close the WebSocket connections, demanding complex, battery-intensive, and in our experience error-prone reconnection process.

Therefore, the application deliberately adopts a stateless HTTP polling mechanism for routine data monitoring (such as battery levels and location updates). This approach is highly resilient to network changes as every request is an isolated event. If a poll fails during a cell tower swap, the system simply updates again on the next interval without requiring extra re-connection mechanisms. Furthermore, the State Management logic we implemented handles the network events in the background without visible problems in the User Interface. When a poll fails, the UI will remember and display the last-known

state stored safely in memory rather than overwriting the screen with errors.

Geolocation Visualization

For spatial monitoring, the system integrates *OpenStreetMap* cartography via the `flutter_map` framework. The use of an open source, free, cartography solution removes dependency on paid external services as Google Maps API, while maintaining high-performance tile rendering. During our testing, the OpenStreetMap performs flawlessly and has remained the choice for the final version of the application. The mobile client parses latitude, longitude, and the localization method (e.g., GNSS, Wi-Fi) directly from the package received from the server. It then plots precise geographic markers overlaid with semi-transparent radius circles (`CircleLayer`) scaled to the accuracy measurement in meters. This explicitly visualizes the hardware's geographical margin of error, preventing false caregiver panic in situations with less accurate localizations.

To facilitate situations where the caretaker wants to navigate to the current position of the wearer, the app provides a button which opens the location on an external navigation system (Google Maps). All these features can be seen in figure 5.6, which presents a view of the bracelet details screen.

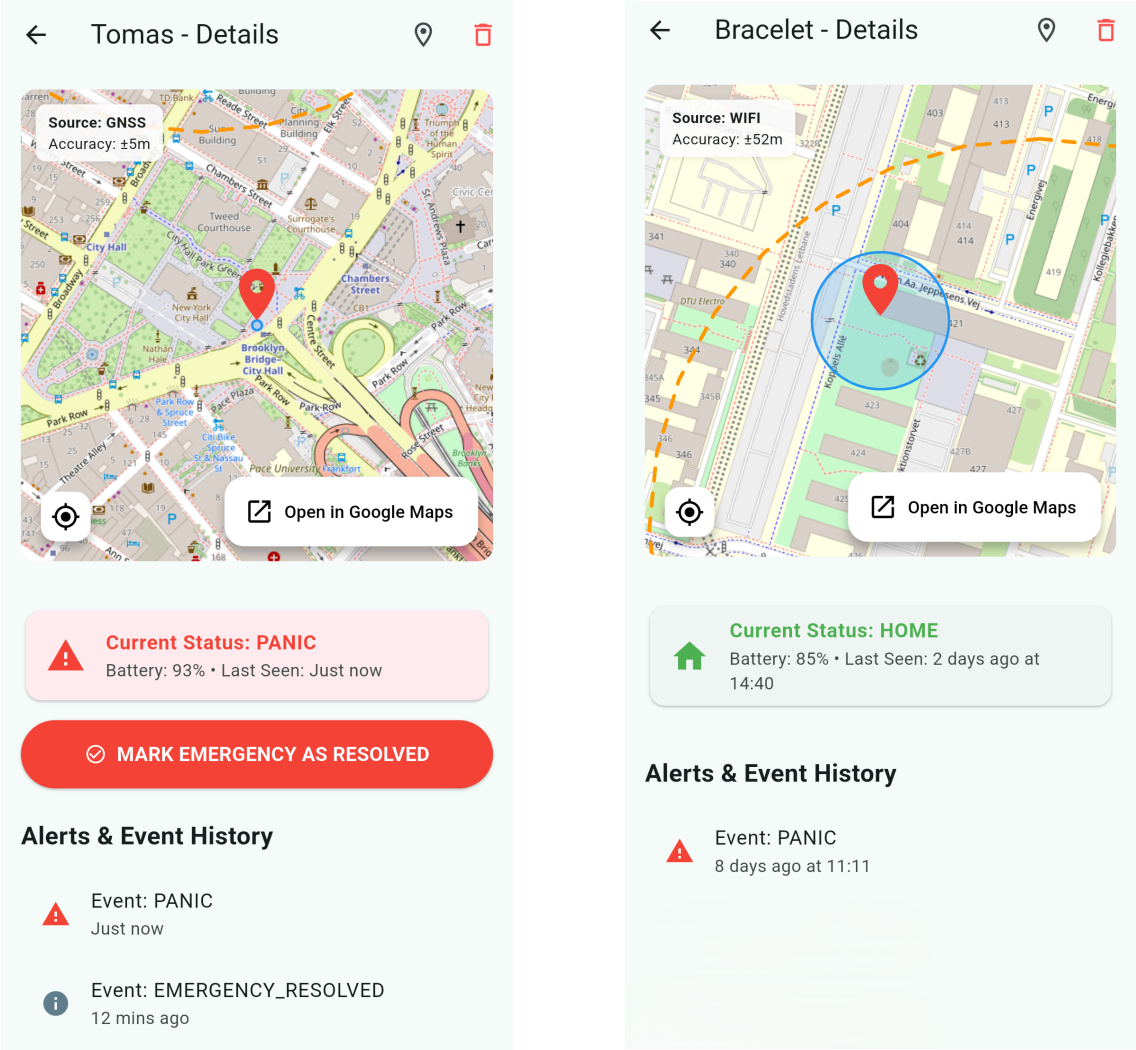


Figure 5.6: The Bracelet Details Screen of the Caregiver Application.

Geofencing Configuration and Monitoring

To further enhance patient safety and provide active monitoring, the application implements both definition and visual representation of a geofencing system. Caregivers can set up a virtual geographic boundary, which is named the ”home location” in the UI, directly from the mobile interface. This is achieved via a dedicated configuration screen featuring an interactive OpenStreetMap interface and a slider to adjust the perimeter's radius dynamically. Immediate feedback is ensured by rendering a scalable circle over the cartography to represent the active area. Upon confirmation, the exact spatial parameters (center latitude, longitude, and radius) are serialized and transmitted to the backend, where they are stored in the database and after each localization event, the server updates the status of the device based on it being located inside or outside of the geofence.

Subsequently, when viewing the patient's real-time location on the details screen presented above, the same geofence is retrieved alongside the normal polling data and rendered as a distinct, dashed circular boundary. This provides immediate visual context, allowing the caregiver to assess whether the wearer is safely within their designated zone, approaching the border or wandering outside.

Critical Alerting: The Dual-Layered Data Strategy

Because a 10-second polling interval while the app is open is insufficient for emergencies, the system utilizes a dual-layered data strategy. While stateless polling handles routine device status, emergency events (e.g., falls or panic button activations) bypass the polling cycle entirely using Firebase Cloud Messaging (FCM). FCM leverages a persistent, battery-efficient background connection managed directly by the Android/iOS operating system, allowing the device to receive reliable, sub-second latency alerts even when the application is closed.

- **Background Execution for incoming notifications:** When the application is terminated or operating in the background, the primary user interface (UI) thread is suspended or destroyed. To process incoming FCM payloads under these conditions, the system leverages a background Dart isolate. Unlike traditional shared-memory threads, a Dart isolate is an independent execution unit with its own memory heap and event loop. When a push notification arrives, the native operating system (Android/iOS) wakes the Flutter engine just enough to spawn this isolate. This allows the application to intercept the payload and trigger system-level emergency alerts completely independently of the main application lifecycle and active polling timers.
- **Foreground Overrides & Escalation:** By default, native mobile operating systems suppress visual ”heads-up” notification banners if the corresponding application is already open in the foreground. To bypass this restriction and guarantee that panic events are never missed, the application parses incoming FCM packets in real-time via an active stream listener. It then triggers an overriding local notification using the `flutter_local_notifications` package. These alerts are forcefully routed through a dedicated Android `NotificationChannel` configured with maximum conceptual importance (`Importance.max`), guaranteeing immediate visual and auditory response no matter what the user's current interaction state is.

5.3.4 Technical Challenges and Optimizations

The development of the mobile client presented several low-level technical issues that required special attention:

1. **Asynchronous State Safety and Memory Leaks:** As previously mentioned, the

Dashboard implements a 10-second HTTP polling loop where network responses naturally arrive asynchronously. During the initial phases of development, we discovered an issue that occurred when an asynchronous operation (such as an API `GET` request) concluded and attempted to trigger a UI rebuild after the user had already navigated away. The moment a user navigates to a different screen, Flutter automatically calls `dispose()` on the Dashboard's `State` object, removing it from the widget tree and marking it as unmounted. Meanwhile, the polling request can still complete in the background, and its callback may try to update the UI with new data by calling `setState()` on this disposed widget, causing a fatal exception.

To address this, we always check the `mounted` flag before calling `setState()`. If the `mounted` flag is `false`, then the widget has been destroyed and removed from the tree, so the code silently discards the network response instead of attempting a UI update. This prevents the "setState() called after dispose()" crash while maintaining memory safety.

2. **Temporal Normalization:** Raw IoT event histories emitted by the physical bracelets are routed through the backend proxy and stamped using Coordinated Universal Time (UTC) to prevent timezone collision errors. However, raw UTC timestamps are not easy to understand by caregivers. The client resolves this via a dedicated utility class (`date_formatter.dart`). Utilizing the `intl` package, the application intercepts the UTC ISO-8601 strings, dynamically offsets them into the caregiver's localized device timezone, and calculates absolute midnight boundaries. This allows the UI to categorize historical panics intuitively as "Today," "Yesterday," or standard localized dates, dramatically improving the user's experience and time comprehension.
3. **Non-Destructive UI Mutations and Network Resilience:** Early iterations of the background polling mechanism suffered from destructive state resets. During a network hiccup (e.g., a micro-drop in Wi-Fi, a server timeout, or a connection refusal), the data layer used the default behaviour and returned an empty array (`[]`). The presentation layer directly presented this empty array to the user interface, momentarily wiping all tracked patients off the screen, only to abruptly redraw them upon the next successful poll. To eliminate this visual flickering, the data layer was adjusted to differentiate between an empty state ("Zero Paired Devices") and a network failure (returning `null`). The Dashboard's state management explicitly intercepts these `null` payloads to abort UI mutation, cleanly preserving the last-known "good" data and shielding the caregiver from network instability.
4. **Custom Rendering for Geofences:** While implementing the visual representation for the geofence boundaries, a rendering limitation within the `flutter_map` library was encountered: the native `CircleLayer` supports solid borders but lacks dashed line patterns. Visually distinguishing the geofence perimeter from the solid line that represents the location-accuracy radius was deemed a critical UI requirement to prevent user confusion. To solve this without adding extra dependencies, the application mathematically generates and represents the boundary of the circle as a series of (lat, long) coordinates.

6 System Evaluation and Testing

This chapter evaluates the system developed throughout this thesis against the implementation targets and requirements established in Chapter 1. This chapter presents a comparative analysis of detection accuracy, a detailed characterization of power consumption across the system's operating states, and an account of the hardware bring-up issues encountered during implementation, providing the basis for assessing whether the system meets its performance and battery life targets.

6.1 Machine Learning and Threshold Model Comparison

6.1.1 Algorithm Performance

Even though a Machine Learning (ML) approach was selected to be the main fall detection algorithm for this project, it is crucial to benchmark its performance against a standard threshold-based model. Traditionally, ML models require greater computational resources and inference time, leading to higher power consumption. However, modern edge AI platforms, such as Edge Impulse, efficiently optimize and compress these models for resource-constrained microcontrollers. Therefore, if the inference latency and power overhead become negligible, the superior accuracy of ML justifies its implementation over a simpler model.

To validate this, a threshold-based model was evaluated against the exact same test dataset used for the ML model. The threshold algorithm follows a standard implementation designed to capture the three sequential phases of a fall: a free-fall state characterized by a near-zero g acceleration magnitude, an impact state represented by a sharp acceleration spike, and a post-impact stillness state where the acceleration stabilizes at 1 g. Furthermore, barometric pressure data was integrated to capture altitude changes. Since this threshold model leverages the identical sensor data pipeline as the ML model, it provides a fair baseline comparison.

To maximize the threshold model's performance, a brute-force grid search optimization was performed using the same training dataset. This hyperparameter sweep evaluated six variables: free-fall threshold (g), impact threshold (g), impact window (s), barometric pressure gating (on/off), pressure delta threshold (hPa), and post-impact stillness window (s). Table 6.1 outlines the parameter space explored during this optimization.

Table 6.1: Parameters used in the threshold-model grid search

Parameter	Range of tested values	Best
Free-fall threshold [g]	0.32 – 0.71	0.50
Impact threshold [g]	1.50 – 2.83	2.75
Impact window [s]	0.09 – 4.50	0.09
Pressure gate	on, off	on
Pressure delta threshold [hPa]	0.00 – 0.09	0.09
Post-impact stillness window [s]	off, 0.6 – 2.0	off

A fall is classified only when the free-fall, impact, and stillness phases occur sequentially within the defined windows and cross their respective thresholds. For an imbalanced dataset (23 falls and 41 non-falls), the F1 score provides a more robust performance metric than raw accuracy, calculated as the harmonic mean of precision and recall:

$$F1 = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}} \tag{6.1}$$

The parameter combination yielding the highest F1 score on the training set was selected for final inference. As indicated in Table 6.1, a high impact threshold combined with a high pressure delta yielded the best results, while the post-impact stillness metric was ultimately excluded by the grid-search due to the high variability of the dataset. Evaluating this optimal configuration on the test set produced the confusion matrix presented in Table 6.2.

Table 6.2: Confusion matrix of the threshold model on test data

Actual / Predicted	**Predicted: Fall**	**Predicted: No Fall**
Actual: Fall	17	6
Actual: No Fall	2	39

Comparing these results directly to the deployed Machine Learning model detailed previously, the ML approach outperforms the tuned threshold model across every metric. As established in Table 5.2, the neural network achieved a 98.4% overall accuracy and a complete absence of false positives (0% FPR). In contrast, the optimal threshold model yielded a 4.9% false positive rate and misclassified significantly more actual falls. The ML model's ability to generalize complex patterns allows it to better distinguish actual emergencies from overlapping daily activities which is exactly where standardized models fail. However, it must be acknowledged that the 64-sample test set is relatively small. Expanding the dataset to a more diverse population could influence these comparative margins in future testing iterations.

6.2 Power Consumption Profiling

All the data presented in this section and all testing related to the power consumption has been made with the Nordic Power Profiler Kit II (PPK2).

6.2.1 ML and Threshold Comparison

To address potential concerns regarding the energetic cost of an ML approach, the full system power consumption was profiled during active software execution for both models. Figure 6.1 illustrates the comparative power draw over a standard 10-second operational window.

As the power profiling reveals, the ML and threshold approaches yield nearly identical average power consumptions of 6.56 mA and 6.59 mA, respectively. This demonstrates that the theoretical increase in inference time associated with the ML model has a negligible impact on the average current draw. The system's power consumption is dominated by active data collection and peripheral communication instead of processor inference calculations. Because both models have equivalent power constraints while the ML model



Figure 6.1: ML vs Threshold power consumption

delivers significantly higher classification performance, this approach is the optimal choice for this application.

6.2.2 GNSS and A-GNSS Comparison

To evaluate the benefit of Assisted GNSS (A-GNSS), a series of localization tests were performed using both conventional GNSS and A-GNSS under identical outdoor conditions. While A-GNSS requires an initial connection to the backend server to download assistance data, this additional communication overhead is small compared to the reduction in satellite acquisition time. As a result, A-GNSS significantly decreases both the time-to-first-fix (TTFF) and the total energy required to obtain a position.

Table 6.3 presents the results obtained.

Compared to conventional GNSS, A-GNSS reduced the average TTFF from 102 s to 10 s, corresponding to approximately a 90% reduction in positioning time. Although A-GNSS has a higher peak current due to the LTE modem activity required to download assistance data, the overall localization duration is much shorter. Consequently, the total charge consumption is reduced from 6.44 C to 1.34 C, representing an energy saving of approximately 79%.

6.2.3 Power Saving Mode for LTE-M

Power Saving Mode (PSM) and Extended Discontinuous Reception (eDRX) are two power saving techniques for LTE-M and NB-IoT devices. eDRX allows a device to remain in RRC Idle mode and only periodically wake up for short periods to check wether the network has an incoming message. Since the developed application does not expect any

Table 6.3: Comparison of GNSS and A-GNSS localization measurements.

Method	Cycle	TTFF [s]	Latitude	Longitude	Avg. [mA]	Peak [mA]	Charge [C]
GNSS	1	91	55.782	12.520	28.22	83.32	5.08
	2	132	55.781	12.521	40.62	83.81	7.32
	3	114	55.782	12.521	35.44	83.66	6.38
	4	60	55.782	12.521	19.41	83.92	3.49
	5	113	55.781	12.521	35.40	84.14	6.38
	Average	**102**	**55.782**	**12.521**	**31.82**	**83.77**	**5.73**
A-GNSS	1	15	55.781	12.520	10.02	373.32	1.21
	2	6	55.782	12.520	6.13	376.58	0.74
	3	14	55.782	12.521	8.12	373.16	0.97
	4	7	55.782	12.520	5.07	373.80	0.61
	5	6	55.782	12.520	7.65	375.23	0.93
	Average	**10**	**55.782**	**12.520**	**7.40**	**374.42**	**0.89**

downlink messages without being triggered by the device, eDRX is not needed for this application and would only increase power consumption with no real purpose.

PSM allows a device to enter a deep sleep state where both the transmitter and receiver are switched off, while the device remains registered with the network. PSM was considered a good candidate for reducing power consumption during the long idle periods between events.

To evaluate PSM, we implemented a test application requesting PSM from the network and measured the modem's current consumption across ten wake-connect-sleep cycles. This was compared against the baseline behavior, which fully disconnects and powers off the modem (`lte_lc_power_off()`) instead of using PSM.

During testing the network granted a periodic TAU (the longest time the device is allowed to sleep before it must check in) of 3600 s, which is sufficient for this application. However, before the modem is allowed to enter PSM, it must first pass through RRC Connected mode, where it stays active until the network releases the connection. To reduce the time spent in this state, we requested Release Assistance Indication (RAI), a feature that lets the device inform the network that no more data will be sent, so the connection can be released immediately instead of waiting for the RRC inactivity timer to expire.

The RAI request was accepted by the modem, but had no effect on the connection duration: the network kept the RRC connection active for approximately 25 seconds after every registration, regardless of the RAI request. This proves that the connection release timing is controlled entirely by the network, and cannot be reduced further from the device side once the request has been correctly sent.

As shown in Table 6.4, PSM reduces the connection time per cycle from 2434 ms to 614 ms, but results in a slightly higher average charge per cycle (247.02 mC vs. 224.00 mC). The RRC-connected tail (visible in Figure 6.2 before PSM sleep consumes roughly the same amount of energy that PSM was meant to save by avoiding a full re-attachment. Because power saving is considered to have priority over a small reconnection time reduction, it was preferred to fully power off the modem between communication cycles.

6.2.4 Battery Life Estimation

Battery life was estimated from the measured charge consumption of each operating state during a representative test run. Figure 6.3 shows the resulting power profile, while Ta-

Table 6.4: Measured current consumption per cycle: PSM vs. full modem power-off

	PSM					LTE OFF			
Cyc.	**T [ms]**	**Avg [mA]**	**Max [mA]**	**Q [mC]**	**Cyc.**	**T [ms]**	**Avg [mA]**	**Max [mA]**	**Q [mC]**
2	557	4.04	130.00	242.33	2	2939	4.06	269.30	243.45
3	610	3.72	135.30	223.29	3	2715	3.85	171.29	230.59
4	595	4.38	121.07	262.54	4	2733	4.00	168.84	240.14
5	872	4.55	141.02	272.89	5	3196	4.15	310.87	249.10
6	596	4.06	145.71	243.42	6	2164	3.38	234.90	202.76
7	554	4.07	196.25	244.31	7	1934	3.42	158.64	205.23
8	554	4.08	163.38	244.59	8	1836	3.57	231.45	214.47
9	596	3.97	157.09	238.46	9	2012	3.63	237.67	217.59
10	597	4.19	145.57	251.35	10	2373	3.54	202.29	212.69
Avg	**614**	**4.12**	**148.38**	**247.02**	**Avg**	**2434**	**3.73**	**220.58**	**224.00**

T = Connection Time, Avg = Average current, Max = Maximum current, Q = Charge



LTE-M fully off | Power Saving Mode

Figure 6.2: Cellular modem on/off vs PSM

ble 6.5 summarizes the measured charge and duration of each state.

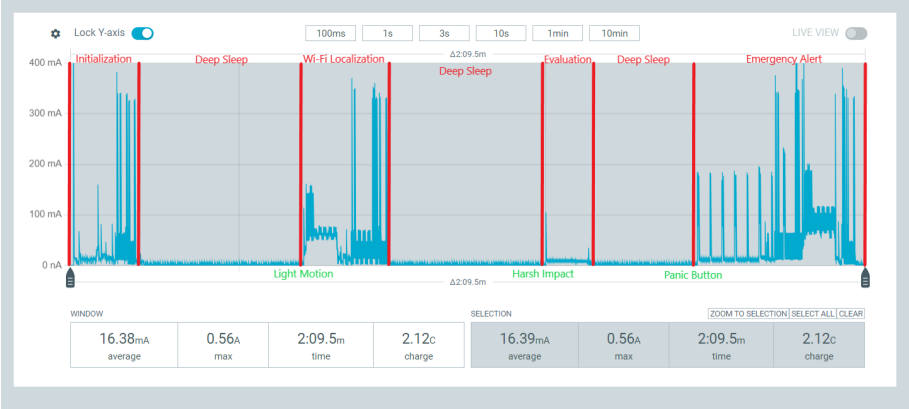


Figure 6.3: Power consumption during various states

The A-GNSS fallback state consists of a standard Wi-Fi localization attempt (0.60 C) followed by an additional 10 s A-GNSS acquisition phase (1.34 C), resulting in a total charge consumption of 1.94 C. All localization measurements include the energy required for cellular communication with the server.

Average Daily Usage Scenario

To obtain a realistic estimate of battery life, a daily usage scenario was constructed based on the expected behavior of an elderly wearer. This section intentionally assumes relatively frequent movement and localization activity in order to approximate a near worst-case operating condition.

Table 6.5: Measured Charge Consumption per Operating State

State	Charge	Duration
Initialization	229 mC	11.00 s
Deep sleep (baseline)	26.8 mC	25.29 s
Localization (Wi-Fi)	0.60 C	14.20 s
Localization (A-GNSS fallback)	1.94 C	24.20 s
Evaluation (harsh impact)	61.65 mC	7.25 s
Emergency alert + localization	1.15 C	26.20 s

During an 8-hour sleep period, five motion-triggered localization events are assumed. The remaining 16 hours consist of:

- 1 hour outside the geofence with active tracking every 3 minutes (20 events),
- 4 hours of continuous movement inside the geofence with localization every 10 minutes (24 events),
- 11 hours of typical low-mobility activity with localization every 30 minutes (22 events).

The total number of localization events during daytime activity is therefore 71, out of which 90% (64 events) are resolved via Wi-Fi scanning and 10% (7 events) fall back to A-GNSS positioning.

In addition, the device is assumed to register approximately 20 harsh-impact events per day, each entering the evaluation state without escalating to a confirmed emergency alert. Initialization is assumed to occur only once per charge cycle. Confirmed falls and panic-button activations are excluded from this estimate. Such events are expected to occur infrequently, and once an emergency is detected, successful alert delivery becomes the primary system objective.

Daily Charge Consumption

The daily charge consumption is obtained by summing the contribution of each operating state:

$$\begin{aligned} Q_{\text{loc,wifi}} &= 64 \times 0.60\,\text{C} = 38.4\,\text{C} \\ Q_{\text{loc,agnss}} &= 7 \times (0.60\,\text{C} + 1.34\,\text{C}) = 13.58\,\text{C} \\ Q_{\text{eval}} &= 20 \times 61.65\,\text{mC} = 1.233\,\text{C} \\ Q_{\text{init}} &= 0.229\,\text{C} \end{aligned} \tag{6.2}$$

The remaining 85 166 s of the day is spent in deep sleep is spent in deep sleep. This corresponds to 85,165 s of baseline sleep, which, at the measured deep sleep drain rate of 1.060 mC/s, and contributes to:

$$Q_{sleep} = 85{,}165.8\,\text{s} \times 1.060\,\text{mC/s} \approx 90.28\,\text{C} \tag{6.3}$$

Summing all contributions yields the total daily charge consumption:

$$Q_{total} = Q_{init} + Q_{sleep} + Q_{loc,wifi} + Q_{loc,agnss} + Q_{eval} \approx \textbf{143.72 C/day} \tag{6.4}$$

Estimated Battery Life

Two battery capacities were considered: a 300 mAh cell, which is the current prototype, and a 500 mAh cell, which is the recommended battery capacity, that does not require an increase in the spatial dimensions of the case.

Using the conversion $1\,\text{mAh} = 3.6\,\text{C}$, the available charge is

$$\begin{aligned} Q_{300} &= 300\,\text{mAh} \times 3.6 = 1080\,\text{C} \\ Q_{500} &= 500\,\text{mAh} \times 3.6 = 1800\,\text{C} \end{aligned} \tag{6.5}$$

With the estimated daily consumption of approximately 143.72 C/day, the expected battery life under the heavy usage estimation described becomes:

$$\begin{aligned} \text{Battery life}_{300} &= \frac{1080\,\text{C}}{143.72\,\text{C/day}} \approx \mathbf{7.5\,days} \\ \text{Battery life}_{500} &= \frac{1800\,\text{C}}{143.72\,\text{C/day}} \approx \mathbf{12.5\,days} \end{aligned} \tag{6.6}$$

Both battery configurations satisfy requirement R9, which demands a minimum battery life of 7 days. The 300 mAh prototype battery meets this threshold with a narrow margin, while the 500 mAh battery provides a substantially larger safety margin, making it the recommended choice (if available).

Deep sleep accounts for approximately 63% of the total daily energy consumption, while localization contributes 36%. Consequently, further battery-life improvements should primarily target reducing the baseline sleep current, with additional gains achievable through optimization of the localization pipeline.

6.3 Hardware Bring-up Challenges

nRF9151 Modem Firmware Update

During initial cellular connectivity testing, the nRF9151 was unable to register with the LTE network. The issue was resolved by updating the modem firmware using the *nRF Connect for Desktop Programmer* through the SEGGER J-Link interface. The nRF9151 contains a dedicated modem subsystem with its own firmware, which is independent from the application firmware running on the Cortex-M33 processor. Therefore, programming the application image alone is not sufficient for full cellular functionality.

The required modem firmware files and compatibility information are provided on the Nordic Semiconductor *nRF9151 Downloads* page [54]. In case of replicating the project, this step should be done on newly manufactured boards, as an outdated modem firmware version may prevent successful network registration.

Wi-Fi Driver Initialization

The initial bring-up of the nRF7002 Wi-Fi companion chip required additional debugging before Wi-Fi scanning could be performed reliably. The main issue was that the module did not have a valid MAC address provisioned from the factory. As a result, the Wi-Fi driver failed during initialization and scan requests could not be started.

Several approaches to assigning a MAC address using the drivers were tried, but they did not initialize the network interface correctly. The issue was resolved by enabling the Zephyr configuration option CONFIG_WIFI_RANDOM_MAC_ADDRESS, which generates a valid random MAC address during initialization. With the MAC address issue resolved, the nRF7002 was able to perform Wi-Fi scans successfully, providing the MAC addresses and RSSI values of nearby access points for the Wi-Fi localization algorithm.

7 Conclusion

This thesis presented the design, implementation, and evaluation of a standalone IoT bracelet intended for monitoring vulnerable users. The developed system combines on-device machine learning for fall detection, tiered localization using Wi-Fi, A-GNSS, and cellular technologies, LTE-M communication, and a cloud-connected caregiver application. The project covered the complete development process, including custom hardware design, embedded firmware, backend services, and a mobile application. Experimental evaluation demonstrated that the proposed architecture fulfills its primary objective of providing reliable localization and emergency monitoring while maintaining low power consumption.

7.1 Requirements Satisfaction

The system requirements established in Chapter 1, Section 1.4 defined the design and performance targets against which the prototype is evaluated. Table 7.1 presents the satisfaction status of each of the 15 requirements alongside a brief justification. Of these, 12 requirements were fully satisfied, two (R1, R11) were partially satisfied, and only R14 was not satisfied. This requirement requested an unobtrusive, non-stigmatizing appearance, which could not be achieved at this stage, as functionality and debugging accessibility were prioritized over enclosure materials and minimal PCB footprint in this prototype iteration.

7.2 Challenges and Limitations

The hardware bring-up and prototype development presented several practical challenges. As mentioned previously, the PCB design and schematics were adapted from a prior thesis project. While this accelerated the initial development phase, it required reverse-engineering specific layout choices. Although adapting an existing design proved effective for rapid prototyping, future iterations would benefit from a complete, ground-up PCB redesign. A custom layout would ensure full control over the board's electrical characteristics and prevent issues such as the detuned RF matching networks, which occurred when the board footprint was reduced without fully accounting for the RF consequences. Resolving these hardware issues was further complicated by a lack of prior experience with advanced PCB routing, highlighting the steep learning curve associated with high-frequency board design.

In the software domain, the nRF Connect SDK presented its own learning curve. While the platform is highly optimized for Nordic components, integrating third-party peripherals proved difficult to debug due to limited documentation. The Zephyr operating system also proved highly sensitive to minor modifications, as a single configuration flag could fundamentally alter system behavior. Identifying the root cause of firmware bugs was challenging due to the complex interaction between these underlying system variables. For instance, the Wi-Fi chip failed to initialize until a MAC address was manually provisioned in the software. Navigating and resolving these software configurations ultimately consumed a significant portion of the development time.

Table 7.1: Requirements Satisfaction Summary

ID	Requirement	Status	Justification
R1	$\geq$99% fall detection rate	Partially satisfied	98.4% accuracy and one false negative test
R2	Limit false positives and alert cancellation option	Fully satisfied	Zero false positives on testing data, alerts can be canceled.
R3	On-device fall detection execution	Fully satisfied	Inference on device
R4	Outdoor localization error below 10 m	Fully satisfied	Target accuracy met under favorable GNSS conditions
R5	GNSS-denied localization	Fully satisfied	System reliably uses Wi-Fi and LTE
R6	Localization without external hardware dependency	Fully satisfied	Dependent only on the server, cell towers, access points and sky visibility.
R7	Reliable network connectivity	Fully satisfied	Consistent LTE-M connectivity
R8	Emergency alert delivery under 30 s	Fully satisfied	Sub-15-second achieved
R9	Minimum 7-day battery life	Fully satisfied	Achieved and proved with power profiling
R10	Information delivery to caregivers	Fully satisfied	All information available in the app
R11	Caregiver configuration options	Partially satisfied	Only geofence configuration is currently exposed
R12	Access restricted to paired caregiver apps	Fully satisfied	Pairing with security code required to access data.
R13	Comfortable wrist-worn form factor	Fully satisfied	Compact, lightweight
R14	Unobtrusive, non-stigmatizing design	Not satisfied	3D printed prototype, no display
R15	Single button with dual functionality	Fully satisfied	Button distinguishes between short and long presses

7.3 Future Work

Although the prototype meets the majority of the project objectives, several limitations identified during development provide clear directions for future work. These upcoming tasks can be categorized by priority, ranging from immediate hardware corrections to long-term feature expansions.

The highest priority for the next hardware revision involves correcting the schematic and layout issues identified during the initial bring-up. First, the RF matching networks must be updated with the newly calculated component values to restore optimal antenna tuning

for both WiFi and GNSS. Second, the OLED display connector footprint must be mirrored in the PCB design software to correct the reversed pin mapping that rendered the current display unusable. Additionally, the routing error regarding the buck-boost converter's output capacitors must be resolved by properly tying them to the ground plane in the layout, eliminating the need for manual post-assembly soldering. Finally, the physical PCB outline should be adjusted to include a dedicated cutout near the emergency button to prevent manual hardware fixes and ensure proper assembly.

Medium-priority improvements focus on optimizing system power consumption and algorithmic reliability. Upgrading the barometer to a model with a larger internal FIFO buffer would allow the main microcontroller to remain in deep sleep for longer periods between sensor readings, further extending battery life. Furthermore, it is relevant to expand the machine learning dataset with a wider variety of simulated falls and daily activities from different users. A more comprehensive dataset would enhance the classification model's overall accuracy. However, maintaining a low False Positive Rate (FPR), as it is in the current model, would have to be a priority during development.

Lower-priority, long-term goals involve expanding the system's feature set and conducting real-world validation. The companion application could be enhanced with advanced features, such as adaptive geofencing, to provide more flexible tracking options. On the hardware side, further miniaturization of the device footprint and the potential integration of a heart rate monitor would provide caregivers with additional, critical health telemetry. Finally, once the hardware and software are fully stabilized, comprehensive user testing with the target elderly demographic should be conducted to validate the device's ergonomics and daily usability.

AI Usage Declaration

AI was used throughout this thesis to support the writing task in its formulation, organization, and grammar correction. We bear full responsibility for all the results obtained, since no content was generated using AI.

Bibliography

[1] Zhihua Wang, Zhaochu Yang, and Tao Dong. “A Review of Wearable Technologies for Elderly Care that Can Accurately Track Indoor Position, Recognize Physical Activities and Monitor Vital Signs in Real Time”. en. In: *Sensors* 17.2 (Feb. 2017), p. 341. issn: 1424-8220. doi: 10.3390/s17020341. url: https://www.mdpi.com/1424-8220/17/2/341 (visited on 02/16/2026).

[2] Mirza Mansoor Baig et al. “A Systematic Review of Wearable Sensors and IoT-Based Monitoring Applications for Older Adults – a Focus on Ageing Population and Independent Living”. en. In: *Journal of Medical Systems* 43.8 (June 2019), p. 233. issn: 1573-689X. doi: 10.1007/s10916-019-1365-7. url: https://doi.org/10.1007/s10916-019-1365-7 (visited on 02/16/2026).

[3] Javier Delahoz and Miguel A. Labrador. “A Survey of Fall Detection Systems: A Taxonomy, Challenges and Future Directions”. In: *Sensors* 14.7 (2014), pp. 12900–12926.

[4] Brittany Williamson et al. “Tracking down a solution: exploring the acceptability and value of wearable GPS devices for older persons, individuals with a disability and their support persons”. In: *Disability and Rehabilitation: Assistive Technology* 12.8 (Nov. 2017). _eprint: https://doi.org/10.1080/17483107.2016.1272140, pp. 822–831. issn: 1748-3107. doi: 10.1080/17483107.2016.1272140. url: https://doi.org/10.1080/17483107.2016.1272140 (visited on 02/16/2026).

[5] Archangel Cloud. *A brief history of social alarm and telecare services in the UK*. https://www.archangel.cloud/resource/history-telecare-and-social-alarms. Accessed: 2024-02-23. 2024.

[6] G. Williams et al. “A combined sensor system for detecting falls in the elderly”. In: *Proceedings of the 20th Annual International Conference of the IEEE Engineering in Medicine and Biology Society*. Vol. 3. 1998, pp. 1138–1141.

[7] Jae Min Kang et al. “A Wrist-Worn Integrated Health Monitoring Instrument with a Tele-Reporting Device for Telemedicine and Telecare”. In: *IEEE Transactions on Information Technology in Biomedicine* 10.4 (2006), pp. 657–664. doi: 10.1109/TITB.2006.881035.

[8] Ilya Katlinsky. *History of the Internet of Things: Key Milestones and Trends*. 2025. url: https://www.itransition.com/iot/history.

[9] Counterpoint Research. *Global Smartwatch Model Tracker: 2026 Analysis*. 2026.

[10] Ryan Molloy and Amie Clark. *The 4 Best Fall Detection Devices in 2026, Tested by Caregivers*. 2026. url: https://www.theseniorlist.com/medical-alert-systems/best/fall-detection/.

[11] Sequans Communications. *What is Cellular IoT?* 2025. url: https://sequans.com/what-is-cellular-iot/.

[12] SkyQuest. *5G Redcap Technology Market Size, Share, Trends & Forecast 2032*. Tech. rep. SkyQuest Technology Consulting, 2026. url: https://www.skyquestt.com/report/5g-redcap-technology-market.

[13] Chunyan Li et al. “A review of IoT applications in healthcare”. In: *Neurocomputing* 565 (2024), p. 127017. issn: 0925-2312. doi: 10.1016/j.neucom.2023.127017.

[14] Faheem Zafari, Athanasios Gkelias, and Kin K. Leung. “A Survey of Indoor Localization Systems and Technologies”. In: *IEEE Communications Surveys & Tutorials* 21.3 (2019), pp. 2568–2599. doi: 10.1109/COMST.2019.2911558.

[15] Global GPS Systems. *The Difference Between GNSS and GPS Explained*. https://globalgpssystems.com/gnss/the-difference-between-gnss-and-gps-explained/. 2024.
[16] Christopher J. Hegarty and Eric Chatre. "Evolution of the Global Navigation Satellite System (GNSS)". In: *Proceedings of the IEEE* 96.12 (2008), pp. 1902–1917. doi: 10.1109/JPROC.2008.2006090.
[17] Delve. *IoT Geolocation: How to Choose the Best Technology for Your Device*. https://www.delve.com/insights/iot-geolocation-how-to-choose-the-best-technology-for-your-device. 2024.
[18] P. Praveen and K. Pranay Kumar. "Wearable Location Tracker During Disaster using AI and IoT". In: *2024 3rd International Conference on Applied Artificial Intelligence and Computing (ICAAIC)*. 2024, pp. 208–213. doi: 10.1109/ICAAIC60222.2024.10575619.
[19] Kais Mekki et al. "Overview of Cellular LPWAN Technologies for IoT Deployment: Sigfox, LoRaWAN, and NB-IoT". In: *2018 IEEE International Conference on Pervasive Computing and Communications Workshops (PerCom Workshops)*. 2018, pp. 197–202. doi: 10.1109/PERCOMW.2018.8480255.
[20] Huajiang Ruan et al. "An Overview of LoRa Localization Technologies". In: *Computers, Materials & Continua* 82.2 (2025), pp. 1645–1680. issn: 1546-2226. doi: 10.32604/cmc.2024.059746. url: http://www.techscience.com/cmc/v82n2/59507.
[21] Sebastian Sadowski and Petros Spachos. "RSSI-Based Indoor Localization With the Internet of Things". In: *IEEE Access* 6 (2018), pp. 30149–30161. doi: 10.1109/ACCESS.2018.2843325.
[22] E. Mok and G. Retscher. "Location determination using WiFi fingerprinting versus WiFi trilateration". In: *Journal of Location Based Services* 1.2 (2007), pp. 145–159. doi: 10.1080/17489720701781905.
[23] Sujata Mohanty, Aruna Tripathy, and Bikramaditya Das. "An overview of a low energy UWB localization in IoT based system". In: *2021 International Symposium of Asian Control Association on Intelligent Robotics and Industrial Automation (IRIA)*. 2021, pp. 293–296. doi: 10.1109/IRIA53009.2021.9588718.
[24] Anh-Van Vu et al. "Real-time Pedestrian Dead Reckoning For IoT Based Platform-Independent Positioning System". In: *2024 IEEE International Conference on Systems, Man, and Cybernetics (SMC)*. 2024, pp. 2599–2606. doi: 10.1109/SMC54092.2024.10831327.
[25] Lu Bai et al. "Upper Limb Position Tracking with a Single Inertial Sensor Using Dead Reckoning Method with Drift Correction Techniques". In: *Sensors* 23.1 (2022), p. 360. doi: 10.3390/s23010360.
[26] C.-Y. Lin et al. "A CNN-SPEED-BASED GNSS/PDR INTEGRATED SYSTEM FOR SMARTWATCH". In: *The International Archives of the Photogrammetry, Remote Sensing and Spatial Information Sciences* XLVIII-1/W1-2023 (2023), pp. 235–241. doi: 10.5194/isprs-archives-XLVIII-1-W1-2023-235-2023.
[27] Pothuri Varma and Veena Anand. "Intelligent scanning period dilation based Wi-Fi fingerprinting for energy efficient indoor positioning in IoT applications". In: *The Journal of Supercomputing* 79 (Dec. 2022). doi: 10.1007/s11227-022-04980-9.
[28] IoT For All. *What's the Difference Between Private and Public LoRaWAN?* Accessed: 2026-03-05. 2024. url: https://www.iotforall.com/difference-private-public-lorawan.
[29] Coursera. *LoRaWAN vs. NB-IoT: What's the Difference?* Accessed: 2026-03-05. 2025. url: https://www.coursera.org/articles/lorawan-vs-nb-iot.
[30] Actility. *Understanding Duty Cycle in LoRaWAN*. Accessed: 2026-03-05. 2024. url: https://www.actility.com/understanding-duty-cycle-lorawan/.

[31] RF Wireless World. *5G NR RedCap vs LTE-M vs NB-IoT: Key differences*. Accessed: 2026-03-05. 2025. url: https://www.rfwireless-world.com/terminology/difference-between-5g-nr-redcap-lte-m-nb-iot.
[32] Onomondo. *NB-IoT vs LTE-M: A comparison of the two IoT technologies*. Accessed: 2026-03-05. 2023. url: https://onomondo.com/blog/nb-iot-vs-lte-m-a-comparison-of-the-two-iot-technology-standards/.
[33] VectorNav Technologies. *MEMS Accelerometers and Gyroscopes*. Accessed: 2026-02-23. 2026. url: https://www.vectornav.com/resources/inertial-navigation-primer/theory-of-operation/theory-mems.
[34] Dave Dlugos. *What is MEMS Pressure Sensor Technology?* Acessed:2026-02-23. Ashcroft. Mar. 2025. url: https://blog.ashcroft.com/blog/mems-pressure-sensor-technology.
[35] Henry Friday Nweke et al. "Deep learning algorithms for human activity recognition using mobile and wearable sensor networks: State of the art and research challenges". In: *Expert Systems with Applications* 105 (2018), pp. 233–261.
[36] Robert David et al. "TensorFlow Lite Micro: Embedded machine learning for TinyML systems". In: *Proceedings of Machine Learning and Systems* 3 (2021), pp. 800–811.
[37] Particle Industries. *Low Power IoT: A Comprehensive Guide to Power Management*. Accessed: 2026-03-08. 2023. url: https://www.particle.io/iot-guides-and-resources/low-power-iot/ (visited on 03/08/2026).
[38] T. M. Sathish Kumar. "Low-Power Design Techniques for Internet of Things (IoT) Devices: Current Trends and Future Directions". en. In: *Progress in Electronics and Communication Engineering* 1.1 (June 2024), pp. 19–25. issn: 3048-7625. doi: 10.31838/ECE/01.01.04. url: https://ecejournals.in/index.php/PECE/article/view/6 (visited on 03/06/2026).
[39] Alexandros Pantelopoulos and Nikolaos G. Bourbakis. "A Survey on Wearable Sensor-Based Systems for Health Monitoring and Prognosis". In: *IEEE Transactions on Systems, Man, and Cybernetics, Part C (Applications and Reviews)* 40.1 (2010), pp. 1–12. doi: 10.1109/TSMCC.2009.2032660.
[40] Rodrigo Roman, Pablo Najera, and Javier Lopez. "Securing the Internet of Things". In: *Computer* 44.9 (2011), pp. 51–58. doi: 10.1109/MC.2011.291.
[41] S. M. Riazul Islam et al. "The Internet of Things for Health Care: A Comprehensive Survey". In: *IEEE Access* 3 (2015), pp. 678–708. doi: 10.1109/ACCESS.2015.2437925.
[42] GeeksforGeeks. *What is Transmission Control Protocol (TCP)?* https://www.geeksforgeeks.org/computer-networks/what-is-transmission-control-protocol-tcp/. Accessed: 2026-02-19. 2024.
[43] Amazon Web Services. *What is MQTT?* https://aws.amazon.com/what-is/mqtt/. Accessed: 2026-02-19. n.d.
[44] Ala Al-Fuqaha et al. "Internet of Things: A Survey on Enabling Technologies, Protocols, and Applications". In: *IEEE Communications Surveys & Tutorials* 17.4 (2015), pp. 2347–2376.
[45] Qian Zhang and Min Chen. *Cloud Computing: Concepts, Technology & Architecture*. Springer, 2018.
[46] Sam Newman. *Monolith to Microservices*. O'Reilly, 2021.
[47] Sam Newman. *Building Microservices*. O'Reilly, 2021.
[48] Claus Pahl and Pooyan Jamshidi. "Microservices: A Systematic Mapping Study". In: *IEEE Cloud Computing* (2019).
[49] Roy Fielding. "Architectural Styles and the Design of Network-based Software Architectures". PhD thesis. University of California, Irvine, 2000.

[50] Ian Fette and Alexey Melnikov. *The WebSocket Protocol*. IETF RFC 6455. 2011.
[51] Marc Lorenz. “Design and Development of a Wearable Device for Fall Detection and On-Demand Position Tracking of Elderly People”. Student ID: s232813. Master Thesis. Kongens Lyngby, Denmark: Technical University of Denmark, Mar. 2025.
[52] Muhammad Shafiq et al. “IoT protocol comparison: MQTT vs CoAP for constrained devices”. In: *IEEE Access* 8 (2020), pp. 23788–23807.
[53] CalculatorHub. *Antenna Wavelength Calculator: Frequency to Antenna Length*. Accessed: 2026-07-22. 2026. url: https://calculatorhub.com/tools/antenna-wavelength-calculator/.
[54] Nordic Semiconductor. *nRF9151 Downloads*. https://www.nordicsemi.com/Products/nRF9151/Download?lang=en. Accessed: 2026-07-23.

A Impedance Calculations

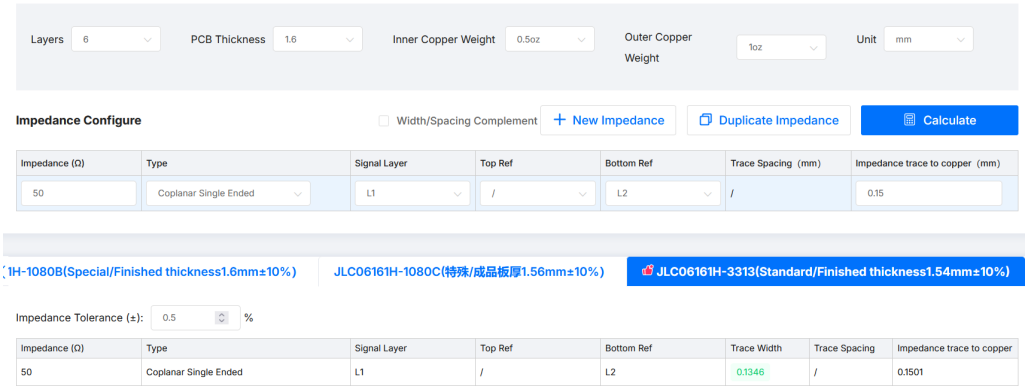


Figure A.1: 50 Ω coplanar waveguide impedance calculation for Layer 2

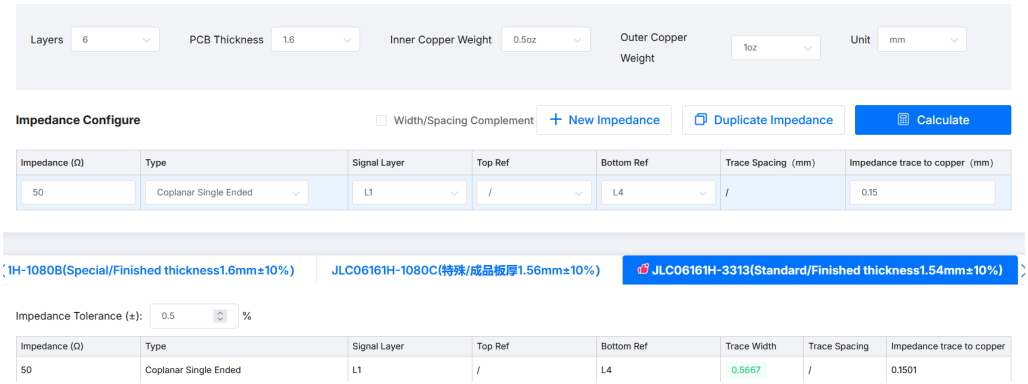


Figure A.2: 50 Ω coplanar waveguide impedance calculation for Layer 4

B Bill of Materials

The complete Bill of Materials, detailing all passive and active components utilized in the hardware prototype, is provided in Table B.1.

Table B.1: Prototype Bill of Materials (BOM)

Name	Description	Designator	Footprint	Qty	LCSC
1.0µF	X5R 25V 1µF ±10% 0603	C1	CAPC1608X06L	1	C15849
10µF	Capacitor, X5R, ±20%, 25V	C2, C3, C5, C7, C8, C9, C10, C12, C13, C14, C23	CAPC1608X06L	11	C19702
1.0nF	1nF 25V X7R ±10% 0201	C4, C6	CAPC0603X03L_C	2	C76943
2.2µF	Capacitor, Ceramic, 2.2µF 10V	C11, C32, C43	CAPC1608X06L	3	C23630
100nF	Capacitor, Ceramic, X5R, ±10%	C15, C16, C17, C21, C25, C26, C28, C31, C33, C38, C60, C61, C62, C63	CAPC0603X03L_C	14	C2649540
10nF	Capacitor, X7R, ±10%	C18, C42	CAPC0603X03L_C	2	C57112
39pF	Capacitor, NP0, ±1%, 25V	C19	CAPC0603X03L_C	1	C3011647
15pF	15pF 25V NP0 ±2% 0201	C20	CAPC0603X03L_C	1	C1853525
47µF	X5R 47µF 10V ±20% 0805	C22	CAPC2012X11L	1	C76636
100pF	25V 100pF C0G ±2% 0201	C24, C29	CAPC0603X03L_C	2	C710791
4.7µF	Capacitor, Ceramic, 4.7µF 25V	C27, C30, C39, C40, C41	CAPC1608X06L	6	C19666
1.0µF	Capacitor, Ceramic, 1.0µF 10V	C34, C36, C48, C49, C55	CAPC1005X04L	5	C52923
2.2µF	Capacitor, Ceramic, 2.2µF 25V	C35	CAPC1005X04L	1	C12530
0.47µF	Capacitor, Ceramic, 0.47µF	C37	CAPC0603X03L_C	1	C85926
22nF	X5R 10V 22nF ±10% 0201	C45	CAPC0603X03L_C	1	C161473
0.22µF	Capacitor, Ceramic, 0.22µF	C46	CAPC0603X03L_C	1	C76936

Continued on next page

Table B.1 – continued from previous page

Name	Description	Designator	Footprint	Qty	LCSC
2.2µF	Capacitor, Ceramic, 2.2µF 10V	C47	CAPC0603X03L_C	1	C335106
RC0603JR	0 Ohm Resistor (Placeholder)	C50, C51	RESC1608X55X25ML1	2	C21189
0.2pF	Capacitor	C52	RF_Pads	1	C76908
3.3pF	Capacitor	C53	RF_Pads	1	C88892
0.1pF	Capacitor	C54	RF_Pads	1	C161338
CC0201MR	CAP CER 1µF 6.3V X5R 0201	C56, C57	FP-CC0201-BA-MFG	2	C272877
CC0805KK	CAP CER 4.7µF 16V X7R 0805	C58	FP-CC0805-0_85-MFG	1	C1779
CC0603KR	CAP CER 2.2µF 16V X7R 0603	C59	FP-CC0603-DA-MFG	1	C23630
BAS40-04	Diode	D1	SOT96P240X115-3N	1	C85101
ESD441DPLR	ESD Protection Diode	D2	CAPC0603X03L_C	1	C52994072
TPD3F303	ESD Protection and EMI Filter	D3	SON50P210X160X65	1	C2865991
LFD_Diplexer	Signal Conditioning 2.4GHz	F1	CAF-50P160X80-6N	1	C389571
BLM21AG102	Ferrite Bead, 1k Ω @ 100MHz	FB1, FB2	INDC2012X11L	2	C85835
120R/0.7A	Ferrite Bead, 120 Ω @ 100MHz	FB3	CAPC1005X04L	1	C337873
USB-C 2.0	USB-C Receptacle, 16 Pin	J1	USB4110	1	C5143397
SF72S006	Position Card Connector	J4	SF72S006VBxR2500	1	C2977286
505110-3091	CONN FFC/FPC BOTTOM 30P	J7	FP-505110-3091-MFG	1	C588008
U.FL-R-SMT	CONN U.FL RCPT STR 50 OHM	J8	FP-U_FL-R-SMT-1_80	1	C88374
2.2µH	1.2A 2.2µH Multilayer inductor	L1	INDC2016X10N	1	C695595
9.1nH	540mA 9.1nH ±2%	L2	INDC1005X04L_WW	1	C113122
3.3µH	1A 3.3µH ±20% 1008	L3	INDC2520X10N	1	C703744
10nH	Inductor	L4, L6	RF_Pads	2	C668467
11nH	Inductor	L5	RF_Pads	1	C22410303
9.5nH	135mΩ 28@250MHz 5.4GHz	L8	RF_Pads	1	C346334
5.5nH	Inductor	L9	RF_Pads	1	C412351
DFE252012P	Inductor Power Shielded	L10	DFE252012P-1R5M	1	C161084
LED RGB	RGB LED 1.60 mm x 1.60 mm	LED2	LED_RGB_0606	1	C458749
CMT-322-65	Loudspeaker or Buzzer	LS1	CMT32265SMTTR	1	C7230967
nRF9151	Low power System-in-Package	M1	LGA50P1210X1110X1	1	C22397843
RV2C010UNT2L	20V 1A 470mΩ@4.5V	Q1	DFN65P100X40-3L	1	C570828
DMN2990UDJ	20V 450mA 350mW	Q2	SOTFL35P100X50-6N	1	C145188
BC817-40W-7	NPN Small Signal Transistor	Q3	DIOD-SOT323-3_V	1	C460176
FDN338P	P-Channel Power Transistor	Q4	FAIR-SSOT3L-3_V	1	C2890130

Continued on next page

Table B.1 – continued from previous page

Name	Description	Designator	Footprint	Qty	LCSC
FDN335N	N-Channel Power Transistor	Q5	FAIR-SSOT3L-3_V	1	C845588
47k	Resistor, ±1%, 0.05W	R1	RESC0603X03L_C	1	C270345
10k	50mW 10kΩ 25V ±200ppm	R2, R4, R15, R19	RESC0603X03L_C	4	C106225
CRCW020110K	Thick Film Resistor	R3	RESC0603X28X15LL0	1	C844303
100R	30V Thick Film Resistor 100Ω	R5	RESC0603X03L_C	1	C2073844
39R	±1% 50mW ±200ppm/°C	R6, R7	RESC0603X03L_C	2	C7467286
RC0402JR	Thick Film Resistor	R8	RESC1005X40X25LL0	1	C25104
1M0	1MΩ 50mW 25V Thick Film	R9, R10, R11, R16	RESC0603X03L_C	4	C473482
4k7	Resistor, ±1%, 0.05W	R12, R13	RESC0603X03L_C	2	C270346
0R	Resistor, ±5%, 0.1W	R14	CAPC1608X06L	1	C95177
1k	50mW 1kΩ 25V Thick Film	R17	RESC0603X03L_C	1	C270365
100k	100kΩ 50mW 25V ±200ppm	R18	RESC0603X03L_C	1	C106224
CRCW020147K	Thick Film Resistor	R21, R22	RESC0603X28X15LL0	2	C270345
CRCW02014K7	Thick Film Resistor	R23, R24	RESC0603X28X15LL0	2	C3000902
AC1206FR	RES SMD 620K Ω 1% 1/4W	R25	FP-AC1206-MFG	1	C4410
KMR233GLFG	32V Round Button SPST-NO	S1	KMR233GLFG	1	C2689601
Switch	SMD slide Switch (ON-ON)	SW1	Switch-Slide-RA-J	1	C2921660
Emergency	Keypad Switch, 1 Switch	SW2	Emergency_Button	1	C19724054
nPM1300	Li-Ion/Li-Polymer USB-C PMIC	U1	Nordic_QFN50P500X	1	C7501206
TPS63031DSKR	High Efficiency Buck-Boost	U2	TPS63031DSKR	1	C15516
BMI270	Accelerometer Gyroscope IMU	U3	BMI270	1	C2836813
MX25U6432F	NOR Flash 64M-bit Serial 1.8V	U4	MX25U6432FZBI02	1	C5820605
SKY65943-11	GNSS Low-Noise Amplifier	U5	SKY65943-11	1	C2654442
NRF7002-QFAA	Wi-fi Dual Band Companion	U6	QFN40P600X600X60	1	C5370744
ICP-20100	Barometric Pressure Sensor	U7	FP-ICP-20100-MFG	1	C5343527
40MHz	XTAL SMD 1612, 40MHz	X1	BT-XTAL_1612	1	C1985632